

PYTHON



django

UDO:

PUESTA EN MARCHA

Introducción a las herramientas
necesarias en la asignatura:
Desarrollo Web en Entorno Servidor

Olga M. Moreno Martín - Javier Herrero Porras

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

ÍNDICE

▶ **Instalación de Python**

▶ **Instalación de PyCharm**

▶ **PyCharm un nuevo IDE**

▶ **Trabajando con PyCharm**

▶ **Shortcuts en PyCharm**

▶ **Instalando paquetes**

▶ **Django**

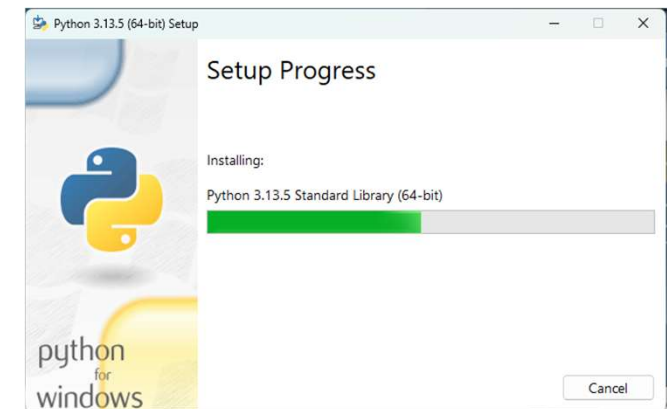
INSTALACIÓN DE PYTHON

Enlace a la instalación: <https://www.python.org/downloads/release/python-3135/>

¿Por qué la versión 3.13? Bueno porque es la última disponible y estable (ya está testeada y no contiene fallos de seguridad conocidos).

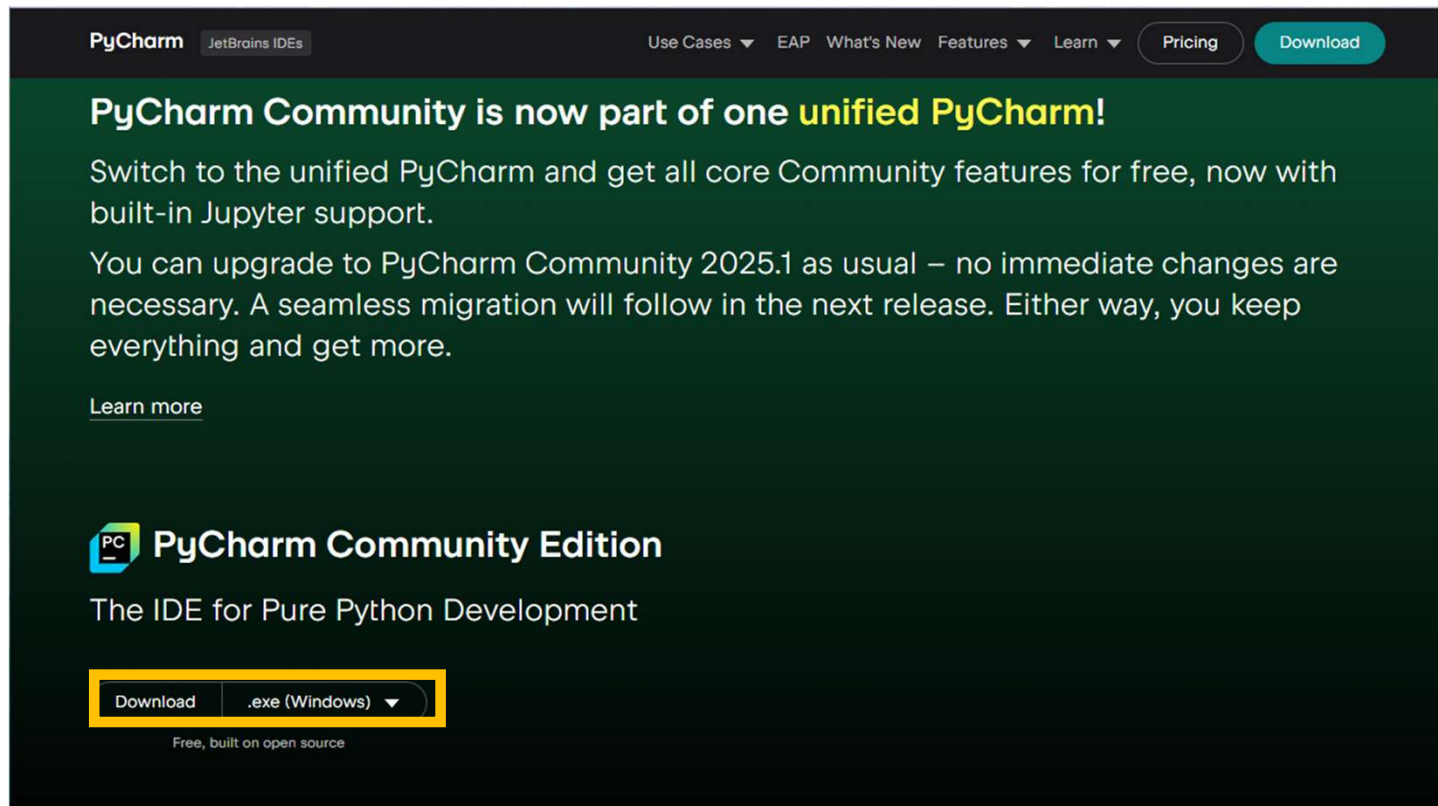
Versión publicada el 10-2024 y fin de soporte 10-2029.

A la larga queremos trabajar con software LTS (Long Term Support) para que nuestras aplicaciones puedan mantenerse muchos años, dado que la etapa del ciclo de vida del software más larga es el mantenimiento.



INSTALACIÓN DE PYCHARM COMMUNITY EDITION

<https://www.jetbrains.com/pycharm/download/>



PyCharm JetBrains IDEs Use Cases ▼ EAP What's New Features ▼ Learn ▼ Pricing Download

PyCharm Community is now part of one unified PyCharm!

Switch to the unified PyCharm and get all core Community features for free, now with built-in Jupyter support.

You can upgrade to PyCharm Community 2025.1 as usual – no immediate changes are necessary. A seamless migration will follow in the next release. Either way, you keep everything and get more.

[Learn more](#)

 **PyCharm Community Edition**

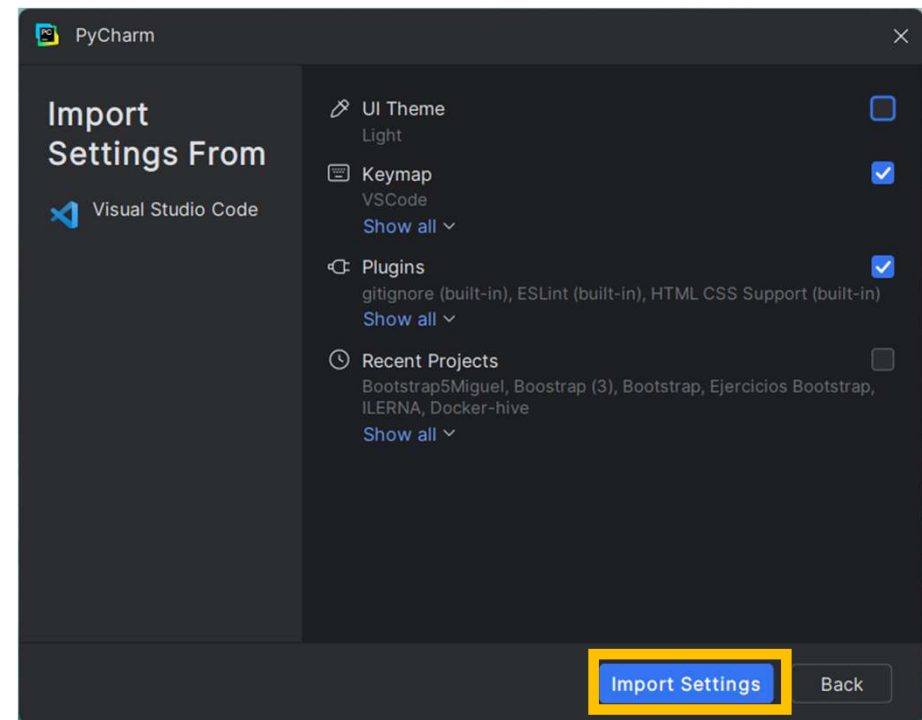
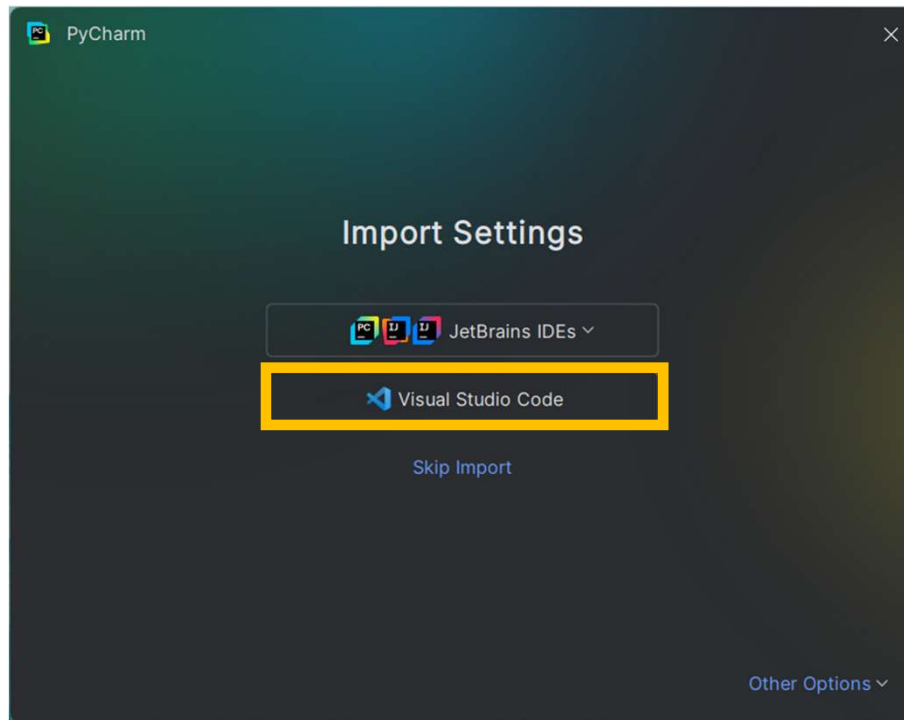
The IDE for Pure Python Development

Download .exe (Windows) ▼

Free, built on open source

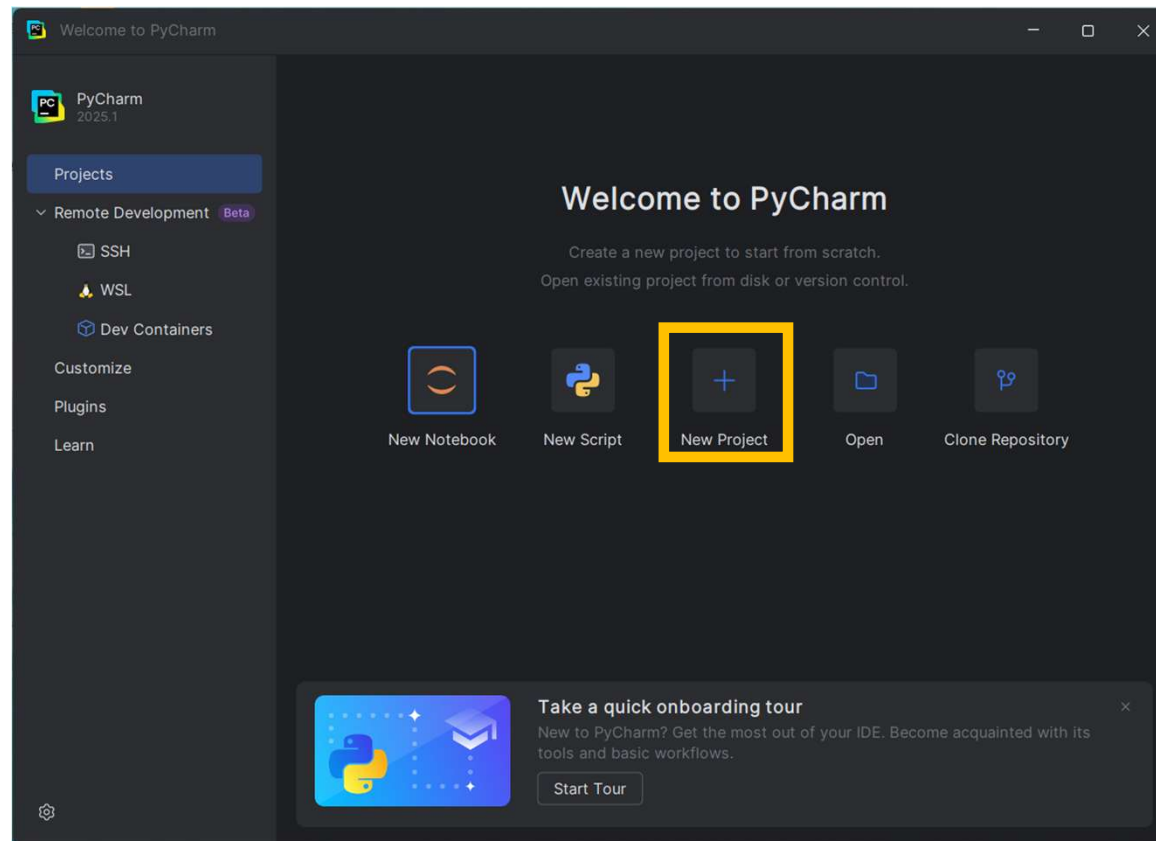
Olga M. Moreno Martín - Javier Herrero Porras

INSTALACIÓN DE PYCHARM COMMUNITY EDITION

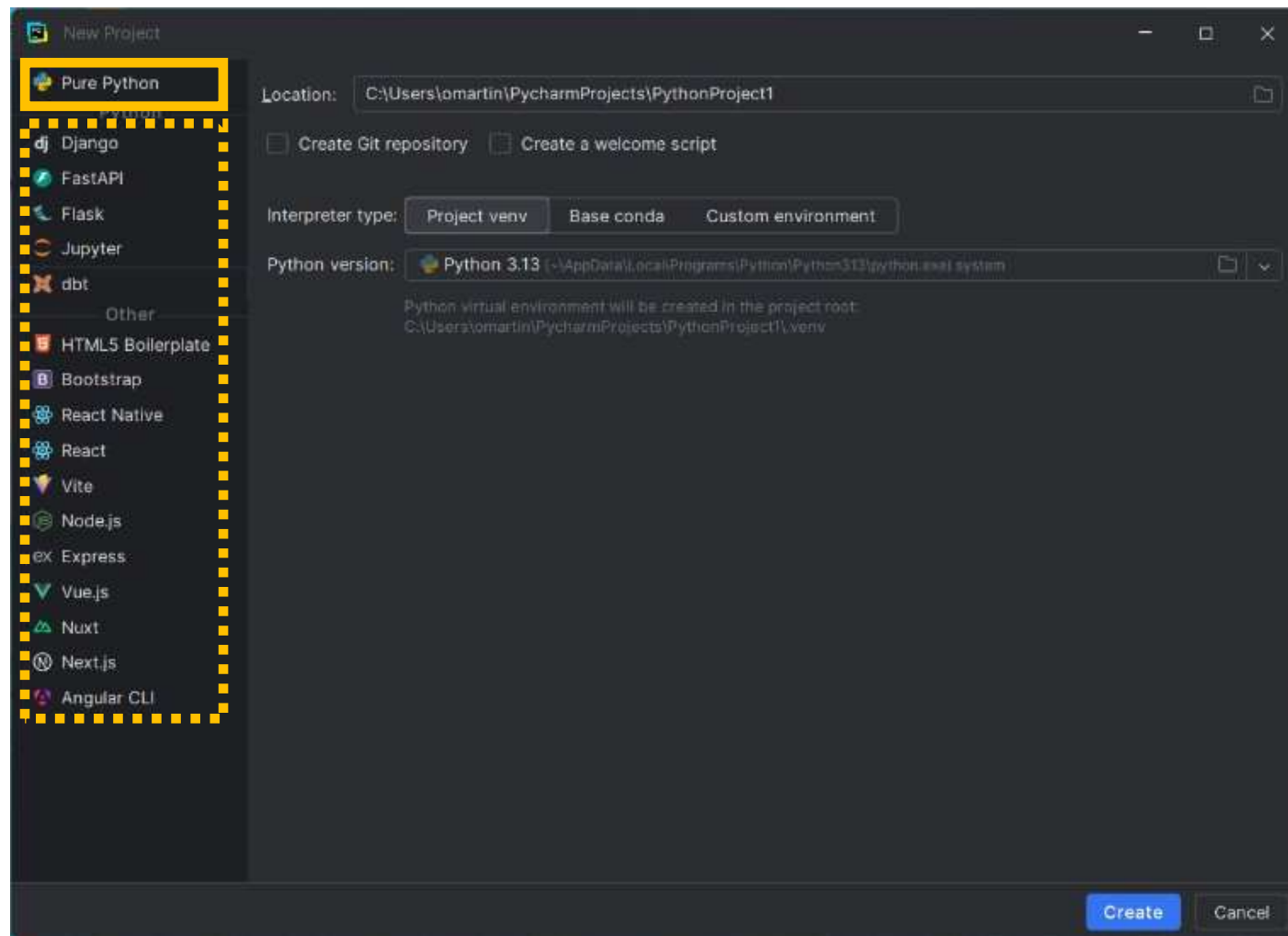


Este es un paso opcional, siempre podemos cambiar luego lo que necesitemos.

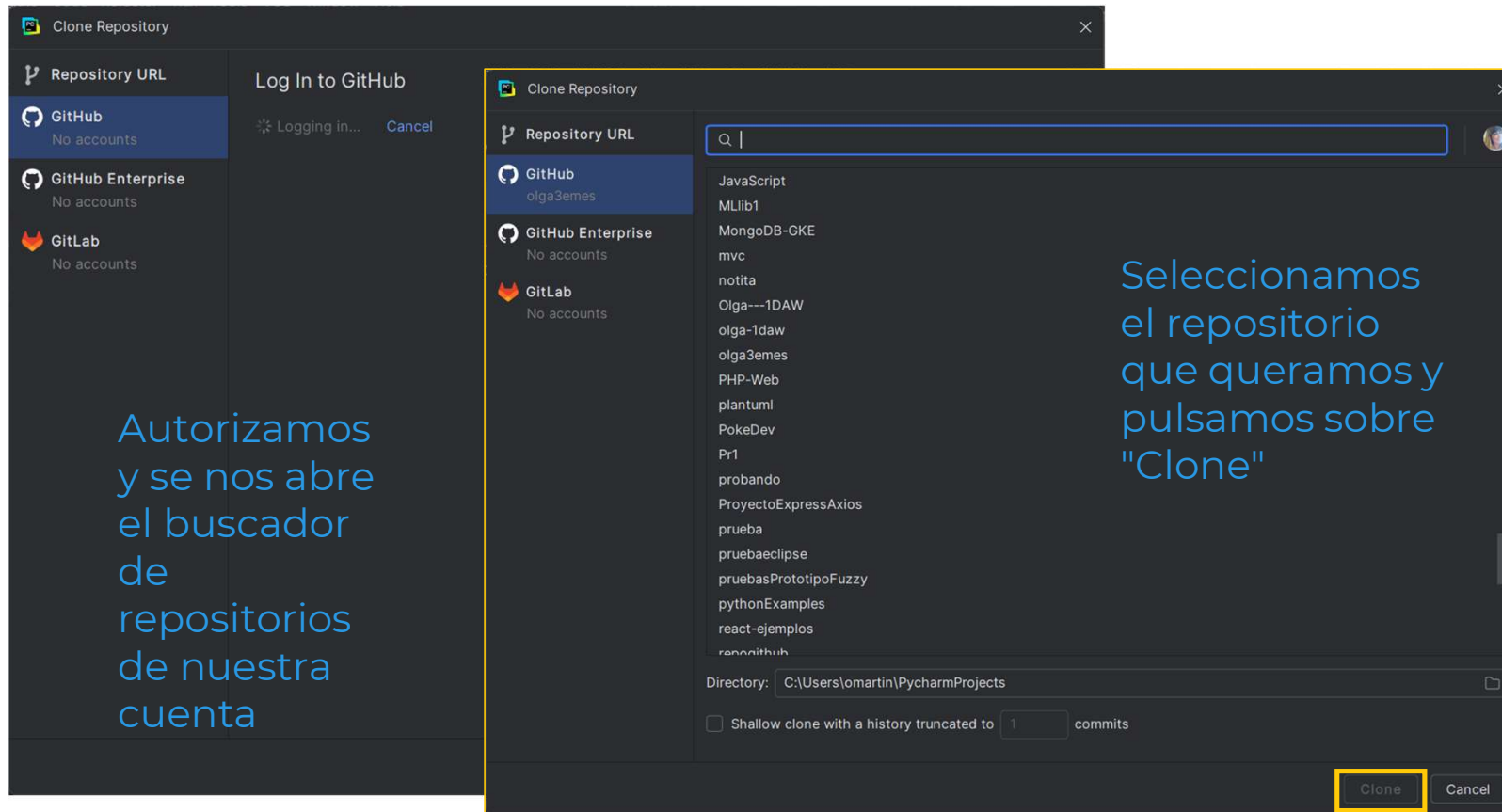
USO DE PYCHARM COMMUNITY EDITION



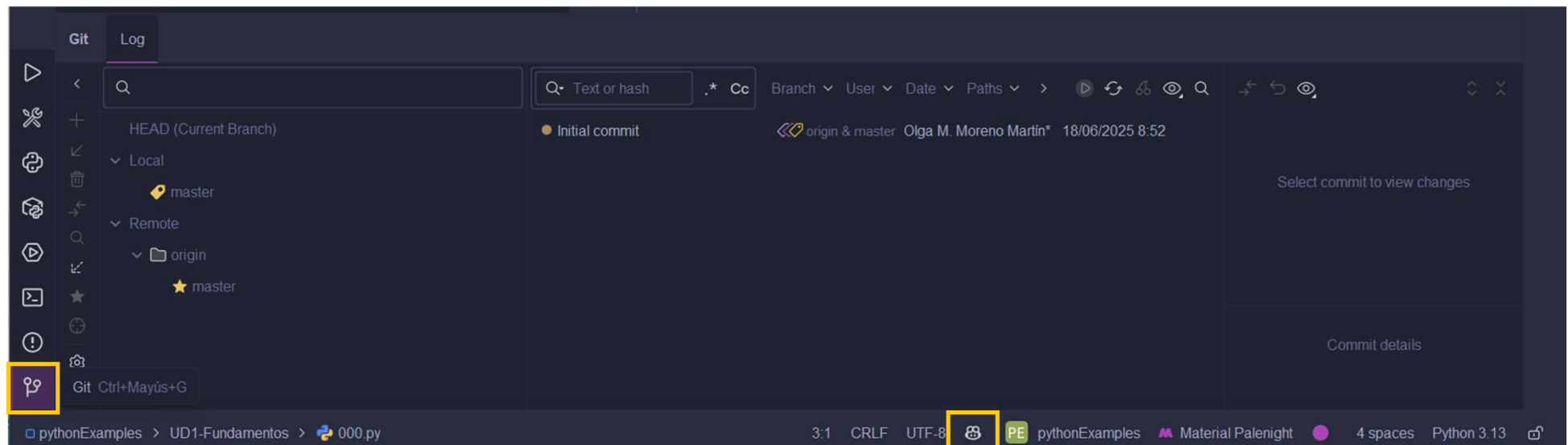
NEW PROJECT



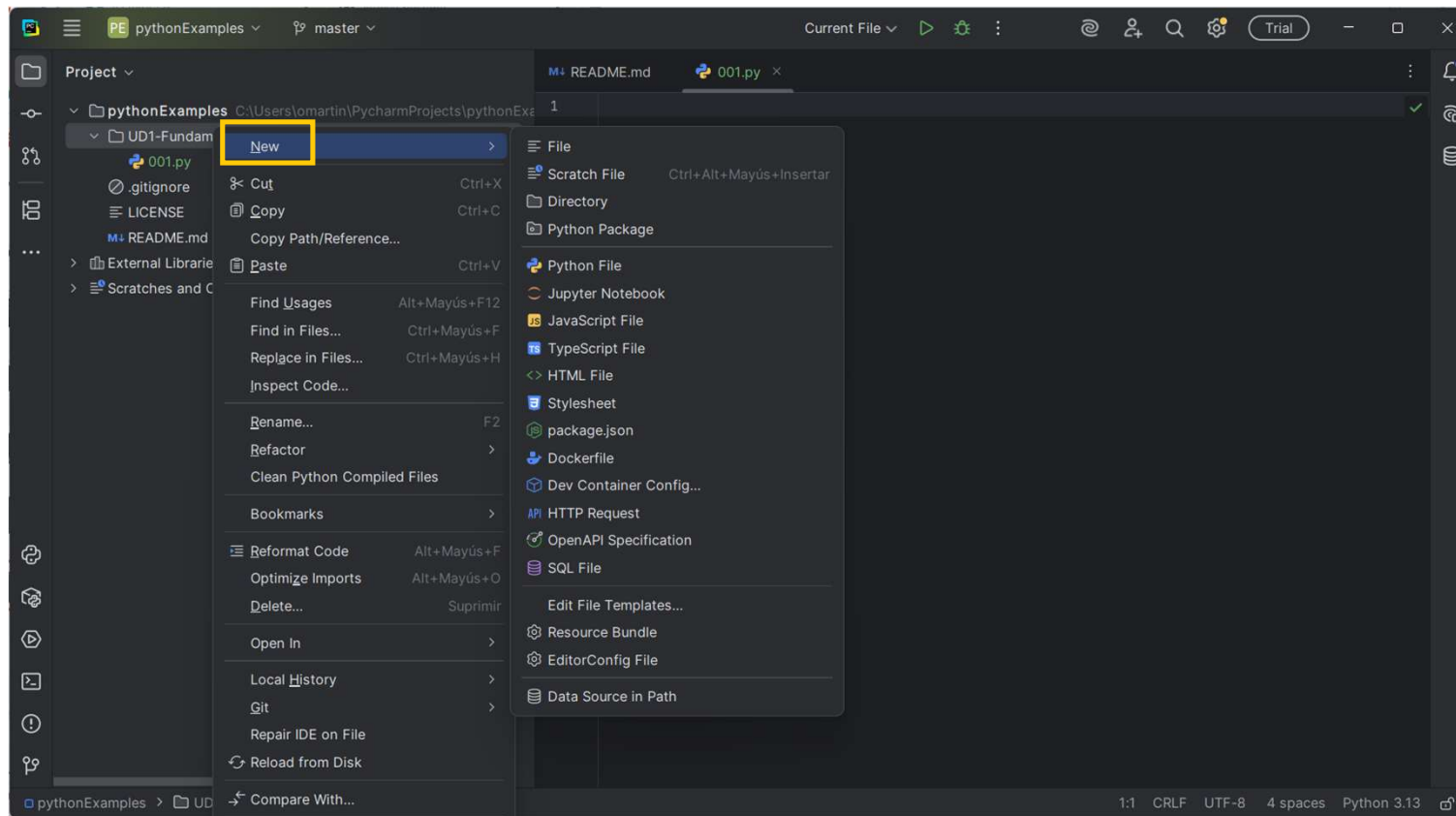
CLONE PROJECT



SINCRONIZACIÓN CON REPOSITORIOS GIT-GITHUB Y COPILOT



PODEMOS CREAR UN MONTÓN DE RECURSOS

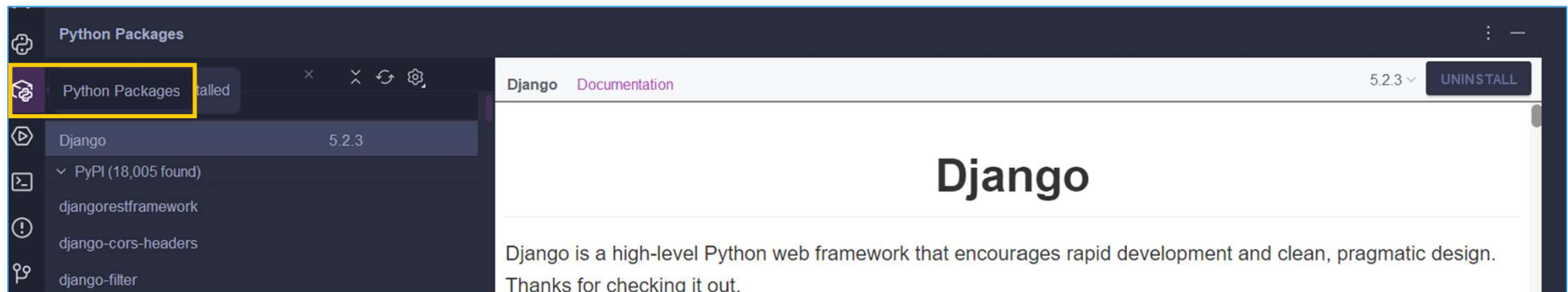


ATAJOS DE TECLADO

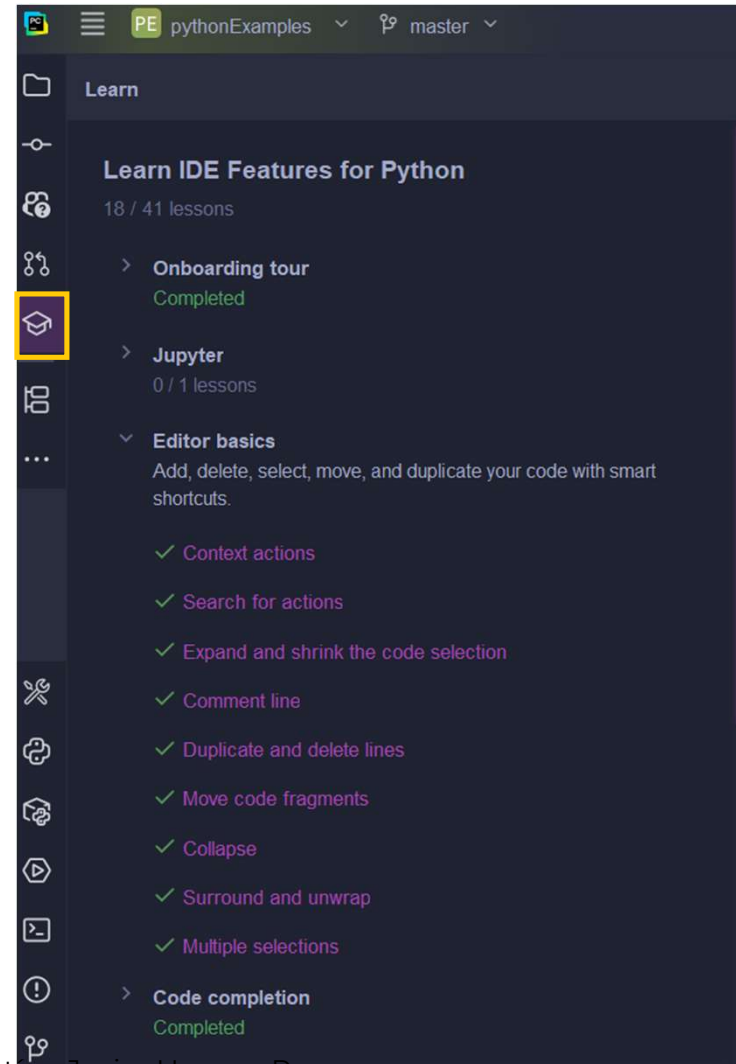
Descargar
PDF con
atajos para
Pycharm [aquí](#)

Atajo	Acción
Doble Shift	Buscar en todas partes Encuentra rápidamente cualquier archivo, acción, clase, símbolo, ventana de herramientas o configuración en PyCharm, en tu proyecto y en el repositorio Git actual.
Ctrl Shift A	Encuentra acción Busca un comando y ejecútalo, abre una ventana de herramientas o busca una configuración.
Ctrl R	Ejecutar/Repetir Ejecutar y volver a ejecutar aplicaciones completas, así como scripts específicos.
Doble Ctrl	Ejecutar cualquier cosa Ejecutar comandos, como abrir un proyecto, iniciar una configuración de ejecución/depuración, ejecutar una utilidad de línea de comandos, etc. Los comandos disponibles dependen del conjunto de complementos y herramientas que hayas configurado para tu proyecto.
Alt Enter	Mostrar acciones de contexto Soluciones rápidas para errores y advertencias destacados, acciones intencionadas para mejorar y optimizar el código.
F2 Shift F2	Navegar entre problemas de código Salta al error resaltado siguiente o anterior.
Ctrl E	Ver archivos recientes Selecciona un archivo abierto recientemente de la lista.
Ctrl W Ctrl Shift W	Ampliar o reducir la selección Aumente o disminuya el alcance de selección según construcciones de código específicas.
Ctrl O/ Ctrl Shift O/	Agregar/eliminar línea o bloquear comentario Comentar una línea o un bloque de código.
Alt F7	Buscar usos Muestra todos los lugares donde se utiliza un elemento de código en tu proyecto.

PARA INSTALAR PAQUETES, LIBRERÍAS Y FRAMEWORKS



**PARA CONOCER MÁS DE
NUESTRO NUEVO IDE
TENEMOS LA OPCIÓN DE
“LEARN”**



Olga M. Moreno Martín - Javier Herrero Porras

INSTALACIÓN DE DJANGO 5.2.3

Enlace a la instalación: <https://www.djangoproject.com/download/>

Con Django usaremos la versión 5.2 que hará uso de la versión 3.13 de Python.

Django tiene una instalación mediante comandos:

Mac/Linux:

```
python -m pip install Django==5.2.3
```

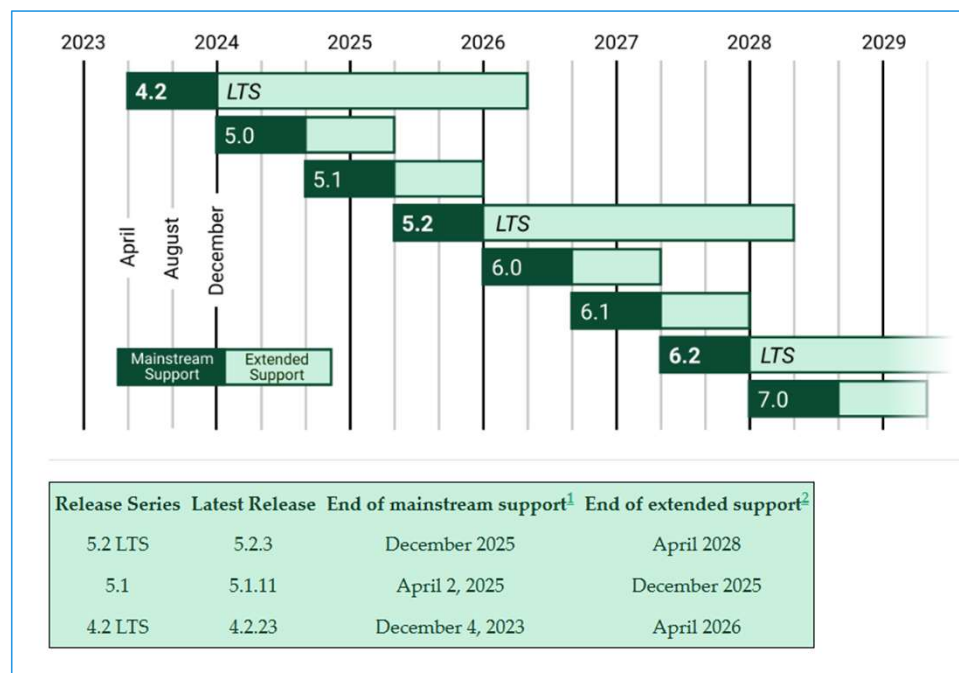
Windows:

```
py -m pip install Django==5.2.3
```

Lo utilizaremos más adelante, sin prisas.



The web framework for
perfectionists with
deadlines.



Olga M. Moreno Martín - Javier Herrero Porras

¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON



django

UD1:

PYTHON- FUNDAMENTOS

Fundamentos del Lenguaje Python

Olga M. Moreno Martín - Javier Herrero Porras

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

ÍNDICE

▶ **Python**

▶ **Primeros pasos**

▶ **Operadores**

▶ **Condicionales**

▶ **Bucles**

▶ **Tipos básicos**

▶ **La función type()**

▶ **Estructuras**

PYTHON - ¿DE DÓNDE HA SALIDO?

Guido Van Rossum, un neerlandés que estudió matemáticas y computación, se dedicó al desarrollo software, creó Python porque su oficina cerró durante las navidades y se aburría.

Python comenzó en **1989** en el Centrum Wiskunde & Informatica (CWI), en principio como un proyecto de afición para mantenerse ocupado durante las vacaciones de Navidad.

El nombre del lenguaje se inspiró en el programa de televisión de la BBC “**Monty Python’s Flying Circus**” debido a que Guido Van Rossum era un gran aficionado del programa.

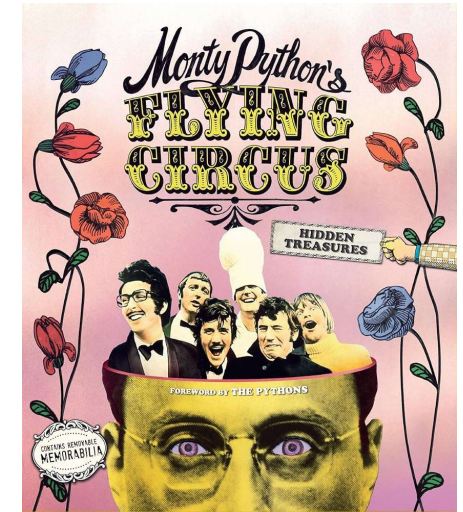
Guido Van Rossum publicó la [primera versión](#) del código Python (versión 0.9.0) en **1991**. Dicha versión ya incluía buenas características, como algunos tipos de datos y funciones para la gestión de errores.

Python 1.0 se lanzó en 1994 con nuevas funciones para procesar fácilmente una lista de datos, como la asignación, el filtrado y la reducción.

Python 2.0 se lanzó el 16 de octubre de 2000, con nuevas características útiles para los programadores, como la compatibilidad con los caracteres Unicode y una forma más corta de recorrer una lista.

El 3 de diciembre de 2008, se lanzó Python 3.0. Incluía características como la función de impresión y más soporte para la división de números y la gestión de errores.

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON - ¿QUIÉN LO USA?

Google utiliza ampliamente Python en sus sistemas de búsqueda web.

YouTube está escrito en gran parte en Python.

El servicio de almacenamiento Dropbox codifica tanto su software de servidor como el de cliente de escritorio principalmente en Python.

Raspberry Pi promueve Python como lenguaje educativo.

BitTorrent comenzó su vida como un programa Python.

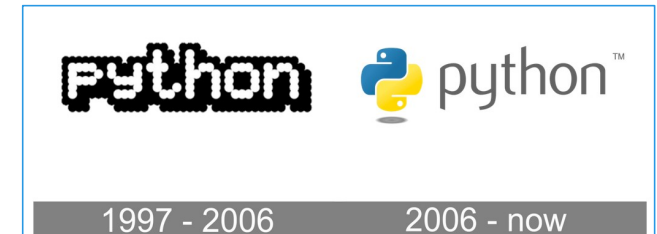
Tanto Netflix como Yelp han documentado el papel de Python en sus infraestructuras de software

Intel, Cisco, Hewlett-Packard, Seagate, Qualcomm e IBM utilizan Python para las pruebas de hardware.

Maya, un potente sistema integrado de modelado y animación 3D, proporciona una API de scripting en Python.

Industrial Light & Magic, Pixar y otros utilizan Python en la producción de películas de animación

Y por supuesto, es conocido como "el lenguaje de la inteligencia artificial" debido a su enorme cantidad de librerías, paquetes e integraciones y su papel en el desarrollo de los LLMs...



PYTHON - ¿PARA QUÉ SIRVE?

Python es un lenguaje de propósito general, es decir, no tiene un único ámbito de aplicación, es utilizado en desarrollo web, seguridad, ciencia de datos, inteligencia artificial, testing y automatización.

Python está presente en multitud de librerías y frameworks:

- Web: **Django**, Flask, FastAPI
- Ciencia de datos y ML: NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch
- Automatización: Selenium, Requests, BeautifulSoup
- Testing: Pytest, unittest
- Gestión de dependencias: pip, venv, pipenv

Su documentación oficial se puede encontrar en:

<https://docs.python.org/3/>



PYTHON - ¿CÓMO ES?

Python tiene un **tipado dinámico y fuerte**:

- **Dinámico**: No necesitamos declarar el tipo de variable, se determina en tiempo de ejecución.
- **Fuerte**: No se permiten operaciones entre tipos incompatibles sin una conversión explícita.

Lenguaje **multiparadigma**:

- Fundamentalmente se usa con orientación a objetos, pero también admite programación funcional y estructurada (imperativa)

Derivado de Perl:

- Intentando eliminar sus características no deseables

Tiene como objetivo conseguir una **sintaxis limpia** y **código legible**.

- Cosa que Perl no consigue.
- Intenta asimilar la sintaxis al pseudocódigo

Multiplataforma y OpenSource (código abierto).

- Licencia propia pero compatible con la GNU.

Interpretado

- Se sacrifica rendimiento por flexibilidad.
- Para que el rendimiento no se vea muy afectado usa bytecode

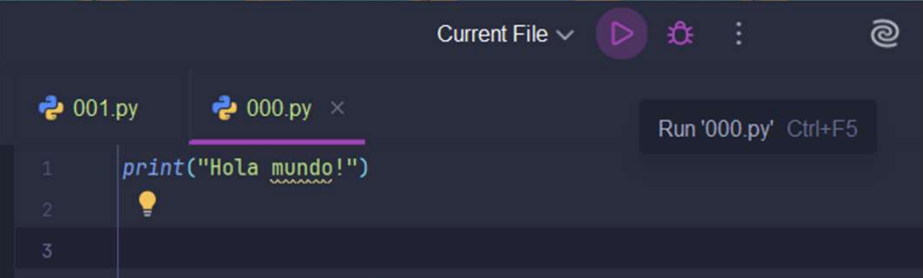
En Python, el **bytecode** es una representación de bajo nivel del código fuente que se genera cuando se compila un programa. No es código máquina directamente ejecutable por el hardware, sino que es interpretado por la máquina virtual de Python (CPython). Actúa como un lenguaje intermedio entre el código fuente y la máquina, permitiendo que Python sea multiplataforma.

PRIMEROS PASOS: HOLA MUNDO

Vamos a dar nuestros primeros pasos. Para ello podemos usar la línea de comandos de Python, que es una alternativa fácil y rápida, aunque también podemos escribir contenido en un fichero y después ejecutarlo.

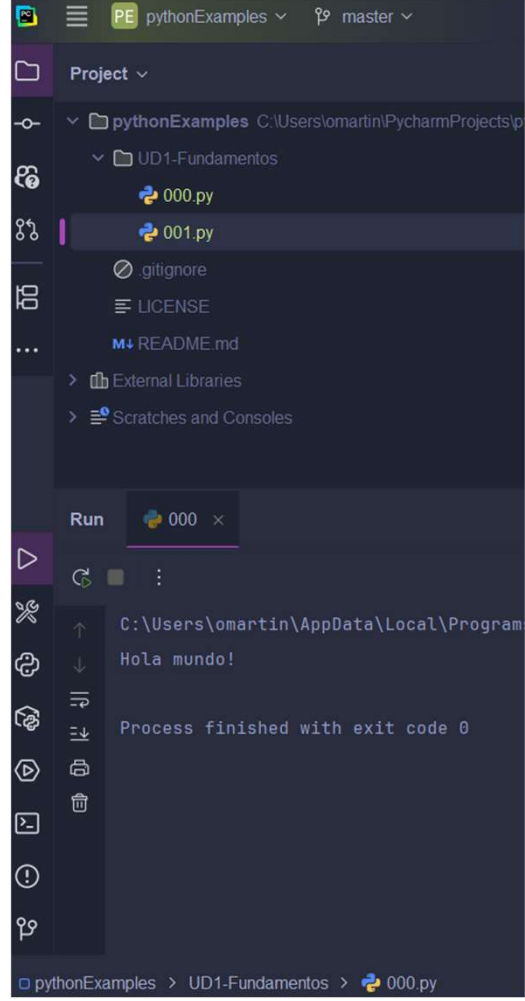
Para ejecutar código: Intérprete: *python* Fichero: *python nombreFichero.py*

Probamos el siguiente comando:



```
1 print("Hola mundo!")
2
3
```

Run '000.py' Ctrl+F5



Project

- pythonExamples C:\Users\omartin\PycharmProjects\p
 - UD1-Fundamentos
 - 000.py
 - 001.py
 - .gitignore
 - LICENSE
 - README.md
 - External Libraries
 - Scratches and Consoles

Run 000 x

C:\Users\omartin\AppData\Local\Program

Hola mundo!

Process finished with exit code 0

pythonExamples > UD1-Fundamentos > 000.py

PRIMEROS PASOS: COMENTARIOS Y ERRORES

Si hay un error de ejecución, se produce un Error.

```
001.py
#La división por cero no está permitida en Python
5/0
#Y tenemos que tener cuidado con las variables que no están definidas
n
#Para evitar errores, podemos usar try y except, que nos permite manejar excepciones
#Lo veremos más adelante...

''' Los comentarios multilínea necesitan tres comillas simples o dobles'''
```

! 1 ! 1 ✓ 20 ^

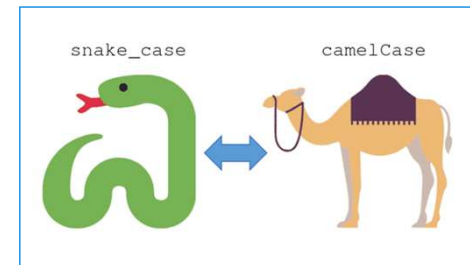
El IDE nos va a ayudar a detectar errores

PRIMEROS PASOS: VARIABLES

- Las variables en Python se escriben en **snake_case**
- No es necesario declararlas previamente. Su valor inicial es None.
- Su nombre puede comenzar con una letra o un guión bajo `_`. **No puede empezar por un número y solo pueden estar formadas por caracteres alfanuméricos y guion bajo (a-Z, 0-9 y `_`).**
- Son **case sensitive**: `var != Var`
- No se declara su tipo, el tipado es dinámico. Se asigna en tiempo de ejecución dependiendo del valor asignado. También puede cambiar (al principio puede almacenar un entero y después almacenar un string).

Existen distintos tipos de datos. Una variable puede almacenar: *str*, *int*, *float*, *list*, *tuple*, *range*, *dict*, *set*, *bool*, *bytes*, etc. Podemos saber el tipo de dato de una variable con la función `type()`.

(Más información en https://www.w3schools.com/python/python_datatypes.asp y <https://docs.python.org/3/library/stdtypes.html>).



```
002.py
#Variables con identificadores válidos
nombre = "Juan"
edad = 30
_indice = 5.5 #ERROR
01nombre = "Ana" #ERROR
.edad = 25 #ERROR
```

PRIMEROS PASOS: CONSTANTES

No existe una forma de definir una variable como constante.

- Una forma aceptada es la de escribir la variable en mayúscula con el guión bajo separando las palabras.
 - Por ejemplo: MAX_OVERFLOW y TOTAL
- No podemos asegurar que el valor de la variable cambie durante la ejecución del programa, aunque podemos utilizar "trucos".

003.py

```
MAX_VALUE =5
```

```
MIN_VALUE = 1
```

```
print(MAX_VALUE)
```

```
print(MIN_VALUE)
```

OPERADORES: ARITMÉTICOS

Ejemplo	Nombre	Resultado
-a	Negación	Opuesto de a
a + b	Suma	Suma de a y b
a - b	Resta	Diferencia de a y b
a * b	Multiplicación	Producto de a y b
a / b	División	Cociente de a y b
a % b	Módulo/ Resto	Resto de a dividido por b
a // b	División entera (floor division)	Cociente entero de a y b
a ** b	Potencia	Resultado de a elevado a b.

En el caso de cadenas, si queremos concatenarlas, utilizaremos el operador **+**

OPERADORES: LÓGICOS

Ejemplo	Nombre	Resultado
a and b	And (y)	true si tanto a como b son true
a or b	Or (o inclusivo)	true si cualquiera de a o b es true
not a	Not (no)	true si a no es true

OPERADORES: ASIGNACIÓN

Ejemplo	Nombre	Resultado
<code>a = b</code>	Asignación	Asigna a a el valor de b
<code>a += b</code>	Asignación de la suma	Le suma a a el valor de b. Equivalente a <code>a = a + b</code> . También se utiliza para concatenar la cadena b a la cadena a.
<code>a -= b</code>	Asignación de la resta	Le resta a a el valor de b. Equivalente a <code>a = a - b</code>
<code>a *= b</code>	Asignación del producto	Asigna a a el producto de a por b. Equivalente a <code>a = a * b</code>
<code>a /= b</code>	Asignación de la división	Asigna a a el cociente de a entre b. Equivalente a <code>a = a / b</code>
<code>a %= b</code>	Asignación del resto	Asigna a a el resto de dividir a entre b. Equivalente a <code>a = a % b</code>
<code>x, y = "Hola", "Mundo"</code>	Asignación de múltiples variables	Permite asignar valores a varias variables en una sola línea
<code>x, y = [1, 2]</code>	Unpacking	Permite extraer los valores de una lista, tupla, etc en varias variables.

Prioridad de los operadores

Recuerda la prioridad. Primero los paréntesis, luego exponenciación, productos/divisiones, sumas/restas, comparaciones, lógicos y por último se realiza la asignación. Más información en <https://docs.python.org/3/reference/expressions.html>

OPERADORES: COMPARACIÓN

Ejemplo	Nombre	Resultado
<code>a == b</code>	Igual	true si a es igual a b (Además deben ser el mismo tipo de dato)
<code>a != b</code>	Diferente	true si a no es igual a b
<code>a < b</code>	Menor que	true si a es estrictamente menor que b
<code>a > b</code>	Mayor que	true si a es estrictamente mayor que b
<code>a <= b</code>	Menor o igual que	true si a es menor o igual que b
<code>a >= b</code>	Mayor o igual que	true si a es mayor o igual que b

OPERADORES: PERTENENCIA

Ejemplo	Nombre
<code>a in [0, 5, 7]</code>	Devuelve verdadero si el valor de <i>a</i> se encuentra en el objeto, en este caso, una lista.
<code>a not in ["Python", "is", "fun"]</code>	Devuelve verdadero si el valor de <i>a</i> no se encuentra en el objeto, en este caso, una lista.

CONDICIONALES

La condición simple se realiza mediante la instrucción `if`. Observad que no se utilizan paréntesis cuando se escribe la condición ni tampoco llaves para delimitar el código dentro de cada bloque. En su lugar se utiliza una **indentación (o sangría)** de 4 espacios.

```
004.py
#Ejemplo de if-else-elif
x=42

if x < 0:
    x=0
    print('Negativo cambiado a cero')
elif x == 0:
    print('Cero')
elif x == 1:
    print('Uno')
else:
    print('Más grande que uno')
```

Olga M. Moreno Martín - Javier Herrero Porras

CONDICIONALES

El lenguaje también cuenta con una forma abreviada para escribir la sentencia `if`.

Además, existe una forma de abreviar la estructura `if-else`. Se utiliza comúnmente cuando solamente hay una instrucción dentro de cada bloque.

```
005.py
# Ejemplo de if simple
nombre = 'Juan'
if nombre: print('Hay un nombre definido:' + nombre)

# Ejemplo de if-else
edad = 18
mensaje = 'Eres mayor de edad' if edad >= 18 else 'Eres menor de edad'
print(mensaje)
```

CONDICIONALES

```
006.py
x=4
match x:
    case 0:
        print('Cero')
    case 1:
        print('Uno')
    case 2 | 3:
        print('Dos o tres')
    case _ if x < 0:
        print('Negativo')
    case _:
        print('Más grande que tres')
```

La sentencia **match** también permite trabajar con condiciones múltiples. La expresión se evalúa una vez y termina si encuentra una coincidencia, por lo que no se necesita de la instrucción `break`.

💡 El último case (que utiliza el guión bajo `_`) funciona como comodín y se ejecuta cuando no se producen coincidencias en ninguno de los case anteriores.

BUCLES

Los bucles se utilizan para ejecutar de forma repetida un conjunto de expresiones mientras una condición se cumpla. En el caso de `for`, se utiliza para iterar sobre los ítems de una secuencia (lista o cadena de texto), en el orden en el que aparecen en la secuencia.

 **Investiga:**

¿Cómo funciona `range()`? ¿Qué opciones tiene?

```
007.py

#BUCLES

# Ejemplo de bucle while
i=1
while i <= 5:
    print(i)
    i += 1

# Ejemplo de bucle for
for j in range(1, 6):
    print(j)

# Ejemplo de bucle for con lista
lista = ['manzana', 'banana', 'cereza']
for fruta in lista:
    print(fruta)

# Ejemplo de bucle for con diccionario
diccionario = {'a': 1, 'b': 2, 'c': 3}
for clave, valor in diccionario.items():
    print(f'Clave: {clave}, Valor: {valor}')

# Ejemplo de bucle for con cadena
cadena = 'Hola'
for letra in cadena:
    print(letra)
```

TIPOS BÁSICOS

Números

- Enteros: 3
- Reales: 65.8 (flotante)
- Complejos: 4 + 7j
- Binario: 0b010
- Octal: 0o642
- Hexadecimal: 0xF3

Cadenas

- Entre comillas simples o dobles: "Hola mundo"

Booleanos:

- Valores True y False

La conversión siempre debe ser explícita usando las funciones: str(), int(), float(), bool()

```
009.py
# Conversión de tipos en Python
x=6
y="3"
# z=x+y TypeError: unsupported operand type(s) for +: 'int' and 'str'
# Para evitar el error de tipo, debemos convertir 'y' a un entero
z=x+int(y)
print(z)
int("3f",16) # Convierte la cadena hexadecimal "3f" a un entero
a=str(x)
print(a) # Imprime '6' como cadena
float(a) # Convierte la cadena '6' a un número de punto flotante
float("-11.24 e8") # Convierte la cadena a un número de punto flotante -112400000.0
bool(x) # Convierte el entero 6 a booleano, devuelve True
bool(0) # Convierte el entero 0 a booleano, devuelve False
int(True) # Devuelve 1
int(False) # Devuelve 0
bool("Hola") # Devuelve True, ya que la cadena no está vacía
bool("") # Devuelve False, ya que la cadena está vacía
str(3.14) # Convierte el número de punto flotante 3.14 a cadena '3.14'
```

STRING O CADENAS

Las cadenas de caracteres son secuencias de caracteres que se utilizan para representar texto.

Para definir las se usan los caracteres de escape habituales “ ”

Triples comillas:

- No es necesario incluir los \n.

Operadores (+ y *), para concatenar y repetir.

```
010.py
#Strings o cadenas de caracteres
triple=""" esto es una linea
y esta es otra linea"""
print(triple) # Imprime el texto con saltos de línea
# Concatenación de cadenas
cadena1 = "Hola"
cadena2 = "Mundo"
cadena_concatenada = cadena1 + " " + cadena2
print(cadena_concatenada) # Imprime 'Hola Mundo'
# Repetición de cadenas
cadena_repetida = cadena1 * 3
print(cadena_repetida) # Imprime 'HolaHolaHola'
# Longitud de una cadena
longitud_cadena = len(cadena1)
```

STRING O CADENAS

Python ofrece la posibilidad de usar f-string. Estas son cadenas de caracteres con el prefijo f o F. Estas cadenas pueden incluir pares de llaves `{ }` con expresiones cuyo valor puede ser sustituido en tiempo de ejecución. Dentro de los pares de llaves también podemos especificar detalles del tipo de dato que se espera y la anchura del campo entre otros.

```
010.py
nombre: str = 'Juan'
telefono: int = 678900123
altura: float = 182.3
print(f'{nombre*2:10s} ==> {telefono:10d} => {altura:.2f}')
# En este caso, el nombre se repetirá dos veces, el teléfono se mostrará como un
# número entero de 10 dígitos y la altura se mostrará con dos decimales.
# También podemos usar el método format() para formatear cadenas de manera similar.
print('{} {} {}'.format( *args: nombre, telefono, altura))
```

STRING O CADENAS

Métodos de cadenas:

Python proporciona una variedad de métodos para manipular cadenas.

Algunos de los métodos más comunes son:

- `upper()`: Convierte la cadena a mayúsculas.
- `lower()`: Convierte la cadena a minúsculas.
- `strip()`: Elimina los espacios en blanco al principio y al final de la cadena.
- `replace(old, new)`: Reemplaza todas las ocurrencias de una subcadena por otra.
- `split(separator)`: Divide la cadena en una lista de subcadenas utilizando un separador.
- `splitlines(keepends)`: Divide la cadena en líneas, opcionalmente conservando los saltos de línea.
- `join(iterable)`: Une los elementos de un iterable en una cadena, utilizando la cadena como separador.
- `find(substring)`: Busca una subcadena y devuelve su índice, o -1 si no se encuentra.
- `count(substring)`: Cuenta cuántas veces aparece una subcadena en la cadena.
- `index(substring)`: Similar a `find`, pero lanza una excepción si la subcadena no se encuentra.
- `rfind(substring)`: Busca una subcadena desde el final de la cadena y devuelve su índice, o -1 si no se encuentra.
- `lfind(substring)`: Busca una subcadena desde el principio de la cadena y devuelve su índice, o -1 si no se encuentra.
- `lindex(substring)`: Similar a `rfind`, pero lanza una excepción si la subcadena no se encuentra.
- `rindex(substring)`: Similar a `rfind`, pero lanza una excepción si la subcadena no se encuentra.
- `lstrip()`: Elimina los espacios en blanco al principio de la cadena.
- `rstrip()`: Elimina los espacios en blanco al final de la cadena.
- `isspace()`: Comprueba si todos los caracteres de la cadena son espacios en blanco.

Olga M. Moreno Martín - Javier Herrero Porras

LA FUNCIÓN TYPE

La función `type()` devuelve el tipo de dato en una variable

```
011.py
# Ejemplo de uso de type()
type(3.14) # Devuelve <class 'float'>
type("Hola") # Devuelve <class 'str'>
type(True) # Devuelve <class 'bool'>
type(42) # Devuelve <class 'int'>
type([1, 2, 3]) # Devuelve <class 'list'>
type({1, 2, 3}) # Devuelve <class 'set'>
type({"clave": "valor"}) # Devuelve <class 'dict'>
```

ESTRUCTURAS DE DATOS

Las colecciones son variables que contienen más de un valor.

Tipos:

- **Listas** (Lo que conocemos por vectores o arrays).
 - Los índices comienzan en 0 (Como C y Java).
 - Es permitido índices negativos (se cuenta desde el final)
 - Pueden contener elementos de distinto tipo.
 - Las matrices multidimensionales se consiguen incluyendo elementos de una lista que a su vez son listas.
- **Conjuntos (Set)**. Conjuntos de elementos que no permiten duplicados.
- **Tuplas** (Vectores inmutables).
- **Diccionarios** (Vectores asociativos).
 - Usan claves en lugar de índices para su acceso.

LISTAS

Una lista es una colección de elementos ordenados y mutables.

Se pueden almacenar diferentes tipos de datos en una lista.

Se definen con corchetes `[]` y los elementos se separan por comas `,`.

Pueden ser formadas por tipos arbitrarios

Pueden ser indexadas con `[i]`.

Se pueden anidar, es decir, meter una dentro de la otra.

Son **dinámicas**, ya que se pueden añadir o eliminar elementos.

```
012.py

# LISTAS

# Declaración:
l = [22, True, "una lista", [1, 2]]

# Acceso:

# Una dimensión:
l = [11, False]
mi_var = l[0] # mi_var vale 11
mi_var = l[-1] # mi_var vale False

# Varias dimensiones:
l = ["una lista", [1, 2]]
mi_var = l[1][0] # mi_var vale 1

# Asignación de un valor:
l = [22, True]
l[0] = 99 # Con esto valdrá [99, True]
```

LISTAS: MÉTODOS

Métodos para listas:

- `append()` añade un elemento al final de la lista.
- `extend()` permite añadir una lista a la lista inicial.
- `insert()` añade un elemento en una posición o índice determinado.
- `remove()` recibe como argumento un objeto y lo borra de la lista.
- `pop()` elimina por defecto el último elemento de la lista, pero si se pasa como parámetro un índice permite borrar elementos diferentes al último.
- `clear()` vacía la lista.

```
012.py
#Métodos de listas:
l = [22, True, "una lista", [1, 2]]
# Añadir un elemento al final de la lista
l.append("nuevo elemento")
# l vale [22, True, "una lista", [1, 2], "nuevo elemento"]
# Añadir varios elementos al final de la lista
l.extend([3, 4])
# l vale [22, True, "una lista", [1, 2], "nuevo elemento", 3, 4]
# Insertar un elemento en una posición específica
l.insert(1, "insertado")
# l vale [22, "insertado", True, "una lista", [1, 2], "nuevo elemento", 3, 4]
# Eliminar un elemento por valor
l.remove("una lista")
# l vale [22, "insertado", True, [1, 2], "nuevo elemento", 3, 4]
# Eliminar un elemento por índice
l.pop(2)
# Elimina el elemento en la posición 2 (True),
# l vale [22, "insertado", [1, 2], "nuevo elemento", 3, 4]
# Eliminar el último elemento
l.pop()
# Elimina el último elemento ("nuevo elemento"), l vale [22, "insertado", [1, 2], 3, 4]
# Limpiar la lista
l.clear() # l vale []
```

LISTAS: MÉTODOS

Métodos para listas:

- `reverse()` invierte el orden de la lista.
- `sort()` ordena los elementos de menos a mayor por defecto. Y también permite ordenar de mayor a menor si se pasa como parámetro `reverse=True`.
- `index()` recibe como parámetro un objeto y devuelve el índice de su primera aparición.
- `count()` recibe un elemento y cuenta cuántas veces está en la lista.
- `copy()` copia una lista.
- `len()` recibe una lista y devuelve la longitud de ésta.

```
012.py

# Contar el número de ocurrencias de un elemento
l = [1, 2, 2, 3, 4]
count_2 = l.count(2) # count_2 vale 2

# Obtener el índice del primer elemento encontrado
index_3 = l.index(3) # index_3 vale 3 (índice del primer 3)

# Ordenar la lista
l = [3, 1, 4, 2]
l.sort() # l vale [1, 2, 3, 4]

# Invertir el orden de los elementos de la lista
l.reverse() # l vale [4, 3, 2, 1]

# Copiar una lista
l = [1, 2, 3]
l_copy = l.copy() # l_copy vale [1, 2, 3]

# Concatenar dos listas
l1 = [1, 2]
l2 = [3, 4]
l_concat = l1 + l2 # l_concat vale [1, 2, 3, 4]

# Repetir una lista
l_repeated = l1 * 3 # l_repeated vale [1, 2, 1, 2, 1, 2]

# Comprobar si un elemento está en la lista
l = [1, 2, 3]
is_2_in_list = 2 in l # is_2_in_list vale True

# Comprobar si un elemento no está en la lista
is_5_not_in_list = 5 not in l # is_5_not_in_list vale True

# Obtener la longitud de la lista
l_length = len(l) # l_length vale 3
```

LISTAS: SLICING O PARTICIONADO

Slicing es una técnica para acceder a partes de una secuencia (como listas, tuplas o cadenas)

```
013.py
#SLICING
l = [99, True, "una lista", [1, 2]]
mi_var = l[0:2] # mi_var vale [99, True]
mi_var = l[0:4:2] # mi_var vale [99, "una lista"]
l = [99, True, "una lista"]
mi_var = l[1:] #mi_var vale [True, "una lista"]
mi_var = l[:2] #mi_var vale [99, True]
mi_var = l[:] # mi_var vale [99, True, "una lista"]
mi_var = l[:2] #mi_var vale [99, "una lista"]
```

```
013.py
#Modificación de listas usando rangos (slicing)
l = [99, True, "una lista", [1, 2]]
l[0:2] = [0, 1] # l vale [0, 1, "una lista", [1, 2]]
l[0:2] = [False] # l vale [False, "una lista", [1, 2]]
```

```
013.py
# cadenas como listas
c = "hola mundo"
c[0] # h
c[5:] # mundo
c[:3] # havo
# Modificación de cadenas (no se puede, son inmutables)
```

TUPLAS

Las tuplas son colecciones de datos **ordenadas e inmutables**, es decir, no se pueden modificar una vez creadas.

Las tuplas se definen con paréntesis **()** y pueden contener elementos de diferentes tipos.

```
014.py

#Tuplas

# Definición de una tupla:
t = (1, 2, True, "python")

# Tupla de un elemento:
t = (1,)

# Acceso: Igual que el de las listas.
mi_var= t[0] # mi_var es 1
mi_var= t[0:2] # mi_var es (1,2)
```

```
014.py

# Tuplas con un solo elemento
# Las tuplas con un solo elemento deben tener una coma al final para diferenciarlas de un valor entre paréntesis.
# Ejemplo de tupla con un solo elemento
t_single = (42,) # Tupla con un solo elemento
```

TUPLAS: MÉTODOS

Métodos de tuplas:

- `count()` cuenta el número de veces que el objeto pasado como parámetro se ha encontrado en la lista.
- `index()` busca el objeto que se le pasa como parámetro y devuelve el índice en el que se ha encontrado. También acepta un segundo parámetro opcional, que indica a partir de que índice empezar a buscar el objeto.

```
014.py
#Métodos de tuplas:
# Las tuplas son inmutables, por lo que no tienen métodos para modificar su contenido.
# Sin embargo, se pueden usar algunos métodos de las secuencias:
# Contar el número de ocurrencias de un elemento
t = (1, 2, 2, 3)
count_2 = t.count(2) # count_2 vale 2
# Obtener el índice del primer elemento encontrado
index_3 = t.index(3) # index_3 vale 3 (índice del primer 3)
```

SET

Los sets en Python son un tipo que permite almacenar varios elementos y acceder a ellos de una forma muy similar a las listas, pero con ciertas diferencias:

- Los elementos de un set son **únicos**, lo que significa que no puede haber elementos duplicados.
- Los sets son **desordenados**, lo que significa que no mantienen el orden de cuando son declarados.
- Sus elementos deben ser **inmutables**.

Para entender mejor los sets, es necesario entender ciertos conceptos matemáticos como **la teoría de conjuntos**.

Para crear un set en Python se puede hacer con `set()` y pasando como entrada cualquier tipo iterable, como puede ser una lista. Se puede ver como a pesar de pasar elementos duplicados y en un orden determinado, al imprimir el set no conserva ese orden y los duplicados se han eliminado.

Los conjuntos se definen con corchetes **{ }**

Olga M. Moreno Martín - Javier Herrero Porras

```
014.py
#Set
# Definición de un set:
s = {1, 2, True, "python"}
# Set de un elemento:
s = {1,} # Set con un solo elemento
# Acceso: Los sets no tienen un orden definido,
# por lo que no se puede acceder a sus elementos por índice.
#Definición de un set con duplicados:
s2 = {1, 2, 2, 3} # Los duplicados se eliminan automáticamente
print(s2) # s2 vale {1, 2, 3}
```

SET: MÉTODOS

Métodos para conjuntos:

- `add()` añade un elemento al conjunto.
- `remove()` recibe como argumento un objeto y lo borra, si el elemento no está en el conjunto lanza un error.
- `discard()` elimina el elemento pasado como parámetro del conjunto, sin lanzar ningún error si el elemento no existe.
- `copy()` copia el conjunto.

```
014.py

# Métodos de sets:
# Añadir un elemento al set
s.add(3) # Añade el elemento 3 al set
# Eliminar un elemento del set
s.remove(2) # Elimina el elemento 2 del set (lanza un error si no existe)
# Eliminar un elemento del set sin lanzar error si no existe
s.discard(2) # Elimina el elemento 2 del set (no lanza error si no existe)
# Comprobar si un elemento está en el set
is_2_in_set = 2 in s # is_2_in_set vale False
# Comprobar si un elemento no está en el set
is_5_not_in_set = 5 not in s # is_5_not_in_set vale True
# Copiar un set
s_copy = s.copy() # Crea una copia del set
```

SET: TRABAJANDO CON TEORÍA DE CONJUNTOS

Explorando operaciones de conjuntos



```
014.py

# Concatenar dos sets (unión)
s1 = {1, 2}
s2 = {3, 4}
s_union = s1 | s2 # s_union vale {1, 2, 3, 4}

# Repetir un set (no tiene sentido, pero se puede hacer)
s_repeated = s1 # Los sets no se pueden repetir, pero se puede asignar el mismo set a otra variable

# Comprobar si un set es un subconjunto de otro
s1 = {1, 2}
s2 = {1, 2, 3}
is_subset = s1.issubset(s2) # is_subset vale True

# Comprobar si un set es un superconjunto de otro
is_superset = s2.issuperset(s1) # is_superset vale True

# Intersección de dos sets
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s_intersection = s1 & s2 # s_intersection vale {2, 3}

# Diferencia de dos sets
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s_difference = s1 - s2 # s_difference vale {1}

# Diferencia simétrica de dos sets
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s_symmetric_difference = s1 ^ s2 # s_symmetric_difference vale {1, 4}
```

SET: CONJUNTOS ORDENADOS

Con los conjuntos no tenemos un método `sort()` para ordenar... y eso nos hace que en algunas ocasiones tengamos que recurrir a un tipo de conjunto especial que ordena automáticamente los elementos de los conjuntos, esta estructura es conocida como **SortedSet** o conjuntos ordenados.

Necesitaremos instalar el paquete `sortedcontainers` e importar `SortedSet`.

Funcionan de la misma forma que los sets.

```
014.py
#SortedSet
# Definición de un SortedSet:
from sortedcontainers import SortedSet
ss = SortedSet([3, 1, 2])
# SortedSet de un elemento:
ss_single = SortedSet([1]) # SortedSet con un solo elemento
# Acceso: Igual que el de los sets, pero ordenado.
```

DICCIONARIOS

Son estructuras que almacenan pares **clave/valor**.

Para acceder a un valor hay que proporcionar la clave.

Se comportan como vectores, pero no tienen un índice numérico para acceder a los elementos.

```
015.py
#Diccionarios

d = {"Love Actually ": "Richard Curtis", "Kill Bill":"Tarantino","Amélie": "Jean-Pierre Jeunet"}
d["Love Actually "] # devuelve "Richard Curtis"
d["Kill Bill"] = "Quentin Tarantino"
f = { } # crear un diccionario vacío
f["Torrente"] = "Santiago Segura" # insertar un elemento
f["Torrente"] = "Santi Segura" # actualización del valor
```

DICCIONARIOS: MÉTODOS

Métodos para listas:

- `get(key)` permite consultar el valor de una clave.
- `keys()` devuelve las claves.
- `values()` devuelve los valores.
- `items()` devuelve los pares clave-valor.
- `pop(key)` eliminar la clave y su valor.
- `copy()` crea una copia SUPERFICIAL del diccionario
- `clear()` vacía la lista.

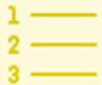
```
015.py

# Métodos de diccionarios
# Añadir un elemento al diccionario
d["Inception"] = "Christopher Nolan" # añade una nueva clave-valor
# Obtener el valor de una clave
valor = d.get("Amélie") # devuelve "Jean-Pierre Jeunet"
# Obtener todas las claves del diccionario
claves = d.keys() # devuelve un objeto dict_keys con las claves
# Obtener todos los valores del diccionario
valores = d.values() # devuelve un objeto dict_values con los valores
# Obtener todos los pares clave-valor del diccionario
pares = d.items() # devuelve un objeto dict_items con los pares clave-valor
# Eliminar un elemento del diccionario
d.pop("Kill Bill") # elimina la clave "Kill Bill" y su valor
# Limpiar el diccionario
d.clear() # elimina todos los elementos del diccionario
# Comprobar si una clave está en el diccionario
existe = "Amélie" in d # devuelve True si "Amélie" es una clave en el diccionario
# Comprobar si una clave no está en el diccionario
no_existe = "Inexistente" not in d # devuelve True si "Inexistente" no es una clave en el diccionario
# Copiar un diccionario
copia_d = d.copy() # crea una copia superficial del diccionario
# Concatenar dos diccionarios
d1 = {"A": 1, "B": 2}
d2 = {"C": 3, "D": 4}
d_concat = {**d1, **d2} # crea un nuevo diccionario con las claves y valores de d1 y d2
```

COMPARACIÓN DE TIPOS DE DATOS: LISTAS, TUPLAS, SETS Y DICCIONARIOS

Listas, Tuplas, Sets y Diccionarios son tipos de datos diferentes en Python.

¿Qué uso en cada caso?... Pues depende de qué tipo de datos me hagan falta:

	 Lista	 Tupla	 Conjunto	 Conjunto Ordenado	 Diccionario
Ordenado	Sí	Sí	No	Sí	No
Mutable	Sí	No	Sí	Sí	Sí
Duplicados	Sí	Sí	No	No	No

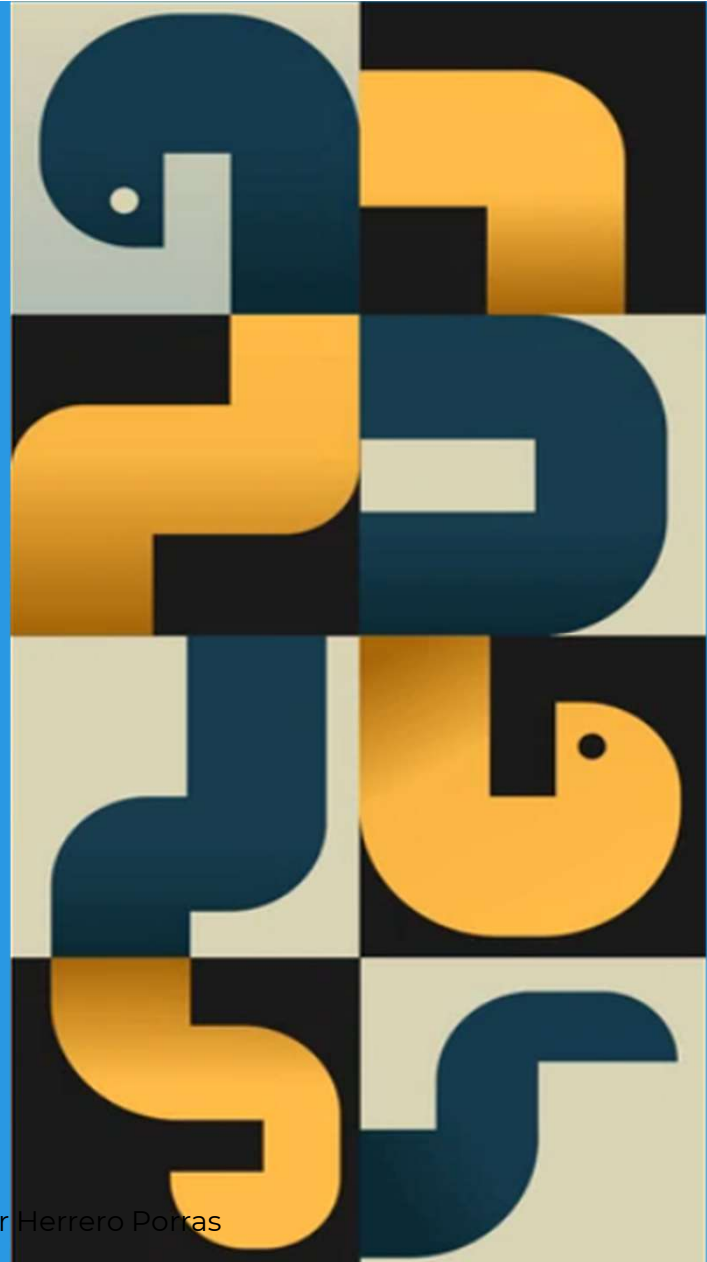
¿PREGUNTAS?

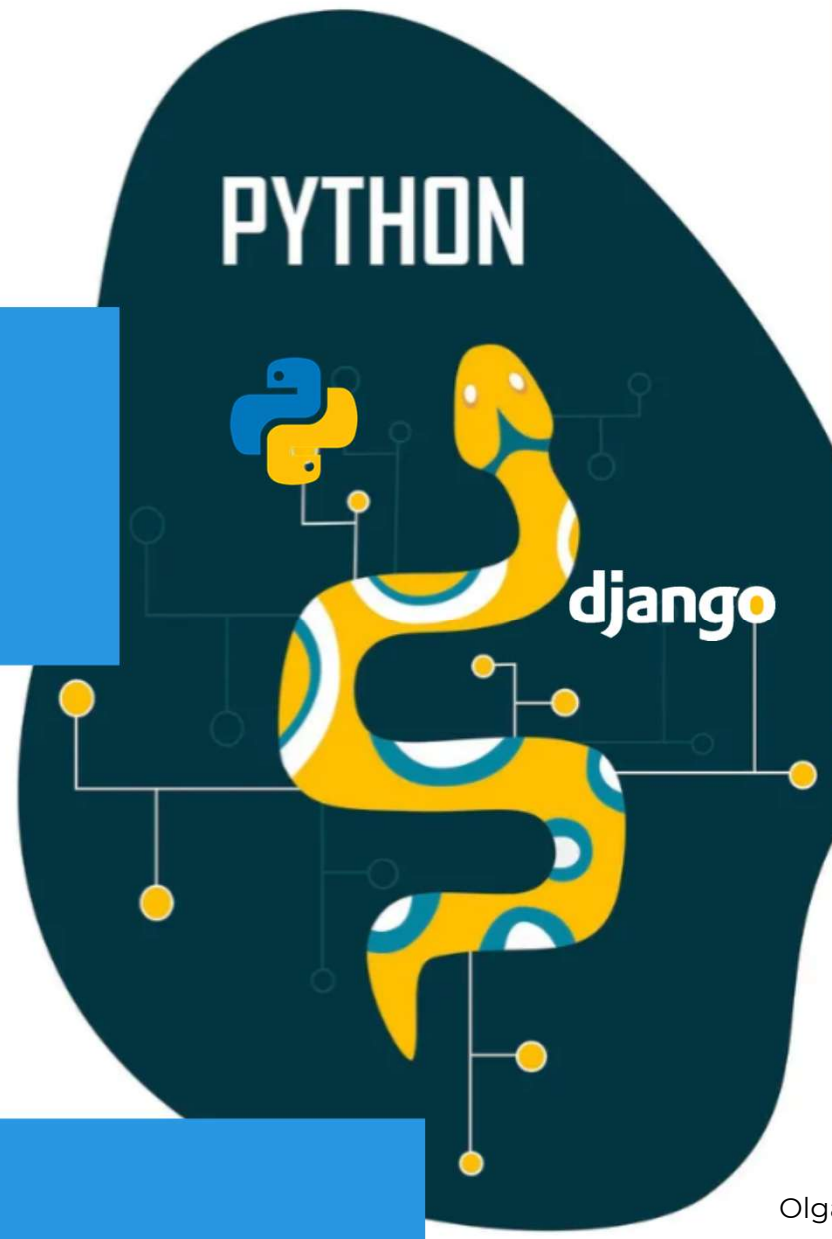
“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



A graphic on the left side of the slide. It features a dark teal, irregular shape. Inside this shape, at the top, is the word 'PYTHON' in white, uppercase letters. Below it is a stylized yellow and blue Python logo. Further down is a yellow and blue Django logo, which is a stylized snake. To the right of the snake is the word 'django' in white, lowercase letters. The entire graphic is surrounded by a network of thin white lines and small yellow circles, suggesting a circuit or data flow.

PYTHON

UD2:

FUNCIONES

—Y ellos, naturalmente, responden a sus nombres, ¿no? —
observó al desgaire el Mosquito.
—Nunca oí decir tal cosa.
—¿Pues de qué les sirve tenerlos —preguntó el
Mosquito— si no responden a sus nombres?

Alicia en el país de las maravillas, Lewis Carroll

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

ÍNDICE

- ▶ **Definición**
- ▶ **Características**
- ▶ **Funciones de Orden superior**
- ▶ **Iteraciones de Orden superior**
- ▶ **Funciones Lambda**
- ▶ **Comprension**
- ▶ **Módulos y paquetes**

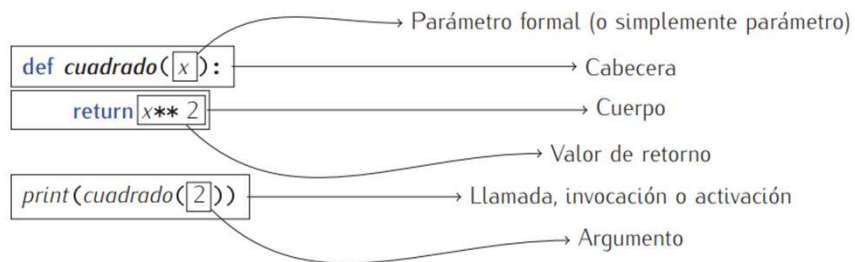
FUNCIONES

Denominaremos activar, invocar o llamar a una función a la acción de usarla.

Las funciones que hemos aprendido a invocar reciben cero, uno o más argumentos separados por comas y encerrados entre un par de paréntesis y pueden devolver un valor o no devolver nada.

Necesitamos la palabra reservada **def** para declararlas seguida del nombre de la propia función.

Seguirán esta estructura:



```
016.py
def mi_funcion(param1, param2): 4 usages new *
    """
    Esta función toma dos parámetros y devuelve una lista que contiene
    los elementos de ambos parámetros.

    :param param1: Primer parámetro, puede ser un número o una cadena.
    :param param2: Segundo parámetro, puede ser un número o una cadena.
    :return: Lista que contiene los dos parámetros.
    """

    return [param1, param2]

# Ejemplo de uso de la función
resultado = mi_funcion( param1: 5, param2: "hola")
print(resultado) # Imprime: [5, 'hola']

# Ejemplo de uso de la función con diferentes tipos de parámetros
resultado2 = mi_funcion( param1: "Python", param2: 3.14)
print(resultado2) # Imprime: ['Python', 3.14]

# Ejemplo de uso de la función con parámetros de tipo cadena
resultado3 = mi_funcion( param1: "cadena1", param2: "cadena2")
print(resultado3) # Imprime: ['cadena1', 'cadena2']

# Ejemplo de uso de la función con parámetros de tipo número
resultado4 = mi_funcion( param1: 10, param2: 20)
```

CARACTERÍSTICAS

Paso de argumentos por valor excepto para colecciones.

Devolución de valor opcional

- Se puede devolver más de un valor

Parámetros pre-inicializados

```
017.py
# Este código muestra cómo asignar múltiples valores
# a múltiples variables en Python.
def f(x, y): 1 usage new *
    return x * 2, y * 2
a, b = f(x: 1, y: 2)
```

```
017.py
#Ejemplo con parámetros opcionales
def mi_funcion(param1, param2="default2"): 2 usages Olga M. Moreno *
    """
    Esta función toma un parámetro obligatorio y uno opcional.
    :param param1: Primer parámetro, obligatorio.
    :param param2: Segundo parámetro, opcional, por defecto "default2".
    :return: Producto de los dos parámetros.
    """
    return param1 * param2
print(mi_funcion(param1: "hola", param2: 2))holahola
print(mi_funcion(param2=2, param1="hola")) holahola
```

CARACTERÍSTICAS

Número variable de parámetros

```
017.py
# Ejemplo con número variable de argumentos
def suma(*args): 1 usage new *
    """
    Esta función suma un número variable de argumentos.
    :param args: Argumentos a sumar.
    :return: Suma de todos los argumentos.
    """
    return sum(args)
print(suma(*args: 1, 2, 3)) # Imprime 6
```

```
017.py
# Ejemplo con argumento como diccionario
def imprimir_info(**kwargs): 1 usage new *
    """
    Esta función imprime información basada en argumentos nombrados.
    :param kwargs: Argumentos nombrados.
    """
    for key, value in kwargs.items():
        print(f"{key}: {value}")

imprimir_info(nombre="Juan", edad=30, ciudad="Madrid")
```

FUNCIONES DE ORDEN SUPERIOR

Las funciones de Python pueden tomar funciones como parámetros y devolver funciones como resultado.

Una función que hace ambas cosas o alguna de ellas se llama función de orden superior.

```
018.py

#Funciones de orden superior
def saludar(lang):
    """ 3 usages new """
    def saludar_es():
        """ new """
        print("Hola")
    def saludar_en():
        """ new """
        print("Hello")
    def saludar_fr():
        """ new """
        print("Salut")
    lang_func = {"es":saludar_es,
                  "en":saludar_en,
                  "fr":saludar_fr}
    return lang_func[lang]

f = saludar("es")
f() # Llama a la función de saludo en español
f = saludar("en")
f() # Llama a la función de saludo en inglés
f = saludar("fr")
f() # Llama a la función de saludo en francés
```

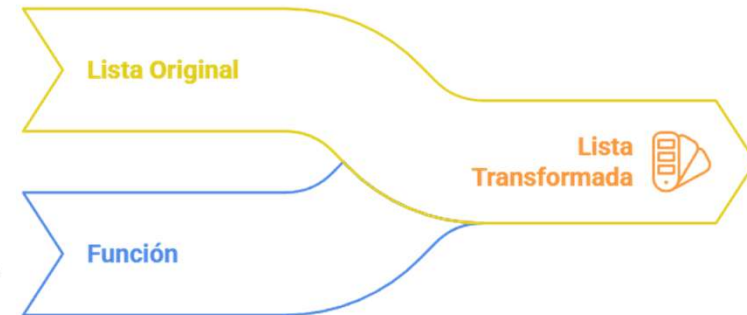
ITERACIONES DE ORDEN SUPERIOR: MAP

`map(function, sequence[, sequence, ...])`

param1 param2 ...paramN

Transformación de Datos con Map

- 1 Lista inicial de elementos a transformar.
- 2 Función aplicada a cada elemento de la lista.



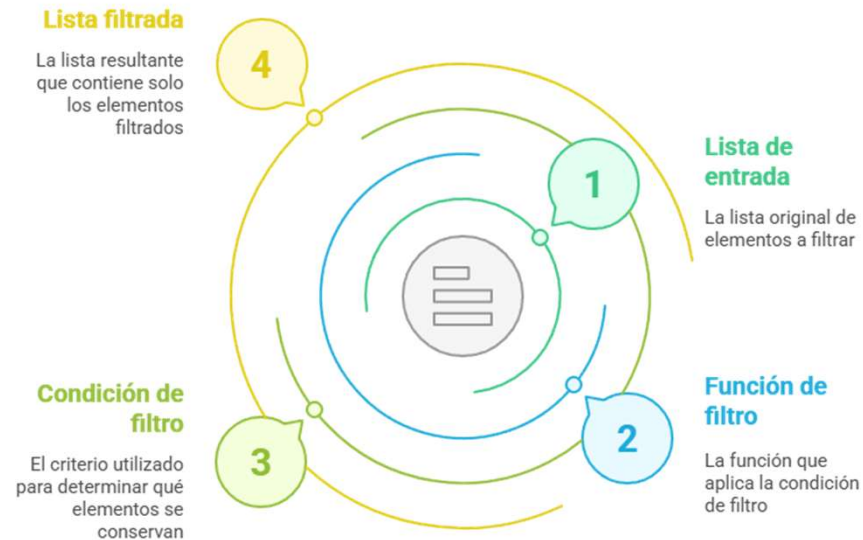
Made with  Napkin

019.py

```
## Ejemplo de uso de la función map para aplicar una función a cada elemento de una lista
def cuadrado(n): 1 usage new *
    return n ** 2
l = [1, 2, 3]
l2 = map(cuadrado, l)
print (list(l2))
print (type(l2))
```

ITERACIONES DE ORDEN SUPERIOR: FILTER

Desglose de la función de filtro de Python

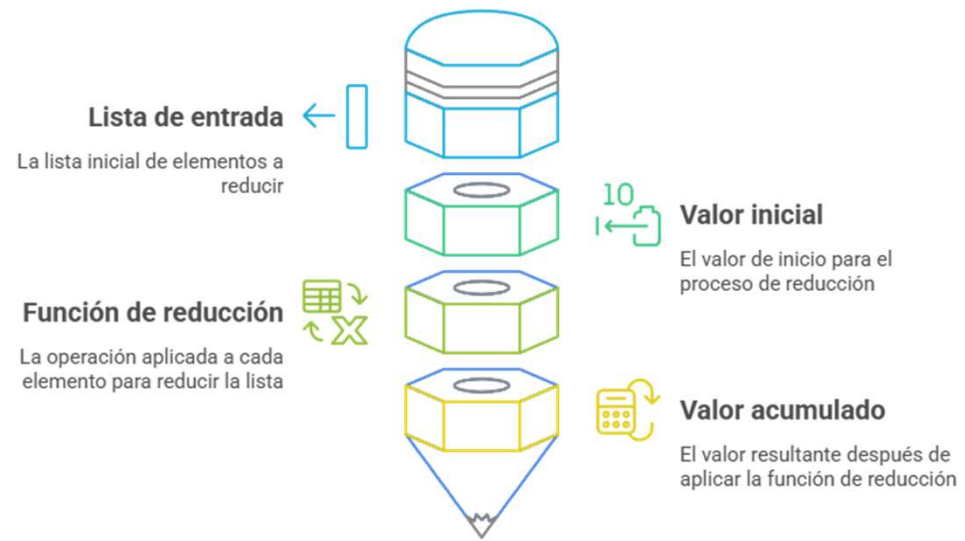


```
019.py
# Ejemplo de uso de la función filter para filtrar elementos de una lista
def es_par(n): 1 usage new *
    return n % 2 == 0
l = [1, 2, 3]
l3 = filter(es_par, l)
print (list(l3))
print (type(l3))
```

Olga M. Moreno Martín - Javier Herrero Porras

ITERACIONES DE ORDEN SUPERIOR: REDUCE

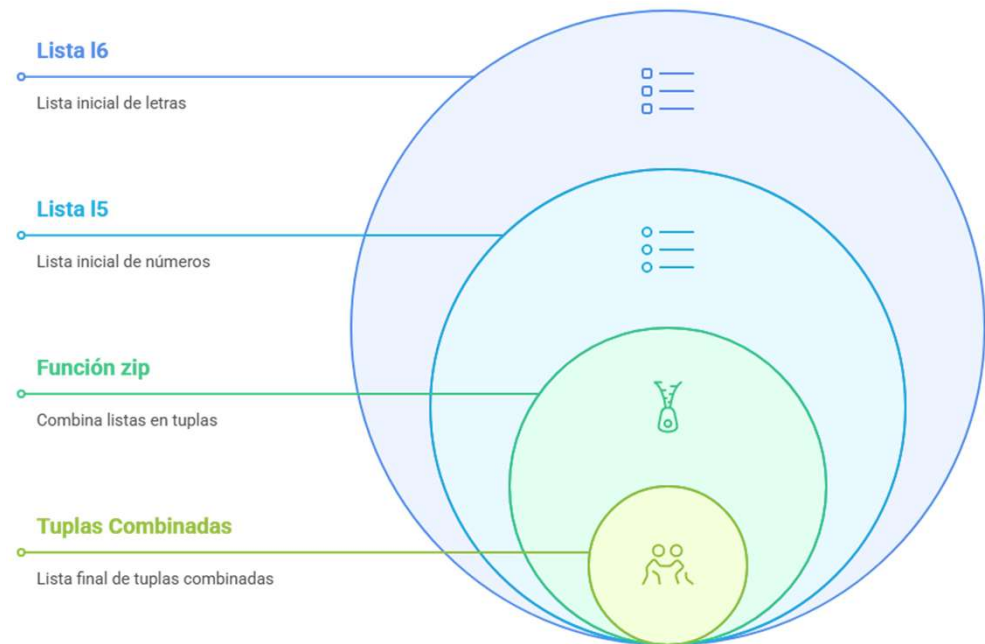
Desglose de la función reduce



```
019.py
# Ejemplo de uso de la función reduce para reducir una lista a un único valor
from functools import reduce
def suma(a, b): 1 usage new *
    return a + b
l = [1, 2, 3]
l4 = reduce(suma, l)
print(l4) # Imprime 6
print(type(l4)) # <class 'int'>
```

ITERACIONES DE ORDEN SUPERIOR: ZIP

Proceso de Combinación de Listas



019.py

```
# Ejemplo de uso de la función zip para combinar dos listas en una lista de tuplas
l5 = [1, 2, 3]
l6 = ['a', 'b', 'c']
l7 = zip(l5, l6)
print(list(l7)) # Imprime [(1, 'a'), (2, 'b'), (3, 'c')]
```

ITERACIONES DE ORDEN SUPERIOR: SORTED, REVERSED, ANY Y ALL

```
019.py
# Ejemplo de uso de la función sorted para ordenar una lista
l8 = [3, 1, 2]
l9 = sorted(l8)
print(l9) # Imprime [1, 2, 3]
```

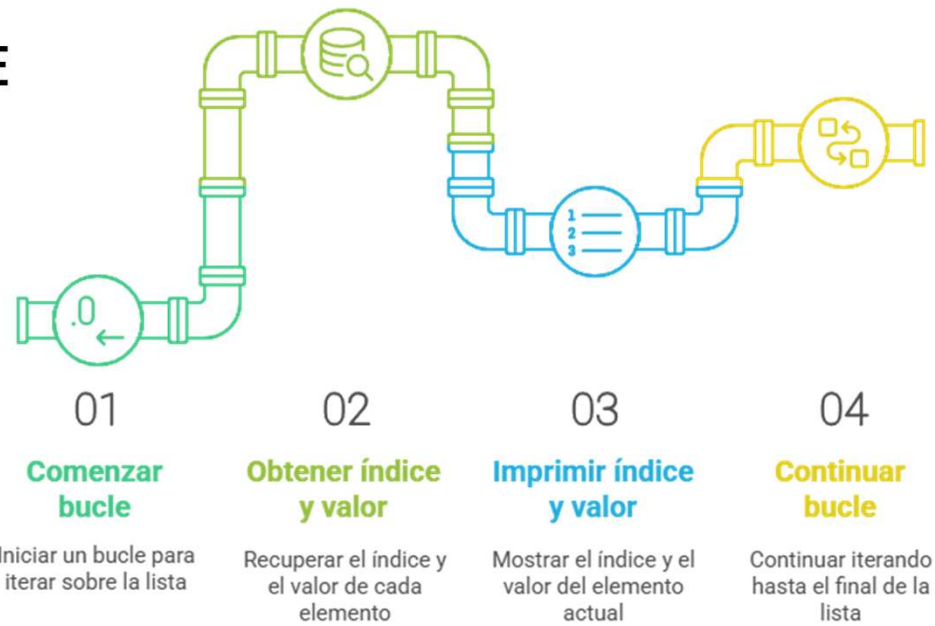
```
019.py
# Ejemplo de uso de la función reversed para invertir una lista
l15 = [1, 2, 3]
l16 = list(reversed(l15))
print(l16) # Imprime [3, 2, 1]
```

```
019.py
# Ejemplo de uso de la función all para verificar si todos los elementos son verdaderos
l12 = [1, 2, 3]
l13 = all(l12)
print(l13) # Imprime True, ya que todos los elementos son verdaderos
```

```
019.py
# Ejemplo de uso de la función any para verificar si al menos un elemento es verdadero
l10 = [0, 1, 2]
l11 = any(l10)
print(l11) # Imprime True, ya que hay elementos verdaderos en la lista
```

Iteración de lista con enumerate

ITERACIONES DE ORDEN SUPERIOR: ENUMERATE



019.py

```
# Ejemplo de uso de la función enumerate para obtener índices y valores de una lista
l14 = ['a', 'b', 'c']
for index, value in enumerate(l14):
    print(f"Índice: {index}, Valor: {value}")
```

FUNCIONES LAMBDA

Funciones anónimas

```
020.py
# Función lambda para sumar dos números
suma = lambda x, y: x + y
print(suma(x=3, y=5)) # Imprime 8

# Función lambda para ordenar una lista de tuplas por el segundo elemento
lista_tuplas = [(1, 3), (2, 1), (3, 2)]
ordenada = sorted(lista_tuplas, key=lambda x: x[1])
print(ordenada) # Imprime [(2, 1), (3, 2), (1, 3)]

# Función lambda para filtrar números pares de una lista
numeros = [1, 2, 3, 4, 5, 6]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares) # Imprime [2, 4, 6]

# Función lambda para aplicar una operación a cada elemento de una lista
numeros = [1, 2, 3, 4, 5]
doble = list(map(lambda x: x * 2, numeros))
print(doble) # Imprime [2, 4, 6, 8, 10]
```

```
020.py
# Ejemplo de uso de la función filter con una función lambda
l = [1, 2, 3]
l2 = filter(lambda n: n % 2.0 == 0, l)
# Está restringido a una sola expresión
print(list(l2)) # Imprime [2]

# Ejemplo de uso de la función map con una función lambda
l3 = map(lambda n: n ** 2, l)
print(list(l3)) # Imprime [1, 4, 9]

# Ejemplo de uso de la función reduce con una función lambda
from functools import reduce
l4 = reduce(lambda a, b: a + b, l)
print(l4) # Imprime 6

# Ejemplo de uso de la función zip con una función lambda
l5 = [1, 2, 3]
l6 = ['a', 'b', 'c']
l7 = zip(l5, l6)
print(list(l7)) # Imprime [(1, 'a'), (2, 'b'), (3, 'c')]
```

COMPRESIÓN (COMPREHENSION)

Basado en el lenguaje funcional Haskell

Permite crear **listas []**, **diccionarios {}** y **conjuntos {}** a partir de uno ya existente.

Con los **generadores ()**:

- Equivalente a la comprensión de listas, pero devuelven un generador, no la lista.
- Es una clase especial de función que genera valores sobre los que iterar.
- Los generadores no crean la lista completa en memoria. En todo caso, podríamos crearla usando:

```
lista = list(mi_generador)
```

Esto haría que podamos mostrar el generador por pantalla y no obtengamos el mensaje:

```
<generator object mi_generador at  
0x10cd30e60>
```

```
021.py  
  
# Comprensión de lista  
l = [1, 2, 3]  
l2 = [n ** 2 for n in l]  
l3 = [n for n in l if n%2==0]  
#No haría falta la función filter  
print(l2) # Imprime [1, 4, 9]  
print(l3) # Imprime [2]  
  
# Comprensión de diccionario  
d = {n: n ** 2 for n in l}  
print(d) # Imprime {1: 1, 2: 4, 3: 9}  
  
# Comprensión de conjunto  
s = {n ** 2 for n in l}  
print(s) # Imprime {1, 4, 9}  
  
# Comprensión de conjunto con condición  
s2 = {n for n in l if n % 2 == 0}  
print(s2) # Imprime {2}  
  
# Comprensión de generador  
g = (n ** 2 for n in l)  
print(list(g)) # Imprime [1, 4, 9]
```

COMPRENSIÓN (COMPREHENSION)

Vamos a seguir viendo ejemplos:

```
021.py

# Comprensión de generador con condición
g2 = (n for n in l if n % 2 == 0)
print(list(g2)) # Imprime [2]

# Comprensión de lista con función
def cuadrado(n):
    return n ** 2

l5 = [cuadrado(n) for n in l]
print(l5) # Imprime [1, 4, 9]

# Comprensión de generador con función
g4 = (cuadrado(n) for n in l)
print(list(g4)) # Imprime [1, 4, 9]

# Comprensión de generador con función y condición
g5 = (cuadrado(n) for n in l if n % 2 == 0)
print(list(g5)) # Imprime [4]

# Comprensión de lista anidada
l4 = [[i, j] for i in range(3) for j in range(2)]
print(l4) # Imprime [[0, 0], [0, 1], [1, 0], [1, 1], [2, 0], [2, 1]]

# Comprensión de diccionario anidado
d2 = {i: {j: i * j for j in range(3)} for i in range(3)}
print(d2) # Imprime {0: {0: 0, 1: 0, 2: 0}, 1: {0: 0, 1: 1, 2: 2}, 2: {0: 0, 1: 2, 2: 4}}

# Comprensión de conjunto anidado
s2 = {i * j for i in range(3) for j in range(2)}
print(s2) # Imprime {0, 1, 2, 3, 4}

# Comprensión de generador anidado
g3 = (i * j for i in range(3) for j in range(2))
print(list(g3)) # Imprime [0, 0, 0, 1, 2, 3, 4]
```

Olga M. Moreno Martín - Javier Herrero Porras

MÓDULOS Y PAQUETES

La agrupación de código se hace usando módulos y paquetes:

Módulo: Se corresponde con un archivo de código fuente (extensión .py). 

Paquete: Se corresponde con un directorio.  En el ejemplo usamos numpy
Se pueden empaquetar en archivos “.egg” (Huevos)  : Equivalentes a los jar.

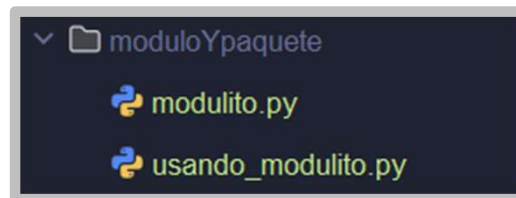
MÓDULOS Y PAQUETES

Los módulos son en sí mismo objetos

Atributos de los módulos:

- `__name__`
- `__doc__`

Ejemplo uso `__name__`:



```
modulito.py

## Ejemplo de un módulo en Python
# Este es un ejemplo de un módulo en Python.
def mi_funcioncita (): 2 usages new *
    print ("una funcion")
# Ejemplo de una clase en un módulo
class MiClase: 1 usage new *
    """Una clase de ejemplo"""
    def __init__ (self): new *
        print ("una clase")
    print ("un módulo")

print ("se muestra siempre")
if __name__ == "__main__":
    print("Se muestra si no es importación")
```

```
usando_modulito.py

import modulito # Importa todo el módulo
from modulito import MiClase # Importa la clase MiClase del módulo
from modulito import mi_funcioncita # Importa la función mi_funcioncita del módulo
from modulito import * # Importa todo del módulo (no recomendado en producción)
from modulito import mi_funcioncita# Importa la función mi_funcioncita del módulo

import numpy as np # Importa numpy con el alias np
from numpy import array as ar # Importa array de numpy con el alias ar
```

MÓDULOS Y PAQUETES

Los paquetes 📦:

- Sirven para organizar módulos
- Son un tipo especial de módulos
- Solemos utilizar paquetes que hemos instalado previamente, en el ejemplo importamos numpy.
- Estructura básica de un paquete:

mi_paquete/

├── __init__.py
├── modulo1.py
├── modulo2.py
└── subpaquete/
 ├── __init__.py
 └── submodulo.py

- __init__.py: Indica que el directorio es un paquete. Puede estar vacío o contener código de inicialización.
- modulo1.py, modulo2.py: Módulos con funciones, clases o variables.
- subpaquete/: Un subdirectorio también con __init__.py para considerarse subpaquete.

```
usando_modulito.py

import modulito # Importa todo el módulo
from modulito import MiClase # Importa la clase MiClase del módulo
from modulito import mi_funcioncita # Importa la función mi_funcioncita del módulo
from modulito import * # Importa todo del módulo (no recomendado en producción)
from modulito import mi_funcioncita # Importa la función mi_funcioncita del módulo

import numpy as np # Importa numpy con el alias np
from numpy import array as ar # Importa array de numpy con el alias ar
```

Así importamos los paquetes

MÓDULOS Y PAQUETES

Si deseas que un paquete sea instalable:

Crea un archivo setup.py en la raíz:

```
from setuptools import setup, find_packages
```

```
setup(
```

```
    name="mi_paquete",
```

```
    version="0.1",
```

```
    packages=find_packages(),
```

```
)
```

```
mi_paquete/  
├── __init__.py  
├── modulo1.py  
├── modulo2.py  
└── setup.py
```

Instálalo en tu entorno: **pip install -e**

Si queremos publicar paquetes tendremos que tener esto en cuenta.

MATH EN PYTHON

En Python, el módulo **math** proporciona funciones matemáticas definidas en el estándar de C. Es muy útil para realizar cálculos numéricos más avanzados que los operadores básicos (+, -, *, /). Para utilizarlo, tenemos que importarlo con:

```
023.py
import math

# Calcular el área de un círculo con radio 5
radio = 5
area = math.pi * math.pow(radio, 2)
print(f"Área del círculo: {area:.2f}") # Área del círculo: 78.54
# .2f -> con dos decimales

# Calcular la hipotenusa de un triángulo rectángulo con lados 3 y 4
hipotenusa = math.sqrt(math.pow(x: 3, y: 2) + math.pow(x: 4, y: 2))
print(f"Hipotenusa: {hipotenusa}") # Hipotenusa: 5.0

# Redondear un número
print(math.floor(3.9)) # 3
print(math.ceil(3.1)) # 4
```

Olga M. Moreno Martín - Javier Herrero Porras

¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON



django

UD3:

POO

"La POO es como un castillo 🏰: tienes torres 🏰 (clases), murallas 🧱 (encapsulación) y caballeros 🗡️ (objetos) protegiendo el reino 🏰 (el sistema)."

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

ÍNDICE

▶ POO

▶ El Zen de Python

▶ Sintaxis

▶ This - self

▶ Operadores

▶ Herencia

▶ Encapsulación

▶ Métodos especiales

▶ Excepciones

P00

La programación orientada a objetos es un **paradigma de desarrollo**, al igual que la programación funcional o la programación imperativa.

El principio es que dentro del programa, todos los conceptos de negocio, las entidades de la "vida real", los "actores" de los algoritmos, se representan por piezas de software llamadas "objetos". Los distintos objetos del programa contienen datos y tienen un comportamiento definido por su implementación. También interactúan entre sí, utilizándose mutuamente para llevar el programa a cumplir su objetivo.

Una de las etapas más importantes (si no la más importante), es el **modelado del programa**. Consiste en enumerar los objetos que necesitará la aplicación y definir las relaciones entre ellos. Esta etapa puede incluir varios niveles de abstracción más o menos altos: el nivel más alto puede incluir solo las grandes partes de la arquitectura, el nivel más bajo solo presenta los objetos "finales" que realmente se desarrollarán, con tantos niveles intermedios como sea necesario dependiendo de la complejidad del proyecto.

La etapa de modelado es un trabajo colaborativo entre personas potencialmente de diferentes áreas del negocio. Por tanto, necesitamos un lenguaje común, que todos puedan entender, para poder llevar a cabo esta tarea con éxito. En este caso, **UML** (Unified Modeling Language).

La filosofía vinculada al lenguaje Python recomienda vaciar la mente y observar lo que ocurre a nuestro alrededor, para inspirarse e incluso reutilizar lo que sea posible. Todo ello se opone a una doctrina que trata de convencer a todos aquellos para los que solo su manera de ver las cosas es correcta. De este modo, en lo relativo a los paradigmas, se le recomienda empezar utilizando el que te resulte más familiar, y más adelante evolucionar hacia los demás, a tu ritmo. Por este motivo **Python es multiparadigma**.

Olga M. Moreno Martín - Javier Herrero Porras

EL ZEN DE PYTHON

La filosofía puede resumirse por «El Zen de Python»

<http://www.python.org/dev/peps/pep-0020/>

Hermoso es mejor que feo;

Explícito es mejor que implícito;

Simple es mejor que complejo;

Complejo es mejor que complicado;

Plano es mejor que anidado;

Disperso es mejor que lento;

La legibilidad cuenta;

Los casos especiales no son suficientemente especiales como para romper las reglas;

Lo pragmático gana a la pureza;

Aunque la funcionalidad tiene prioridad sobre la pureza

Los errores nunca deberían dejarse pasar silenciosamente;

A menos que se silencien explícitamente;

Cuando te enfrentes a la ambigüedad, rechaza la tentación de adivinar;

Debería haber una, y preferiblemente solo una, manera obvia de hacerlo;

Aunque puede que no sea obvia a primera vista a menos que seas holandés (observación: Guido van Rossum es holandés);

Ahora es mejor que nunca;

Aunque nunca suele ser mejor que «ahora mismo»;

Si la implementación es difícil de explicar, es una mala idea;

Si la implementación es sencilla de explicar, puede que sea una buena idea;

Los espacios de nombres son una gran idea -¡tenemos más de esas!

SINTAXIS

Declarar una clase es tan sencillo como declarar una función: una palabra clave, seguida del nombre de la clase, seguida de un bloque que contiene el código de la clase:

```
class MiClase:
```

Y crear una instancia:

```
mi_instancia = MiClase()
```

Destacamos la presencia de los paréntesis para gestionar la construcción del objeto. No es necesario en absoluto utilizar una palabra clave. ¿Por qué? Porque los lenguajes estáticos van a realizar automáticamente operaciones para construir el objeto en memoria y todo el proceso es predecible respecto a los atributos declarados en la clase.

En Python, no ocurre así. Todo es un objeto, una instancia es un objeto como cualquier otro y la manera en la que se construye estará definida por el código presente en el método `__new__` y no por el código escrito en el lenguaje. Gracias a esta flexibilidad, podemos modificar la manera en la que se crea un objeto en Python y resolver así fácilmente todos los patrones de diseño correspondientes a la construcción.

Más allá de este método, existe el método de inicialización `__init__`, también llamado constructor, para mantener el mismo vocabulario que en los demás lenguajes, aunque en realidad el responsable sea el método `__new__`.

THIS? - SELF

En la mayoría de lenguajes de programación, existe una palabra clave (generalmente this) que representa la instancia en curso. El propio lenguaje hace un poco de magia arreglándoselas para encontrar la instancia correcta.

En Python, no hay magia alguna. La instancia en curso no es una palabra clave, sino el primer parámetro de cada método, usaremos la palabra reservada **self**.

Este es uno de los aspectos más desconcertantes para aquellos que hayan programado con objetos en algún otro lenguaje.

```
Punto.py
class Punto: 4 usages new *
    """Representa un punto en el espacio"""

    def __init__(self, x, y, z): new *
        """Metodo de inicialización de un punto en el espacio"""
        self.x = x
        self.y = y
        self.z = z

    def mostrar(self): 1 usage new *
        """Metodo temporal utilizado para mostrar nuestro punto"""
        print("Point ({}, {}, {})".format(*args: self.x, self.y, self.z))
```

```
Punto.py
p = Punto( x: 1, y: 2, z: 3)
p.mostrar()
```

OPERADORES

Recordaremos que en Python, todo es un objeto. Cuando se utiliza un operador, Python va a invocar, en realidad, a un método especial del operador sobre el operando de la izquierda y a pasarle el operando de la derecha como parámetro (si existe, lo que depende del operador en cuestión).

Basta con crear un método con un nombre especial para que el operador asociado exista para la clase.

- Ejercicio 1: Agrega el operador de suma a la clase Punto, sabiendo que se utiliza el método especial `__add__` (y que un punto puede sumarse con otro punto).
- Ejercicio 2: Agrega el operador de sustracción (método `__sub__`) así como el operador de multiplicación (método especial `__mul__`), sabiendo que un punto se multiplica por un escalar (número).
- Ejercicio 3: El método especial `__str__` es el que se utiliza por `print` para mostrar un objeto, sea cual sea. Sobrecárgalo para utilizarlo en lugar del método `mostrar`.
- Ejercicio 4: Ahora investiguemos el concepto de mutabilidad, ¿qué es diferente con el operador `+=` que con `+`? Implementa los ejercicios 1 y 2 con estos nuevos métodos.

HERENCIA

En Python, el lenguaje se ha diseñado en primer lugar para ser multiparadigma con, entre otros, el soporte al paradigma orientado a objetos, y acepta la **herencia múltiple**, lo que significa que una clase puede heredar de varias clases.

Antes de entrar en tratar con la herencia múltiple, vamos a usar la **herencia por especialización**.

Este tipo de herencia la usaremos ante problemáticas como:

"Tengo dos objetos que se comportan más o menos de la misma forma, pero con algunas diferencias."
Solución: "Voy a crear una clase para describir los comportamientos idénticos, y a continuación, dos subclases que hereden de ella y, en cada una, incorporaré los comportamientos diferentes."

```
Punto_herencia_especializacion.py

class Punto2D(Punto): 2 usages  Olga M.Moreno Martín
    """Representa un punto en el plano"""

    def __init__(self, x, y):  Olga M.Moreno Martín
        """Método de inicialización de un punto en el plano"""
        super().__init__(x, y, z: 0)

    def __str__(self):  Olga M.Moreno Martín
        return f"Punto2D ({self.x}, {self.y})"

p = Punto2D( x: 1, y: 2)
p += Punto2D( x: 3, y: 0)
print(p)
```

HERENCIA

Y la segunda problemática que da lugar a la herencia, es la **programación por composición**:

"Tengo comportamientos que se encuentran en varios objetos diferentes, pero ninguno es semejante." Solución: Voy a crear componentes muy básicos, cada uno que describa un comportamiento, y voy a definir a continuación mis objetos como combinaciones de varios comportamientos.

Retomando nuestro ejemplo, los operadores y la manera en la que el punto evoluciona en el espacio o el plano es algo propio del punto. Por el contrario, es posible aislar uno de los comportamientos: el hecho de que pueda visualizarse. De hecho, muchos objetos de todo tipo pueden necesitar mostrarse. De modo que es posible hacer lo que llamaremos un Mixin:

Olga M. Moreno Martín - Javier Herrero Porras

```
Punto_herencia_componentes.py

class MostrableMixin: 1 usage  Olga M.Moreno Martín *
    # Clase mixin para mostrar objetos de forma personalizada
    str_format = "PrettyPrintableObject"
    def __str__(self):  Olga M.Moreno Martín *
        #Representación automática de un objeto,
        # basado en el uso de una cadena de formateo que es un atributo de la clase
        return self.str_format.format(self=self)

class NombreAutomaticoMixin: 3 usages  Olga M.Moreno Martín
    ordinal = 65
    def __init__(self):  Olga M.Moreno Martín
        self.letra = chr(NombreAutomaticoMixin.ordinal)
        NombreAutomaticoMixin.ordinal += 1

class Punto(MostrableMixin, NombreAutomaticoMixin): 6 usages  Olga M.Moreno Martín
    """Representa un punto en el espacio"""
    str_format = "Punto {self.letra} ({self.x}, {self.y}, {self.z})"
```

```
Punto_herencia_componentes.py

class Punto2D(Punto): 1 usage  Olga M.Moreno Martín *
    """Representa un punto en el plano"""

    str_format = "Point2D {self.letra} ({self.x}, {self.y})"

    def __init__(self, x, y):  Olga M.Moreno Martín
```

HERENCIA

Habr  ocasiones en las que necesitemos que una clase use m todos de otras, como hemos dicho, en Python, la herencia puede ser m ltiple. Veamos un ejemplo ilustrativo:

```
Cocodrilo_herencia_multiple.py
class Terrestre(): 2 usages new *
    def __init__(self, patas): #Definimos los Atributos new *
        self.patas = patas
class Acuatico(): 2 usages new *
    def __init__(self, aletas): #Definimos los Atributos new *
        self.aletas = aletas
class Cocodrilo(Terrestre, Acuatico): 2 usages new *
    def __init__(self, patas, aletas, longitud): new *
        Terrestre.__init__(self, patas) #Llamada al primer
        Acuatico.__init__(self, aletas) #Llamada al segundo
        self.longitud = longitud
```

```
004.py
from Cocodrilo_herencia_multiple import Cocodrilo

def main(): 1 usage new *
    coco = Cocodrilo( patas: 4, aletas: 0, longitud: "5 metros")
    print(f"El cocodrilo tiene {coco.patas} patas, "
          f"{coco.aletas} aletas y mide {coco.longitud}.")

if __name__ == "__main__":
    main()
```

ENCAPSULACIÓN

En Python, los métodos privados se definen con un doble guion bajo al inicio del nombre del método.

Esto indica que el método no debe ser accedido directamente desde fuera de la clase.

Sin embargo, Python no impide completamente el acceso a estos métodos, solo los hace menos accesibles.

Para establecer **un método privado** **debe ir precedido de __** (2 guiones bajos)

Realmente no es privado, refleja intención de privado. Python renombra el método anteponiendo el nombre de la clase (name mangling).

```
005.py
class Ejemplo: 1 usage
    def publico(self): 1 usage
        print("Publico")
    def __privado(self):
        print("Privado")

def main(): 1 usage
    ej = Ejemplo()
    ej.publico()
    # ej.__privado()
    # Esto generará un error porque __privado es un metodo privado
    ej.Ejemplo__privado()
    # Esto funciona porque se accede al metodo privado usando su nombre mangled

if __name__ == "__main__":
    main()
```

ENCAPSULACIÓN: ATRIBUTOS GETTERS, SETTERS Y DELETTERS

Aunque Python nos permite la implementación de nuestras operaciones como cualquier lenguaje de POO, un getter es un método que permite obtener el valor de un atributo privado de una clase. Tradicionalmente, se define como un método normal (por ejemplo, `get_dia()`).

Una property en Python es una forma más elegante de definir **getters, setters y deleters** usando el decorador **@property**. Permite acceder a los atributos como si fueran públicos, pero internamente usa métodos para controlar el acceso y la validación.

Diferencia clave:

- El getter es solo el método para obtener el valor.
- La property es un mecanismo que puede incluir getter, setter y deleter, y permite acceder al atributo como si fuera público, pero con control interno.

Es recomendable usar properties en lugar de getters cuando deseamos proporcionar una interfaz más limpia y fácil de usar para los atributos de una clase. Algunas situaciones específicas son:

- **Acceso similar a atributos públicos:** Las properties permiten acceder a los valores como si fueran atributos públicos, pero con control interno. Esto mejora la legibilidad del código.
- **Encapsulamiento con simplicidad:** Si necesitas agregar lógica al acceso o modificación de un atributo (como validaciones), las properties lo hacen sin cambiar la forma en que se accede al atributo.
- **Compatibilidad con código existente:** Si decides cambiar un atributo público a privado y agregar lógica de acceso, puedes usar properties sin modificar el código que ya accede al atributo.

MÉTODOS ESPECIALES

`__init__(self, args)`: Método llamado después de crear el objeto para inicializar los valores de los atributos de instancia

`__new__(cls, args)`: Método constructor de clase, usado para inicializar atributos de clase o para implementar Singleton

`__del__(self)`: Método llamado cuando el objeto va a ser borrado.

`__str__(self)`: Método llamado para crear una cadena de texto que represente a nuestro objeto (`toString()`).

`__len__(self)`: Indica la longitud del objeto

`__lt__`, `__gt__`, `__le__`, `__ge__`, `__eq__`, `__ne__`:

Definen los operadores relacionales de comparación entre objetos: `<` `>` `<=` `>=` `==` y `!=`.

`__hash__(self)`: Método llamado para devolver el hash del objeto, normalmente, deriva de sus atributos.

`__clone__(self)`: Método llamado para crear un objeto idéntico al utilizado como parámetro

ENCAPSULACIÓN Y MÉTODOS ESPECIALES

```
class Fecha: 2 usages new *

def __init__(self, dia, mes, anio): new *
    """Inicializa la fecha con día, mes y año"""
    self._dia = dia
    self._mes = mes
    self._anio = anio

@property 7 usages new *
def dia(self):
    """Getter para el día"""
    return self._dia

@property 7 usages new *
def mes(self):
    """Getter para el mes"""
    return self._mes

@property 7 usages new *
def anio(self):
    """Getter para el año"""
    return self._anio
```

```
@dia.setter new *
def dia(self, valor):
    """Setter para el día"""
    if 1 <= valor <= 31:
        self._dia = valor
    else:
        raise ValueError("Día debe estar entre 1 y 31")

@mes.setter new *
def mes(self, valor):
    """Setter para el mes"""
    if 1 <= valor <= 12:
        self._mes = valor
    else:
        raise ValueError("Mes debe estar entre 1 y 12")

@anio.setter new *
def anio(self, valor):
    """Setter para el año"""
    if valor >= 0:
        self._anio = valor
    else:
        raise ValueError("Año no puede ser negativo")
```

```
@dia.deleter new *
def dia(self):
    """Elimina el día"""
    del self._dia

@mes.deleter new *
def mes(self):
    """Elimina el mes"""
    del self._mes

@anio.deleter new *
def anio(self):
    """Elimina el año"""
    del self._anio

def __str__(self): new *
    """Representación de la fecha como cadena de caracteres"""
    return f"{self.dia:02d}/{self.mes:02d}/{self.anio}"

def __eq__(self, otra): new *
    """Compara dos fechas para ver si son iguales"""
    if isinstance(otra, Fecha):
        return (self.dia == otra.dia and
                self.mes == otra.mes and
                self.anio == otra.anio)
    return False

def __clone__(self): new *
    """Crea una copia de la fecha"""
    return Fecha(self.dia, self.mes, self.anio)

def __hash__(self): new *
    """Devuelve un hash de la fecha"""
    return hash((self.dia, self.mes, self.anio))
```

La instrucción **raise** en Python se utiliza para lanzar una excepción.

`raise ValueError("Año no puede ser negativo")`

Detiene la ejecución y muestra el mensaje de error si el valor del año es negativo.

ENCAPSULACIÓN Y MÉTODOS ESPECIALES

```
006.py
from Fecha import Fecha

def main():
    """usage new"""
    # Crear una instancia de Fecha
    fecha = Fecha(dia: 15, mes: 8, anio: 2023)

    # Imprimir la fecha
    print(f"Fecha inicial: {fecha.dia}/{fecha.mes}/{fecha.anio}")

    # Modificar la fecha
    fecha.dia = 20
    fecha.mes = 12
    fecha.anio = 2024

    # Imprimir la fecha modificada
    print(f"Fecha modificada: {fecha.dia}/{fecha.mes}/{fecha.anio}")

    # Eliminar el día
    del fecha.dia
```

```
006.py
# Intentar imprimir el día eliminado (esto generará un error)
try:
    print(f"Día eliminado: {fecha.dia}")
except AttributeError as e:
    print(f"Error: {e}")

# Crear una copia de la fecha
fecha_copia = fecha.__clone__()
print(f"Fecha copia: {fecha_copia.dia}/{fecha_copia.mes}/{fecha_copia.anio}")

# Comparar dos fechas
fecha2 = Fecha(dia: 20, mes: 12, anio: 2024)
if fecha == fecha2:
    print("Las fechas son iguales.")
else:
    print("Las fechas son diferentes.")

# Imprimir el hash de la fecha
print(f"Hash de la fecha: {hash(fecha)}")

if __name__ == '__main__':
    main()
```

MÉTODOS ESPECIALES: PATRÓN SINGLETON

```
Singleton.py
class EjemploSingleton(): 4 usages new *
    __instancia = None
    def __init__(self): new *
        # Constructor privado para evitar la creación de instancias
        if EjemploSingleton.__instancia is not None:
            raise Exception("Esta clase es un singleton, no se puede crear otra instancia.")
    def __new__(cls): new *
        # Metodo para controlar la creación de instancias
        if cls.__instancia is None:
            cls.__instancia = super(EjemploSingleton, cls).__new__(cls)
        return cls.__instancia

a = EjemploSingleton()
b = EjemploSingleton()
print (a is b) # True , misma referencia a objeto
```

MÉTODOS ESPECIALES: DECORADOR TOTAL ORDERING

```
007.py
from functools import total_ordering

@total_ordering 3 usages Olga M. Moreno Martín
class Cuadrado:
    def __init__(self, lado): Olga M. Moreno Mar
        self.lado = lado
    def __lt__(self, otro): Olga M. Moreno Martín
        return self.lado < otro.lado
    def __eq__(self, otro): Olga M. Moreno Martín
        return self.lado == otro.lado

    def __str__(self): Olga M. Moreno Martín
        return f"Cuadrado de lado {self.lado}"
```

El decorador **@total_ordering** permite que se definan automáticamente todos los operadores de comparación a partir de la definición de uno cualquiera y el de igualdad.

Como resultado se puede utilizar el operador ">=" aunque no se haya definido

```
007.py
from Cuadrado import Cuadrado

def main(): 1 usage Olga M. Moreno Martín
    # Crear una instancia de Cuadrado
    cuadrado = Cuadrado(5)
    # Imprimir el cuadrado
    print(f"Cuadrado inicial: {cuadrado}")
    # Modificar el lado del cuadrado
    cuadrado.lado = 10
    # Imprimir el cuadrado modificado
    print(f"Cuadrado modificado: {cuadrado}")
    # Comparar dos cuadrados
    cuadrado2 = Cuadrado(7)
    if cuadrado == cuadrado2:
        print("Los cuadrados son iguales.")
    elif cuadrado >= cuadrado2:
        print("El primer cuadrado es mayor que el segundo.")
    else:
        print("El primer cuadrado es menor que el segundo.")

if __name__ == '__main__':
    main()
```

EXCEPCIONES

Para capturar excepciones en Python, tenemos la famosa cláusula **try**, pero en esta ocasión en vez de ir con **catch**, nos encontramos con **except**, **else** y **finally**.

Cuya aplicación hace exactamente lo que cada palabra reservada nos hace intuir. Por cuál, contaremos con 4 palabras y por tanto, acciones que nos ayudarán a gestionar adecuadamente nuestras excepciones.

Debemos recalcar que el uso de **try** y **except** es de obligatoriedad, mientras que **else** y **finally** son opcionales.

```
008.py
#Excepciones
# Ejemplo de manejo de excepciones en Python
def dividir(a, b): 1 usage new *
    if b == 0:
        raise ZeroDivisionError("No se puede dividir por cero")
    return a / b

def main(): 1 usage new *
    try:
        resultado = dividir( a: 10, b: 0)
        print(f"Resultado: {resultado}")
    except ZeroDivisionError as e:
        print(f"Error: {e}")
    else:
        print("División exitosa")
    finally:
        print("Fin del programa")

if __name__ == '__main__':
    main()
```

EXCEPCIONES

Podemos gestionar con **varios except**, o con **uno solo y varios parámetros**, la captura de excepciones con el bloque try.

Investiga:

¿Cuál crees que es mejor práctica y por qué?

```
009.py
# Capturando varias excepciones con un bloque try
try:
    x = int("4eK")
    print(no existe)
except NameError as e:
    print(f"Error de nombre: {e}")
except ValueError as e:
    print(f"Error de valor: {e}")
# Capturando varias excepciones con un bloque try
try:
    print(no existe)
    x = int("4eK")
except (NameError, ValueError) as e:
    print(f"Error: {e}")
```

EXCEPCIONES: CLASES DE EXCEPCIÓN PERSONALIZADAS

```
010.py
# Clase MyException
class MyException(Exception): 2 usages new *
    def __init__(self, mensaje): new *
        super().__init__(mensaje)
        self.mensaje = mensaje
    def __str__(self): new *
        return f"MyException: {self.mensaje}"

# Clase que lanza una excepción personalizada
class ExceptionLauncher: 1 usage new *
    def lanzar_excepcion(self): 1 usage new *
        raise MyException("Esta es una excepción personalizada")

# Clase que maneja excepciones
class ExceptionManager: 1 usage new *
    def manejar_excepcion(self): 1 usage new *
        try:
            lanzador = ExceptionLauncher().lanzar_excepcion()
        except MyException as e:
            print(f"Se ha capturado una excepción: {e}")
```

```
010.py
# Clase principal que ejecuta el programa para que podamos ver la prueba
class main(): 1 usage new *
    def exec(self): 1 usage new *
        manejador = ExceptionManager()
        manejador.manejar_excepcion()

# Prueba del programa
if __name__ == "__main__":
    programa = main().exec()
```

EXCEPCIONES

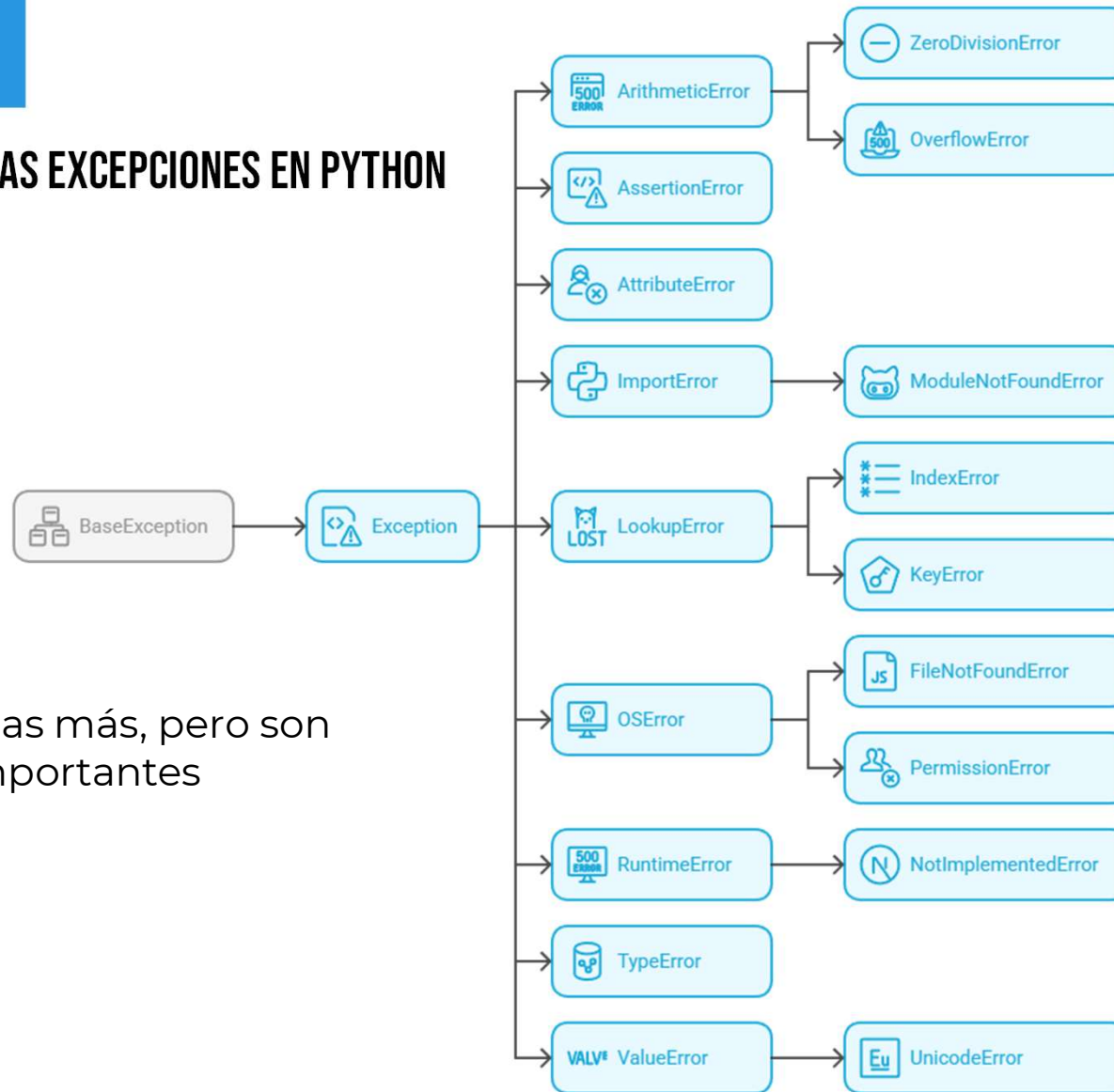
También podemos hacerlas más manejables:

```
MyError.py

class MyError(Exception): 2 usages new *
    def __init__(self, valor): new *
        self.valor = valor
    def __str__(self): new *
        return "Error: the value is " + str(self.valor)

try:
    resultado = 30
    if resultado > 20:
        raise MyError(resultado)
except MyError as e:
    print(e)
```

JERARQUÍA DE LAS EXCEPCIONES EN PYTHON



Hay muchas más, pero son las más importantes

¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON



django

UD4:

ACCESO A DATOS

—“Si torturas los datos, lo confesarán todo.”
— *Ronald Coase.*

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

ÍNDICE

- ▶ **Ficheros**
- ▶ **Ficheros CSV**
- ▶ **Acceso a Base de Datos**
- ▶ **Gestión de Transacciones**
- ▶ **Operaciones CRUD**
- ▶ **Operaciones con Base de Datos**
- ▶ **Test - CRUD**

FICHEROS EN PYTHON (ABRIR)

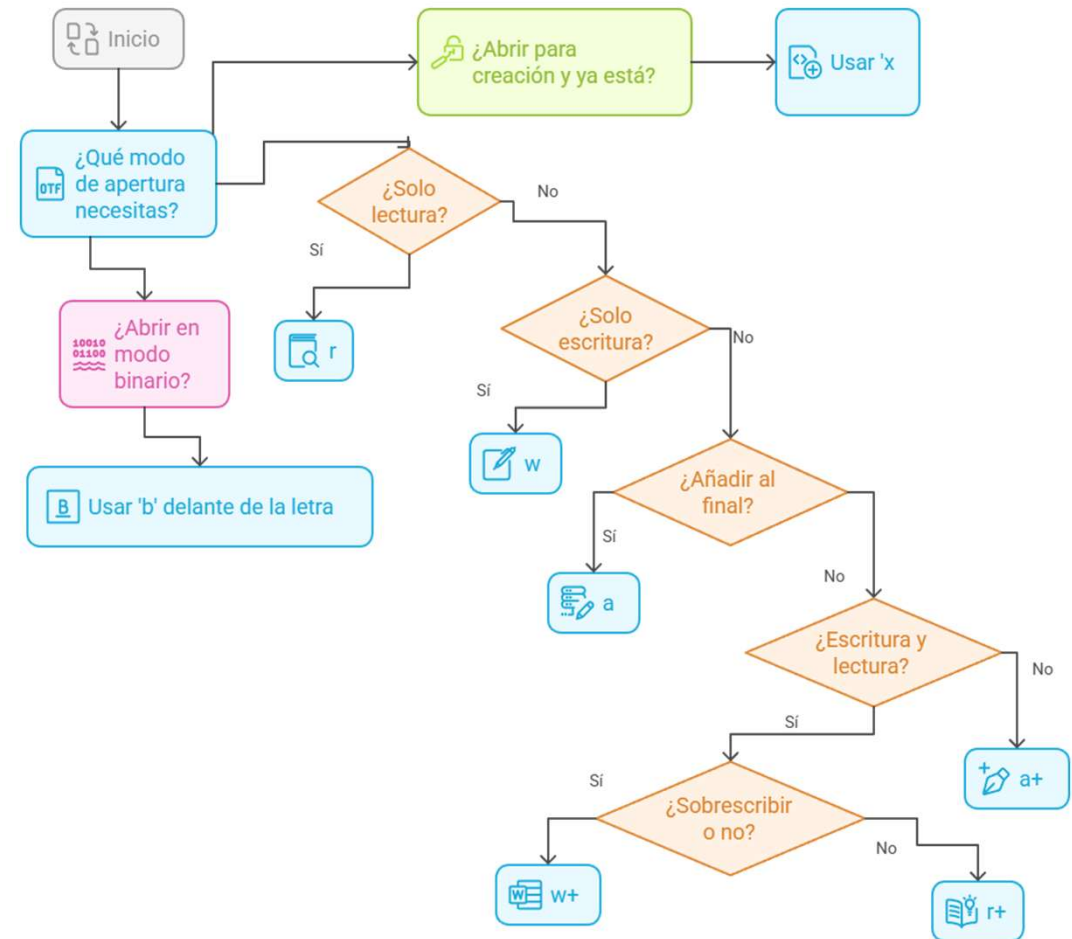
Un fichero es un objeto file

`file= open(path,modoAcceso,tamBuffer, encoding)`

Path: relativo o absoluto

ModoAcceso: r,w,x,a,b,t,+ `f=open("archivo.txt","w")`

Modo	Descripción
'r'	Leer (por defecto). El archivo debe existir.
'w'	Escribir (sobrescribe si existe).
'a'	Añadir al final del archivo.
'x'	Crear nuevo archivo (falla si ya existe).
'b'	Modo binario (usar junto con r/w/a: ej. 'rb').
'+'	Lectura y escritura (ej. 'r+', 'w+').



FICHEROS PYTHON (LEER)

Método	Descripción
f.read()	Lee todo el contenido como string
f.readline()	Lee una sola línea
f.readlines()	Lee todas las líneas como lista

completo = f.read()

parte = f2.read(512) # Leer 512 caracteres (modo texto)

```
002.py
# Lectura de un fichero de texto línea a línea
dirFichero = './fichero_leer.txt'
reader = open(dirFichero, 'r')
for line in reader:
    print (line)
reader.close()
```

```
004.py
#abrir fichero con autocierre y codificación leyendo línea a línea
with open('fichero.txt', encoding='utf-8') as f:
    for linea in f:
        print(linea, end='')
```

```
004.py
#Como el código anterior es muy útil
#podemos utilizarlo y crear una función reutilizable
def lineas_de_fichero(file:str,encoding='utf-8') -> list[str]:
    with open(file,encoding=encoding) as f:
        lineas_de_fichero = [linea.rstrip('\n')
                           for linea in f]
    return lineas_de_fichero
```

Buenas prácticas:

- Evitar read() en archivos muy grandes
- Leer línea por línea si es necesario

FICHEROS EN PYTHON (ESCRITURA)

Métodos: write, writelines

Movimiento del puntero: seek(desplazamiento[,posicion])

tell(): nº bytes desde el principio

Valor	Posición
0	Principio
1	Posición actual
2	Final

```
005.py
# Lectura y escritura simultáneas
with open("miarchivo.txt", "r+") as f:
    datos = f.read()
    f.seek(0)
    f.write("Encabezado añadido\n")
#Añade al principio del archivo sin borrarlo
```

```
005.py
#Escritura en un archivo
with open('f_e.txt', mode='w', encoding='utf-8') as f:
    f.write('Este texto se almacenará en el f.')
#Comprobemos si se ha realizado la escritura correctamente:
with open('f_e.txt', encoding='utf-8') as f:
    for linea in f:
        print(linea, end='')
```

```
005.py
# Diseñamos igualmente la función reutilizable write.
def write(file:str,texto:str) -> None: new *
    with open(file, "w", encoding='utf-8') as f:
        f.write(texto)
```

FICHEROS EN PYTHON (CIERRE)

Métodos: close

Al finalizar el proceso de un fichero es necesario indicar que el recurso queda disponible y para ello debemos cerrar el fichero.

Truco: Si usamos la partícula with al abrir el fichero, se cerrará automáticamente.

```
003.py
#Escritura de una lista en un fichero
lineas = ['Uno', 'Dos', 'Tres']
dirFichero = './fichero_escribir.txt'
fichero = open(dirFichero, 'w')
for l in lineas:
    fichero.write(l + '\n')
fichero.close()
```

```
001.py
import os

# Comprobar si un archivo existe antes de abrirlo
if os.path.exists("archivo.txt"):
    with open("archivo.txt", "r") as f:
        print(f.read())
else:
    print("El archivo no existe.")
```

```
006.py
with open("imagen.jpg", "rb") as f:
    contenido = f.read()

#Abre un archivo binario, útil para imágenes, PDF, etc.
#OJO con read() si es un archivo grande
```

FICHEROS CSV

Un tipo de fichero de texto que usaremos mucho es el llamado formato CSV (por Comma-Separated Values). Estos ficheros se utilizan para almacenar datos de tipo tabular, al estilo de una hoja de cálculo. Son muy utilizados en el ámbito del análisis de datos.

```
009.py
import csv
#with open("datos_2.txt",encoding=encoding) as f:
#    lector = csv.reader(f, delimiter=delimiter)
#    lineas = [linea for linea in lector]
#Diseñamos la función líneas csv para reutilizar este código.
def líneas_de_csv(file:str, delimiter:str="," ,encoding='utf-8')-> list[list[str]]:
    with open(file,encoding= encoding) as f:
        lector = csv.reader(f, delimiter = delimiter)
        lineas_de_fichero = [linea for linea in lector]
    return lineas_de_fichero
```

EJERCICIOS

Dado el fichero datos.txt cuyo contenido es:

```
0,0,1,2
0
-1,2
14,3,7,8

0,4,0,9,2,0,1,2
1,0,7,6
```

Ej1: Contar las líneas que tiene el fichero.

Ej2: Contar cuántas líneas vacías hay en el fichero datos.txt

Ej3: Contar cuántos números hay el fichero y sumarlos, mostrar ambos resultados.

ACCESO A BASE DE DATOS (DRIVER)

Para establecer conexión con una base de datos desde Python es necesario utilizar el driver específico

•El paquete que proporciona el driver habrá que instalarlo como librería local (comando **pip**) y después importarlo desde el programa Python

Algunos de los drivers son:

- pyodbc – Driver ODBC
- pymssql – Driver para SQL Server
- psycopg2 – Driver para Postgresql
- **mysqlclient – Driver para MySQL**
- sqlite3 – Driver para SQLite3

ACCESO A BASE DE DATOS (CONEXIÓN)

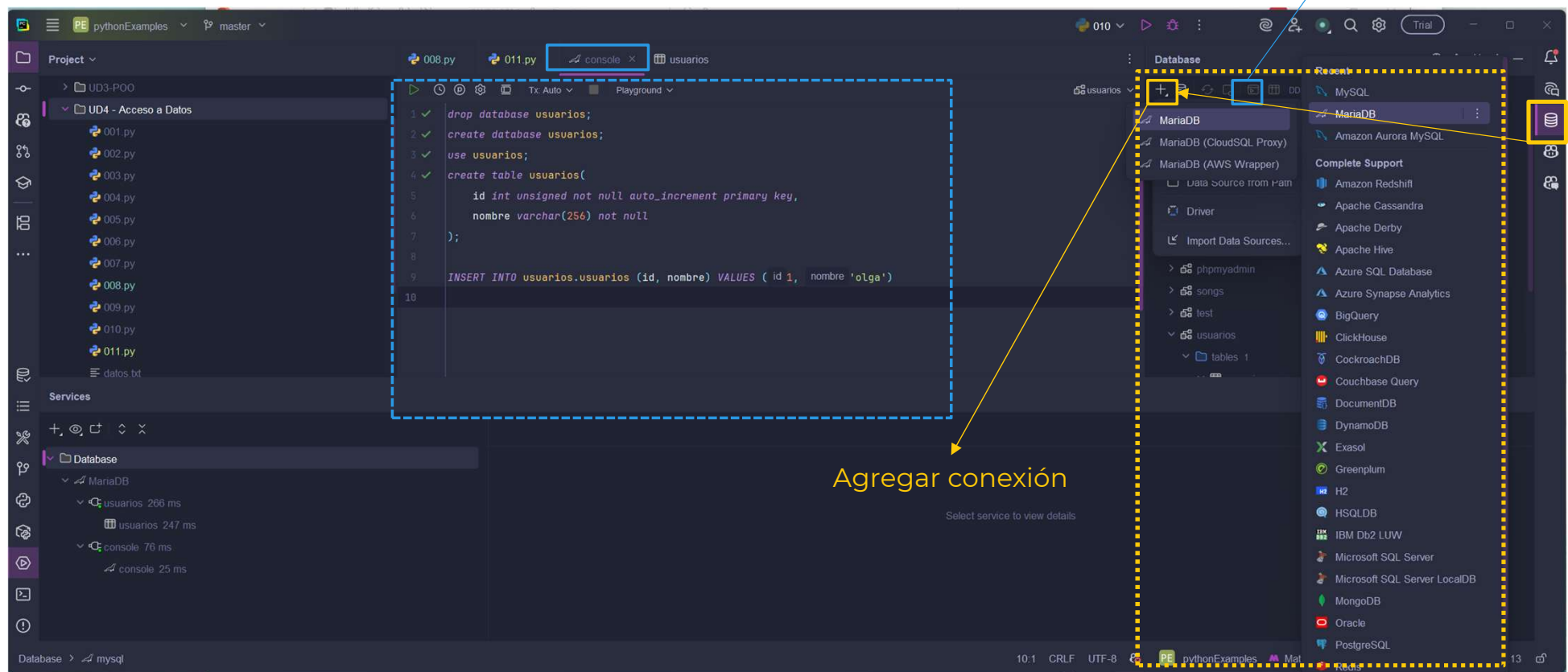
Para crear una conexión con la base de datos hay que tener 2 aspectos en cuenta:

- Se debe importar el paquete correspondiente
- Se deben establecer los parámetros de conexión que se necesiten: usuario, password, localización del servidor postgresql, puerto,...

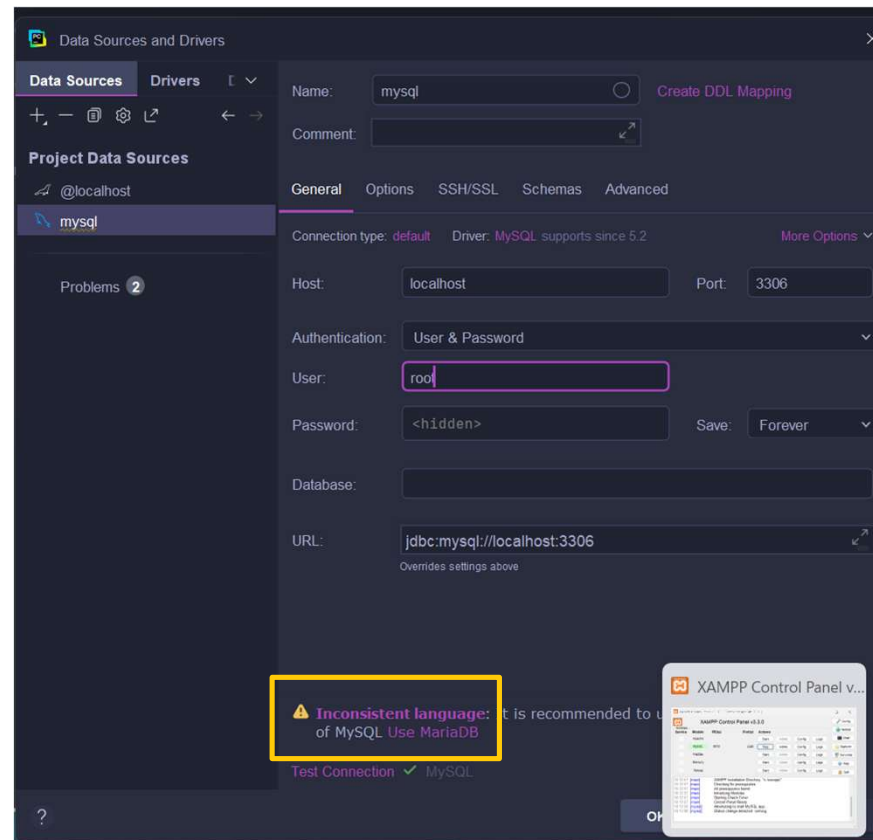
```
007.py
import MySQLdb
import logging
#logging es para generar logs - Buena práctica
def get_connection(): 1 usage
    """
    Establece una conexión con la base de datos MySQL.
    Returns:
    | conn (MySQLdb.connections.Connection): Objeto de conexión a la base de datos.
    Raises:
    | MySQLdb.DatabaseError: Si hay un error en la conexión.
    """
    try:
        conn = MySQLdb.connect(
            host='localhost',
            user='usuario',
            passwd='',
            db='mi_basededatos',
            charset='utf8mb4'
        )
        print('Connection established')
        return conn
    except MySQLdb.DatabaseError as e:
        #También podemos usar Error como clase general
        logging.error(f"Error al conectar con la base de datos: {e}")
        raise

get_connection()
```

INTEGRAR CON PYCHARM



¿Y POR QUÉ NO CON MYSQL? PODEMOS... PERO ¿ESTÁS USANDO XAMPP?



Olga M. Moreno Martín - Javier Herrero Porras

SCRIPTS DE CREACIÓN DE BASE DE DATOS

Vamos a crear nuestra base de datos, que va a ser muy sencilla:

```
usuarios.sql

drop database usuarios;
create database usuarios;
use usuarios;
create table usuarios(
    id int unsigned not null auto_increment primary key,
    nombre varchar(256) not null
);

INSERT INTO usuarios.usuarios (id, nombre) VALUES ( id 1, nombre 'olga')
```

CONEXIÓN A BASE DE DATOS CON GESTIÓN DE TRANSACCIONES

Creamos la conexión y activamos la opción de **autocommit a false** para gestionar transacciones.

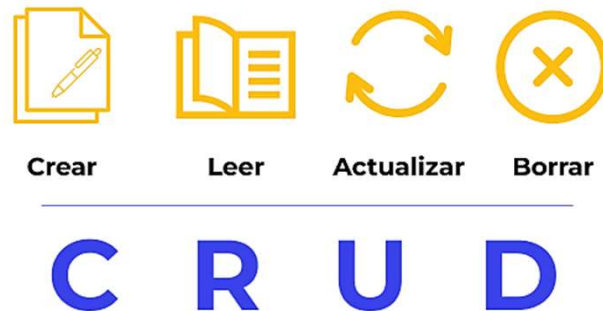
Recordatorio:

- Las transacciones son bloques de operaciones que se ejecutan como una unidad atómica: *todas se completan o ninguna se aplica.*
- Sirven para garantizar la integridad de los datos, especialmente en operaciones críticas como transferencias o actualizaciones múltiples.

```
011.py
import MySQLdb

def get_connection(): 3 usages new *
    """
    Establece y devuelve una conexión a la base de datos MySQL.
    """
    try:
        conn = MySQLdb.connect(
            host='localhost',
            user='root',
            passwd='',
            db='usuarios',
            charset='utf8mb4',
            autocommit=False # Desactivamos autocommit para gestionar transacciones manualmente
        )
        return conn
    except MySQLdb.DatabaseError as e:
        print(f"[ERROR] Conexión fallida: {e}")
        return None
```

INCISO... ¿QUÉ SON LAS OPERACIONES CRUD?



Operación	Acción	Descripción breve
Create (Crear)	+ Insertar datos	Añadir nuevos registros a una tabla o colección.
Read (Leer)	🔍 Consultar datos	Obtener o visualizar datos existentes (con SELECT, por ejemplo).
Update (Actualizar)	✏️ Modificar datos	Cambiar valores de registros existentes (usando UPDATE).
Delete (Eliminar)	🗑️ Borrar datos	Eliminar registros de la base de datos (con DELETE).

INSERCIONES EN BASE DE DATOS - CREATE

```
011.py
def insertar_usuario(nombre): 1 usage new *
    conn = get_connection()
    if conn:
        try:
            cursor = conn.cursor()
            cursor.execute("START TRANSACTION")
            cursor.execute("INSERT INTO usuarios (nombre) VALUES (%s)", (nombre,))
            conn.commit()
            print("✅ Usuario insertado correctamente.")
        except MySQLdb.DatabaseError as e:
            conn.rollback()
            print(f"[ERROR] Error al insertar usuario: {e}")
        finally:
            cursor.close()
            conn.close()
```

Gestión de
transacciones

Evitar
inyección SQL



Crear

C

CONSULTA A LA BASE DE DATOS - LECTURA

```
011.py
def buscar_usuario(nombre): 3 usages new *
    conn = get_connection()
    if conn:
        try:
            cursor = conn.cursor()
            cursor.execute("START TRANSACTION READ ONLY")
            cursor.execute("SELECT id, nombre FROM usuarios WHERE nombre LIKE %s", (f"%{nombre}%",))
            resultados = cursor.fetchall()
            for fila in resultados:
                print(f" ID: {fila[0]}, Nombre: {fila[1]}")
            conn.commit()
        except MySQLdb.DatabaseError as e:
            conn.rollback()
            print(f"[ERROR] Error al buscar usuario: {e}")
        finally:
            cursor.close()
            conn.close()
```

Gestión de transacciones

Evitar inyección SQL



Leer

R

CONSULTA A LA BASE DE DATOS – LECTURA (¿CÓMO PAGINAMOS?)

```
011.py
def buscar_usuarios(limit=5, offset=0): new *
    """
    Recupera una lista paginada de usuarios desde la base de datos.
    Args:
        limit (int): Número de usuarios a mostrar (por página).
        offset (int): Desplazamiento desde el primer registro (para paginación).
    """

    conn = get_connection()
    if conn:
        try:
            cursor = conn.cursor()
            cursor.execute("START TRANSACTION READ ONLY")
            cursor.execute("SELECT id, nombre FROM usuarios ORDER BY id ASC LIMIT %s OFFSET %s", (limit, offset) )
            resultados = cursor.fetchall()
            for fila in resultados:
                print(f"🐦 ID: {fila[0]}, Nombre: {fila[1]}")
            conn.commit()
        except MySQLdb.DatabaseError as e:
            conn.rollback()
            print(f"[ERROR] Error al obtener usuarios: {e}")
        finally:
            cursor.close()
            conn.close()
```



Leer

R

EDICIONES EN BASE DE DATOS - ACTUALIZAR

```
011.py

def actualizar_nombre_usuario(user_id, nuevo_nombre): 1 usage new*
    """
    Actualiza el nombre de un usuario según su ID de forma segura.
    """

    conn = get_connection()
    if conn:
        try:
            cursor = conn.cursor()
            cursor.execute("START TRANSACTION")
            cursor.execute("UPDATE usuarios SET nombre = %s WHERE id = %s", (nuevo_nombre, user_id) )
            conn.commit()
            print("✏ Nombre actualizado correctamente.")
        except MySQLdb.DatabaseError as e:
            conn.rollback()
            print(f"[ERROR] No se pudo actualizar el nombre: {e}")
        finally:
            cursor.close()
            conn.close()
```



ELIMINACIONES EN BASE DE DATOS - BORRAR

```
011.py

def eliminar_usuario_por_id(user_id): 1 usage new *
    conn = get_connection()
    if conn:
        try:
            cursor = conn.cursor()
            cursor.execute("START TRANSACTION")
            cursor.execute("DELETE FROM usuarios WHERE id = %s", (user_id,))
            conn.commit()
            print("✗ Usuario eliminado correctamente.")
        except MySQLdb.DatabaseError as e:
            conn.rollback()
            print(f"[ERROR] No se pudo eliminar el usuario: {e}")
        finally:
            cursor.close()
            conn.close()
```



Borrar



TEST - PROBANDO LAS CONSULTAS

```
011.py

if __name__ == "__main__":
    #Primero ejecutar el fichero usuarios.sql
    buscar_usuario("Olg")
    actualizar_nombre_usuario( user_id: 1, nuevo_nombre: "Olga M.")
    eliminar_usuario_por_id(1)
    buscar_usuario("Olg")
    insertar_usuario("Carlos")
    buscar_usuario("Carlos")
    insertar_usuario("Juan Carlos")
    insertar_usuario("Javier")
    insertar_usuario("Soraya")
    insertar_usuario("Pablo")
    insertar_usuario("Paula")
    insertar_usuario("Olga")

    # Página 1 → primeros 5 usuarios
    buscar_usuarios(limit=5, offset=0)

    # 💡 TRUCO: el offset se puede calcular como limit*(num_pagina -1)

    # Página 2 → siguientes 5 usuarios
    #buscar_usuarios(limit=5, offset=5)
```

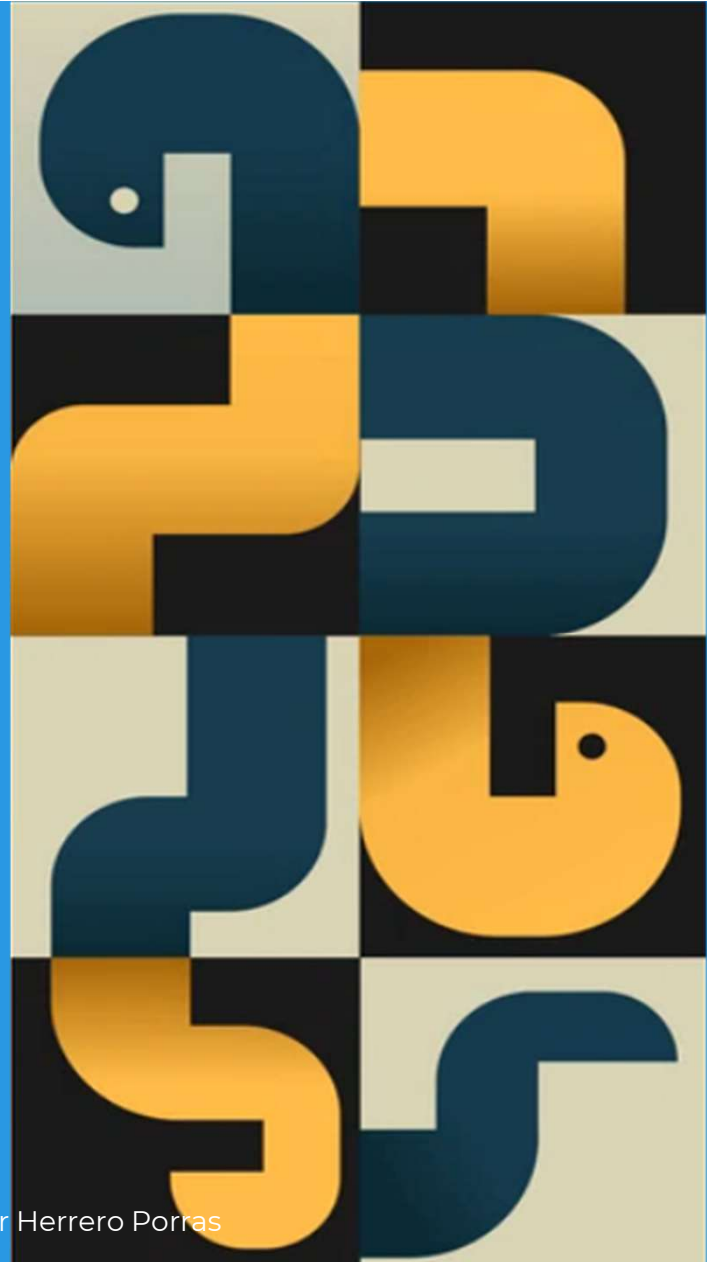
¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON



django

UD5:

BIBLIOTECAS ESTÁNDAR Y CONCEPTOS AVANZADOS

—Y

Olga M. Moreno Martín - Javier Herrero Porras

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>



ÍNDICE

Bibliotecas Estándar

- ▶ Entrada/Salida (I/O) de archivos
- ▶ Procesamiento de texto y datos
- ▶ Matemáticas y estadísticas
- ▶ Fechas y tiempo
- ▶ Estructuras de datos
- ▶ Programación funcional y orientada a objetos
- ▶ Errores y depuración
- ▶ Concurrencia y paralelismo

- ▶ Redes y protocolos de Internet
- ▶ Seguridad y criptografía (básica)
- ▶ Compresión y archivo
- ▶ Desarrollo y compilación

Aspectos Avanzados

Testing

BIBLIOTECAS ESTÁNDAR

Las bibliotecas estándar de Python (o **Standard Library**) son un conjunto de módulos que vienen incluidos con cualquier instalación de Python, sin necesidad de instalar paquetes adicionales. Estas bibliotecas cubren una amplia variedad de funcionalidades que permiten realizar tareas comunes de programación de forma sencilla y eficiente.

Hay una clasificación conforme al uso que hacemos de estas bibliotecas:

1. Entrada/Salida (I/O) de archivos
2. Procesamiento de texto y datos
3. Matemáticas y estadísticas
4. Fechas y tiempo
5. Estructuras de datos
6. Programación funcional y orientada a objetos
7. Errores y depuración
8. Concurrencia y paralelismo
9. Redes y protocolos de Internet
10. Seguridad y criptografía (básica)
11. Compresión y archivo
12. Desarrollo y compilación

ENTRADA/SALIDA Y ARCHIVOS

os – Interacción con el sistema operativo.

io – Manejo de flujos de entrada/salida.

sys – Acceso a variables y funciones del intérprete.

shutil – Operaciones de alto nivel con archivos y directorios.

pathlib – Manipulación orientada a objetos de rutas de archivos.

fileinput – Entrada desde archivos o stdin.

```
001.py
import os

# Obtener el directorio/carpeta actual
print("Directorio actual:", os.getcwd())
# Listar archivos de una carpeta/directorio
print("Contenido:", os.listdir("."))
# Crear un nuevo directorio/carpeta
os.mkdir("nueva_carpeta")

import io

# Crear un flujo de texto en memoria
archivo_virtual = io.StringIO()
archivo_virtual.write("Hola, mundo")
archivo_virtual.seek(0)
print(archivo_virtual.read()) # Imprimir
```

```
001.py
from pathlib import Path
ruta = Path("nueva_carpeta") / "copia_archivo.txt"
print("Ruta completa:", ruta)

# Verificar si existe
if ruta.exists():
    print("El archivo existe")

# Obtener nombre del archivo
print("Nombre del archivo:", ruta.name)

import fileinput

# Leer múltiples archivos como si fuera uno solo
for linea in fileinput.input(files=('archivo1.txt', 'archivo2.txt')):
    print(linea.strip())
```

```
001.py
import sys

# Mostrar argumentos pasados al script
print("Argumentos:", sys.argv)

# Salir del programa con código de salida
# sys.exit(0)

import shutil

# Copiar archivo
shutil.copy( src: "archivo.txt", dst: "copia_archivo.txt")

# Mover archivo
shutil.move( src: "copia_archivo.txt", dst: "nueva_carpeta/copia_archivo.txt")
```

PROCESAMIENTO DE TEXTO Y DATOS

re – Expresiones regulares.

string – Constantes y herramientas relacionadas con cadenas.

textwrap – Formateo de texto.

difflib – Comparación de secuencias (como cadenas).

csv – Lectura y escritura de archivos CSV.

json – Codificación y decodificación en JSON.

xml.etree.ElementTree

– Procesamiento de XML.

```
002.py

#Expresiones regulares
import re
texto = "Mi número es 654-123-456"
patron = r"\d{3}-\d{3}-\d{3}"
resultado = re.search(patron, texto)
if resultado:
    print("Teléfono encontrado:", resultado.group())

import string
# Mostrar caracteres alfabéticos y dígitos
print(string.ascii_letters) # abc...XYZ
print(string.digits)        # 0123456789
# Eliminar puntuación
texto = "¡Hola, mundo!"
limpio = texto.translate(str.maketrans('', '', string.punctuation))
print(limpio) # Hola mundo
```

```
002.py

#Formatear párrafos
import textwrap
parrafo = "Python es un lenguaje de programación muy versátil y fácil de aprender."
formateado = textwrap.fill(parrafo, width=30)
print(formateado)

#Comparación de secuencias
import difflib
texto1 = "Hola mundo"
texto2 = "Hola mundo cruel"
diferencias = difflib.ndiff(texto1.split(), texto2.split())
print("\n".join(diferencias))
```

PROCESAMIENTO DE TEXTO Y DATOS

```
002.py
import csv
# Escritura
with open("datos.csv", mode="w", newline="") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerow(["nombre", "edad"])
    escritor.writerow(["Ana", 23])
    escritor.writerow(["Luis", 30])
# Lectura
with open("datos.csv", mode="r") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)
```

```
002.py
import json
datos = {"nombre": "Carlos", "edad": 28, "activo": True}
# Convertir a JSON y guardar
with open("usuario.json", "w") as f:
    json.dump(datos, f)
# Leer JSON
with open("usuario.json", "r") as f:
    cargado = json.load(f)
    print(cargado)
```

```
002.py
import xml.etree.ElementTree as ET
# Crear estructura XML
root = ET.Element("personas")
persona = ET.SubElement(root, tag="persona", nombre="Lucía")
ET.SubElement(persona, tag="edad").text = "35"
# Escribir archivo
tree = ET.ElementTree(root)
tree.write("personas.xml")
# Leer archivo
tree = ET.parse("personas.xml")
root = tree.getroot()
for persona in root.findall("persona"):
    print("Nombre:", persona.attrib["nombre"])
    print("Edad:", persona.find("edad").text)
```

MATEMÁTICAS Y ESTADÍSTICAS

math– Funciones matemáticas básicas.

cmath – Funciones para números complejos.

decimal – Aritmética decimal de alta precisión.

fractions – Operaciones con fracciones.

random – Generación de números aleatorios.

statistics – Funciones estadísticas básicas.

003.py

```
#Números complejos
import cmath
z = complex(3, 4)
print("Módulo:", abs(z))
print("Fase:", cmath.phase(z))
print("Raíz cuadrada de -1:", cmath.sqrt(-1)) # Resultado: 1j
```

003.py

```
#Matemáticas - cálculo y álgebra
import math
print("Raíz cuadrada de 25:", math.sqrt(25))
print("Seno de 90°:", math.sin(math.radians(90)))
print("Factorial de 5:", math.factorial(5))
print("Constante π:", math.pi)
```

003.py

```
#Trabajar con decimales de alta precisión - Bancos, económicas, etc.
from decimal import Decimal, getcontext
getcontext().prec = 6 # Precisión de 6 cifras decimales
a = Decimal("1") / Decimal("7")
print("Decimal con alta precisión:", a)
```

003.py

```
#Trabajar con fracciones
from fractions import Fraction
f1 = Fraction(1, 3)
f2 = Fraction(2, 5)
resultado = f1 + f2
print("Suma de fracciones:", resultado)
# Resultado: 11/15
```

MATEMÁTICAS Y ESTADÍSTICAS

003.py

```
import random
print("Número aleatorio entre 1 y 10:", random.randint(a: 1, b: 10))
print("Elemento aleatorio:", random.choice(["manzana", "pera", "uva"]))
print("Lista barajada:")
lista = [1, 2, 3, 4]
random.shuffle(lista)
print(lista)
```

003.py

```
import statistics
datos = [1, 2, 3, 4, 4, 5]
print("Media:", statistics.mean(datos))
print("Mediana:", statistics.median(datos))
print("Moda:", statistics.mode(datos))
print("Desviación estándar:", statistics.stdev(datos))
```

FECHAS Y TIEMPO

datetime – Fechas y horas.

time – Funciones de tiempo.

calendar – Calendarios y operaciones relacionadas.

```
004.py
import time

# Hora actual en formato Unix (segundos desde 1970)
timestamp = time.time()

print("Timestamp actual:", timestamp)

# Pausar la ejecución por 2 segundos
print("Esperando...")
time.sleep(2)
print("¡Listo!")

# Formatear hora local
local = time.localtime()

print("Hora local formateada:", time.strftime("%H:%M:%S", local))
```

```
004.py
from datetime import datetime, date, timedelta

# Fecha y hora actual
ahora = datetime.now()
print("Ahora:", ahora)

# Crear una fecha específica
cumple = date(year=2025, month=7, day=9)
print("Cumpleaños:", cumple)

# Sumar días
mañana = ahora + timedelta(days=1)
print("Mañana:", mañana.strftime("%Y-%m-%d %H:%M:%S"))
```

```
004.py
import calendar

# Mostrar el calendario de julio de 2025
print("Calendario julio 2025:")
print(calendar.month(2025, 7))

# Saber si un año es bisiesto
print("¿2024 es bisiesto?:", calendar.isleap(2024))

# Día de la semana (0=lunes, 6=domingo)
dia = calendar.weekday(2025, 7, 9)
print("Día de la semana (9 julio 2025):", calendar.day_name[dia])
```

ESTRUCTURAS DE DATOS

collections – Tipos de datos especializados

heapq – Cola de prioridad (heap).

array – Vectores de datos numéricos.

queue – Colas FIFO y LIFO.

```
005.py
#COLAS DE PRIORIDAD
import heapq
numeros = [5, 3, 8, 1]
heapq.heapify(numeros)
print("Min-heap:", numeros)
# Insertar nuevo elemento
heapq.heappush(*args: numeros, 2)
print("Heap con nuevo elemento:", numeros)
# Extraer el menor
menor = heapq.heappop(numeros)
print("Elemento más pequeño:", menor)
```

```
005.py
#COLECCIONES
from collections import deque, Counter, namedtuple
# deque: cola doble
cola = deque(["a", "b", "c"])
cola.append("d")
cola.appendleft("z")
print("Deque:", cola)
# Counter: contar elementos
letras = Counter("abracadabra")
print("Contador:", letras)
# namedtuple: tupla con nombre
Punto = namedtuple( typename: "Punto", field_names: "x y")
p = Punto(3, 4)
print("Punto:", p.x, p.y)
```

```
005.py
#ARRAYS - VECTORES
import array
# Crear un array de enteros
a = array.array( typecode: 'i', initializer: [1, 2, 3, 4])
a.append(5)
print("Array de enteros:", a.tolist())
```

```
005.py
#COLAS - SEGURAS PARA HILOS
from queue import Queue, LifoQueue
# FIFO Queue
cola = Queue()
cola.put("uno")
cola.put("dos")
print("FIFO:", cola.get(), cola.get())
# LIFO Queue (pila)
pila = LifoQueue()
pila.put("uno")
pila.put("dos")
print("LIFO:", pila.get(), pila.get())
```

PROGRAMACIÓN FUNCIONAL Y ORIENTADA A OBJETOS

functools – Herramientas para programación funcional.

operator – Funciones equivalentes a operadores.

itertools – Iteradores eficientes.

dataclasses – Clases de datos (desde Python 3.7).

```
006.py
#funciones equivalentes a operadores
import operator
a, b = 10, 3
print("Suma:", operator.add(*args: a, b))
print("Multiplicación:", operator.mul(*args: a, b))
print("Mayor que:", operator.gt(*args: a, b))
```

```
006.py
#Funciones de orden superior (las hemos visto)
from functools import reduce, lru_cache
# Reduce: suma de todos los elementos
numeros = [1, 2, 3, 4]
suma = reduce(lambda x, y: x + y, numeros)
print("Suma total:", suma)
# Memoización con lru_cache
@lru_cache(maxsize=None) 2 usages new *
def factorial(n):
    return 1 if n == 0 else n * factorial(n - 1)
print("Factorial(5):", factorial(5))
```

```
006.py
import itertools
# Producto cartesiano
prod = list(itertools.product([1, 2], ['a', 'b']))
print("Producto cartesiano:", prod)
#[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
# Combinaciones
comb = list(itertools.combinations(iterable=[1, 2, 3], r=2))
print("Combinaciones:", comb) #[(1, 2), (1, 3), (2, 3)]
# Ciclo infinito
ciclo = itertools.cycle("AB")
for _ in range(4):
    print(next(ciclo), end=" ") # A B A B
print()
```

```
006.py
#Dataclasses
from dataclasses import dataclass
@dataclass 1 usage new *
class Persona:
    nombre: str
    edad: int
p = Persona("Lucía", 30)
print("Persona:", p.nombre, p.edad)
```

ERRORES Y DEPURACIÓN

logging – Registro de mensajes de log.

warnings – Manejo de advertencias.

traceback – Rastreo de errores.

pdb – Depurador interactivo.

```
007.py
#LOGS
import logging
# Configuración básica
logging.basicConfig(level=logging.DEBUG, format="%(levelname)s - %(message)s")
logging.debug("Mensaje de depuración")
logging.info("Mensaje informativo")
logging.warning("Mensaje de advertencia")
logging.error("Mensaje de error")
logging.critical("Mensaje crítico")
```

```
007.py
#pdb - Depurador interactivo
import pdb
def sumar(a, b): 1 usage new *
    pdb.set_trace()
    return a + b
print("Resultado:", sumar( a: 2, b: 3))
```

```
007.py
#Rastreo de errores
import traceback
try:
    1 / 0 # Error intencional
except ZeroDivisionError:
    print(f"Ocurrió un error:{traceback.print_exc()}")
```

```
007.py
#WARNINGS
import warnings
# Generar una advertencia
warnings.warn( message: "Esta es una advertencia personalizada", UserWarning)
```

CONCURRENCIA Y PARALELISMO

threading – Programación con hilos.

```
008.py

import threading
import time

#Programación con hilos

def tarea(nombre): 4 usages
    print(f"{nombre} ha comenzado")
    time.sleep(2)
    print(f"{nombre} ha terminado")

# Crear hilos
hilo1 = threading.Thread(target=tarea, args=("Hilo 1",))
hilo2 = threading.Thread(target=tarea, args=("Hilo 2",))

# Iniciar
hilo1.start()
hilo2.start()

# Esperar a que terminen
hilo1.join()
hilo2.join()
```

CONCURRENCIA Y PARALELISMO

multiprocessing –
Procesos paralelos.

```
008.py

#Procesos paralelos
import multiprocessing
import time

def proceso(nombre):
    print(f"{nombre} iniciado")
    time.sleep(2)
    print(f"{nombre} finalizado")

if __name__ == "__main__":
    p1 = multiprocessing.Process(target=proceso, args=("Proceso 1",))
    p2 = multiprocessing.Process(target=proceso, args=("Proceso 2",))
    p1.start()
    p2.start()
    p1.join()
    p2.join()

# Importante:
# multiprocessing requiere proteger el bloque if __name__ == "__main__": para evitar errores en Windows.
```

CONCURRENCIA Y PARALELISMO

asyncio – Programación
asíncrona (event loop).

```
008.py

import asyncio

async def tarea(nombre): 2 usages
    print(f"{nombre} ha comenzado")
    await asyncio.sleep(2)
    print(f"{nombre} ha terminado")

async def main():
    await asyncio.gather(
        tarea("Tarea 1"),
        tarea("Tarea 2")
    )
    asyncio.run(main())
```

CONCURRENCIA Y PARALELISMO

Programación con hilos

Ideal para tareas enlazadas a E/S, pero puede sufrir del bloqueo global del intérprete (GIL).

Programación con procesos paralelos

Adecuado para tareas enlazadas a CPU, evitando el GIL, pero con mayor sobrecarga.

Programación asíncrona

Eficiente para tareas concurrentes sin hilos ni procesos, pero requiere un manejo cuidadoso de las devoluciones de llamada.



REDES Y PROTOCOLOS DE INTERNET

socket – Comunicación a bajo nivel entre computadoras.

http.client, http.server – HTTP cliente/servidor.

```
009.py
import socket
# Cliente TCP
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("example.com", 80))
# Enviar una solicitud HTTP simple
cliente.sendall(b"GET / HTTP/1.1\r\nHost: example.com\r\n\r\n")
respuesta = cliente.recv(4096)
print(respuesta.decode())
cliente.close()
```

```
009.py
import http.client
conn = http.client.HTTPSConnection("www.example.com")
conn.request(method="GET", url="/")
respuesta = conn.getresponse()
print("Status:", respuesta.status)
print("Contenido:", respuesta.read().decode())
conn.close()
```

```
009.py
# Ejecuta este script en terminal
from http.server import SimpleHTTPRequestHandler, HTTPServer
puerto = 8000
servidor = HTTPServer(server_address=("localhost", puerto), SimpleHTTPRequestHandler)
print(f"Servidor en http://localhost:{puerto}")
servidor.serve_forever()
```

REDES Y PROTOCOLOS DE INTERNET

urllib – Manejo de URLs

smtplib – Envío de correos por SMTP.

imaplib, poplib – Lectura de correos electrónicos.

ftplib – Conexiones FTP.

```
009.py
import urllib.request
import urllib.parse

url = "https://httpbin.org/get"
respuesta = urllib.request.urlopen(url)
print("Código HTTP:", respuesta.getcode())
print("Contenido:", respuesta.read().decode())
```

```
009.py
import smtplib

# ATENCIÓN: para funcionar necesitamos una cuenta SMTP válida
remite = "tucorreo@example.com"
destinatario = "destino@example.com"
mensaje = "Subject: Prueba SMTP\n\nEste es un mensaje enviado desde Python."

with smtplib.SMTP(host="smtp.example.com", port=587) as servidor:
    servidor.starttls()
    servidor.login(user="usuario", password="contraseña")
    servidor.sendmail(remite, destinatario, mensaje)
```

```
009.py
import imaplib

# ATENCIÓN: solo si tienes servidor IMAP real
mail = imaplib.IMAP4_SSL("imap.example.com")
mail.login(user="usuario", password="contraseña")
mail.select("inbox")

status, mensajes = mail.search(charset=None, criteria="ALL")
print("IDs de mensajes:", mensajes)

mail.logout()
```

```
009.py
from ftplib import FTP

# FTP público de prueba (anónimo)
ftp = FTP("ftp.debian.org")
ftp.login()
ftp.cwd("debian")
ftp.retrlines("LIST")
ftp.quit()
```

SEGURIDAD Y CRIPTOGRAFÍA (BÁSICA)

hashlib – Algoritmos hash
(MD5, SHA-1, SHA-256,
etc.).

hmac – HMAC (clave +
hash).

secrets – Generación de
contraseñas seguras.

```
010.py

import hashlib

# Ejemplo de uso de hashlib para crear hashes de un texto
texto = "hola mundo".encode()

# Hash MD5
md5 = hashlib.md5(texto).hexdigest()
print("MD5:", md5)

# Hash SHA-1
sha1 = hashlib.sha1(texto).hexdigest()
print("SHA-1:", sha1)

# Hash SHA-256
sha256 = hashlib.sha256(texto).hexdigest()
print("SHA-256:", sha256)

# Hash SHA-512
sha512 = hashlib.sha512(texto).hexdigest()
print("SHA-512:", sha512)
```

```
010.py

import hmac
import hashlib

# Ejemplo de uso de HMAC con SHA-256
mensaje = b"mensaje secreto"
clave = b"clave_compartida"
firma = hmac.new(clave, mensaje, hashlib.sha256).hexdigest()
print("HMAC (SHA-256):", firma)
```

```
010.py

import secrets
import string

# Token hexadecimal aleatorio
print("Token seguro:", secrets.token_hex(16))

# Contraseña segura de 12 caracteres
alfabeto = string.ascii_letters + string.digits
contrasena = ''.join(secrets.choice(alfabeto) for _ in range(12))
print("Contraseña segura:", contrasena)
```

COMPRESIÓN Y ARCHIVO

zipfile, tarfile –
Lectura/escritura de
archivos comprimidos.

gzip, bz2, lzma –
Compresión/descompresión.

```
011.py
import zipfile
# Crear un ZIP
with zipfile.ZipFile( file: "ejemplo.zip", mode: "w") as zipf:
    zipf.writestr( zinfo_or_arcname: "saludo.txt", data: "¡Hola desde un archivo zip!")

# Leer contenido del ZIP
with zipfile.ZipFile( file: "ejemplo.zip", mode: "r") as zipf:
    print("Contenido del ZIP:", zipf.namelist())
    print("Contenido de saludo.txt:", zipf.read("saludo.txt").decode())
```

```
011.py
import tarfile
# Crear archivo tar
with tarfile.open( name: "ejemplo.tar.gz", mode: "w:gz") as tar:
    tar.add("saludo.txt") # Asegúrate de que exista
# Leer archivo tar
with tarfile.open( name: "ejemplo.tar.gz", mode: "r:gz") as tar:
    print("Archivos en el TAR:", tar.getnames())
```

```
011.py
import gzip
# Comprimir texto
with gzip.open( filename: "archivo.txt.gz", mode: "wt") as f:
    f.write("Este es un texto comprimido con gzip.")
# Leerlo
with gzip.open( filename: "archivo.txt.gz", mode: "rt") as f:
    print("Contenido gzip:", f.read())
```

DESARROLLO Y COMPILACIÓN

argparse – Análisis de argumentos de línea de comandos.

subprocess – Ejecución de comandos externos

typing – Tipado estático (anotaciones).

compile, eval, exec – Evaluación dinámica de código.

```
013.py
import subprocess

# Ejecutar 'ls' o 'dir' dependiendo del sistema operativo
comando = ["ls"] # o ["dir"] en Windows
resultado = subprocess.run(comando, capture_output=True, text=True)
print("Salida del comando:")
print(resultado.stdout)
```

```
013.py
import argparse

parser = argparse.ArgumentParser(description="Saluda al usuario")
parser.add_argument(*name_or_flags: "nombre", help="Nombre del usuario")
args = parser.parse_args()
print(f"¡Hola, {args.nombre}!")

# Para ejecutar este script desde la línea de comandos, guarda el código en un archivo llamado `saludo.py`
# y luego ejecuta el siguiente comando en la terminal:
# python saludo.py TuNombre
```

```
013.py
from typing import List

def suma_lista(numeros: List[int]) -> int:
    return sum(numeros)

print("Resultado:", suma_lista([1, 2, 3]))

# compile + eval
codigo = "3 * (2 + 1)"
codigo_compilado = compile(codigo, "<string>", "eval")
resultado = eval(codigo_compilado)
print("Resultado con eval:", resultado)

# exec para ejecutar múltiples líneas
bloque = """
for i in range(3):
    print("Línea", i)
"""

exec(bloque)
```

ASPECTOS AVANZADOS - ITERADORES

En Python, los iteradores son objetos que permiten recorrer elementos uno a uno (por ejemplo, en un bucle for). Comprender cómo funcionan los iteradores y cómo crearlos manualmente es esencial para escribir código más eficiente y pythonic.

Un iterador es cualquier objeto que implemente los métodos:

`__iter__()` → devuelve el objeto iterador.

`__next__()` → devuelve el siguiente valor o lanza `StopIteration` cuando termina.

```
014.py

#ITERADORES

class MyIterator:
    def __init__(self, data):
        self.data = data
        self.index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.index >= len(self.data):
            raise StopIteration
        value = self.data[self.index]
        self.index += 1
        return value

# Uso del iterador
data = [1, 2, 3, 4, 5]
iterator = MyIterator(data)
for item in iterator:
    print(item)
```

ASPECTOS AVANZADOS - GENERADORES

```
014.py
# Generador

def my_generator(data):
    for item in data:
        yield item

# Uso del generador
gen = my_generator(data)
for item in gen:
    print(item)

# Comprensión de generadores
gen_comp = (x * x for x in range(5))
for item in gen_comp:
    print(item)
```

```
014.py
# Iteradores y generadores con funciones integradas

def square(x):
    return x * x

def even_numbers(n):
    for i in range(n):
        if i % 2 == 0:
            yield i

# Uso de la función integrada map
numbers = [1, 2, 3, 4, 5]
squared_numbers = map(square, numbers)
for num in squared_numbers:
    print(num)
```

ASPECTOS AVANZADOS - DECORADORES

Los **decoradores** son funciones que **modifican el comportamiento de otras funciones**, sin cambiar su código. Son muy potentes y se usan ampliamente para tareas como logging, control de acceso, validación, memorización, entre otras.

```
015.py

import time

def timing_decorator(func): 1 usage new *
    def wrapper(*args, **kwargs): new *
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"Function '{func.__name__}' executed in {end_time - start_time:.4f} seconds")
        return result
    return wrapper

@timing_decorator 1 usage new *
def ejemplo_funcion(n):
    """Función que simula un procesamiento intensivo."""
    total = 0
    for i in range(n):
        total += i
    return total

if __name__ == "__main__":
    resultado = ejemplo_funcion(1000000)
    print(f"Resultado: {resultado}")

#Este código define un decorador que mide el tiempo de ejecución de una función.
```

ASPECTOS AVANZADOS – CONTEXT MANAGER

Los **Context Managers** (o administradores de contexto) son herramientas que gestionan recursos de forma automática, garantizando su correcta apertura y cierre, incluso si ocurre un error.

Se usan principalmente con la instrucción **with**, y su mayor ventaja es que liberan recursos correctamente (como archivos, conexiones, bloqueos, etc.) sin que tengas que preocuparte por ello.

Lo hemos visto en el tratamiento de archivos (UD4). Y ahora podemos ver en el ejemplo como crear uno personalizado.

```
016.py

# CONTEXT MANAGER

class MyContextManager: 1 usage new *
    def __enter__(self): new *
        print("Entering the context")
        return self

    def __exit__(self, exc_type, exc_value, traceback): new *
        print("Exiting the context")
        if exc_type is not None:
            print(f"An exception occurred: {exc_value}")
        return True # Suppress the exception

    def do_something(self): 1 usage new *
        print("Doing something inside the context")

# Uso del Context Manager
with MyContextManager() as manager:
    manager.do_something()

# Puedes descomentar la siguiente línea para ver cómo se gestiona una excepción
# raise ValueError("An error occurred")
```

ASPECTOS AVANZADOS - TESTING

El **testing** es una de las fases más importantes en el ciclo de desarrollo de nuestra aplicación. **Permite validar que nuestro código cumple los requisitos para los que ha sido diseñado.**

Muchas veces se obvia esta fase porque ralentiza el desarrollo de funcionalidades, pero ayuda a:

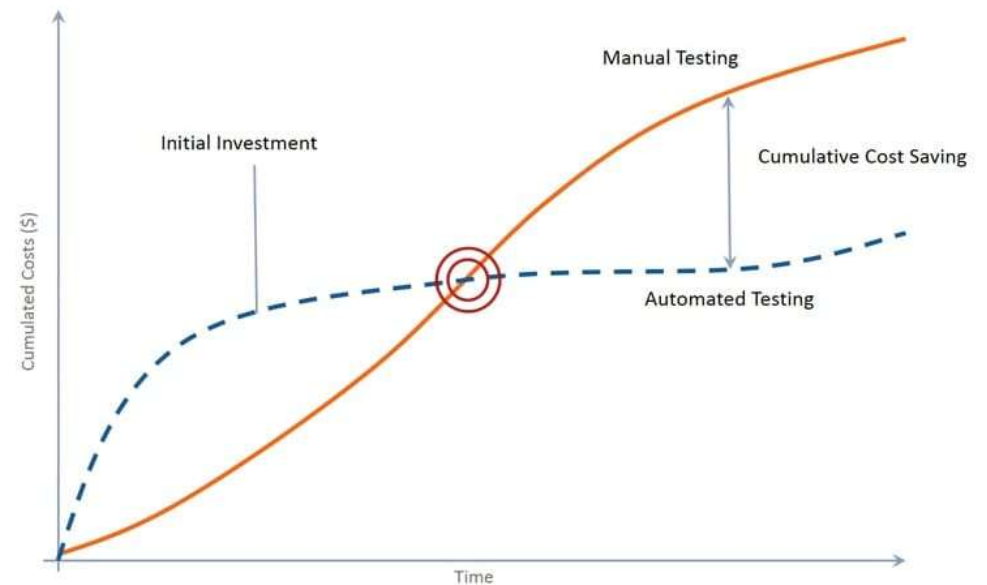
- Tener la seguridad de que hay menos bugs o errores en nuestro código (OJO, siempre existirán).
- Añadir nuevas funcionalidades sin temor a romper el código que ya existe.
- Entender mejor nuestro código, tanto para nuestro yo del futuro como para otros desarrolladores.
- Detectar fallos antes de desplegar nuestras aplicaciones al entorno de producción.



ASPECTOS AVANZADOS - TESTING

De acuerdo con la forma de ejecución tenemos:

- **Test manuales:** Son tests que son ejecutados por uno o varios probadores. Estos validan algún aspecto del software siguiendo distintos caminos o probando distintas combinaciones.
- **Test automáticos:** Se realizan mediante scripts que ejecutan una serie de instrucciones, buscando inconsistencias entre los resultados esperados y los obtenidos. Al ejecutarse, generan un reporte que el equipo revisa para detectar fallos y aplicar las correcciones necesarias.



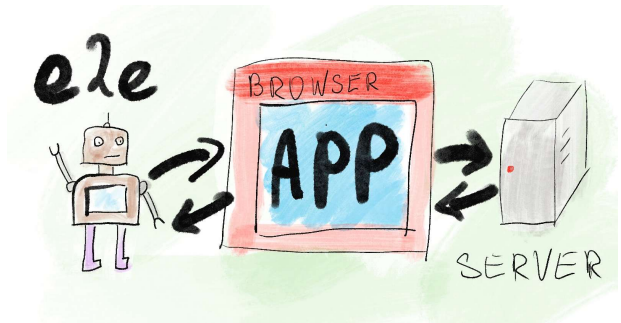
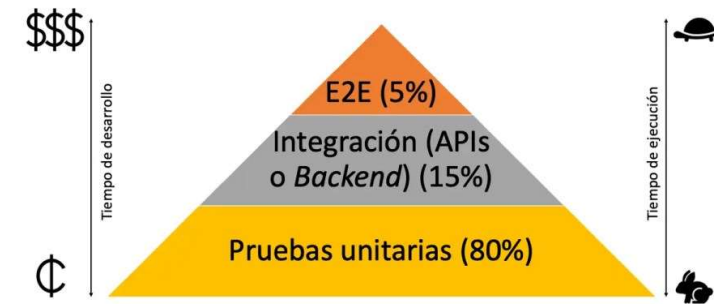
¿Qué ventajas e inconvenientes creéis que tienen ambas opciones?

ASPECTOS AVANZADOS - TESTING

¿Cuáles son los principales tipos de test que podemos realizar?

- **Pruebas unitarias (unit tests):** Se encargan de probar las funciones o métodos de nuestro software. Son muy rápidas (ya que ejecutan muy poco código) pero la implementación puede ser compleja (se debe aislar toda dependencia externa).
- **Pruebas de integración (integration tests):** Verifican si distintos módulos de la aplicación trabajan correctamente en conjunto (por ejemplo, varias aplicaciones de Django). Tardan más tiempo en ejecutarse que las pruebas unitarias.
- **Pruebas de extremo a extremo (end-to-end):** Replican el comportamiento de los usuarios con la aplicación completa, por ejemplo, cargar una página, iniciar sesión, realizar un pago, etc. Son muy útiles porque cubren flujos completos, pero son costosas de realizar y tardan mucho tiempo en ejecutarse.

Existen otras pruebas como funcionales, de regresión, de humo, de aceptación, rendimiento...

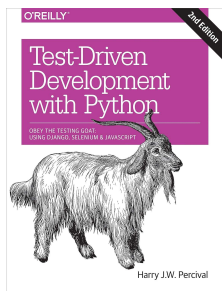


ASPECTOS AVANZADOS - TESTING

¿Cuándo se escriben los tests?

Se pueden seguir distintas prácticas. La más popular es **Test Driven Development (TDD)**, que consiste en escribir primero las pruebas (tests) antes de implementar el código (development).

Primero se escribe un test que prueba una función o método que no existe, por tanto, este test fallará (red). Después se implementa dicha función con el código mínimo necesario para que pase (green) y por último se mejora dicho código (refactorización) manteniendo el resultado exitoso en el test.



Para profundizar: Test-Driven Development with Python: Obey the Testing Goat: Using Django, Selenium, and JavaScript
Autor: Harry J.W. Percival

ASPECTOS AVANZADOS - TESTING

Vamos a trabajar con la librería **unittest**, que nos permite realizar test unitarios a nuestro código. Esta librería trabaja con ciertos conceptos como:

- **Test fixture**: Representa los datos que manejarán los tests cuando se ejecutan (una base de datos temporal).
- **Test case**: Representa uno o varios test, los cuales se crean a partir de una clase base llamada TestCase. Cada método de la nueva clase que comienza por test_ se considera una prueba individual.
- **Test runner**: Se encarga de buscar, cargar y ejecutar los tests, así como de mostrar los resultados.
- **Métodos de aserción**: Métodos que proporciona la clase TestCase para comprobar si el resultado obtenido es el esperado. En la tabla se pueden ver los métodos más utilizados.

Método	Comprobación
assertEqual(a, b)	$a == b$
assertNotEqual(a, b)	$a != b$
assertTrue(x)	$bool(x)$ es <i>True</i>
assertFalse(x)	$bool(x)$ es <i>False</i>
assertIs(a, b)	a es b
assertIsNot(a, b)	a no es b
assertIsNone(x)	x es <i>None</i>
assertIsNotNone(x)	x no es <i>None</i>
assertIn(a, b)	a contenido en b
assertNotIn(a, b)	a no contenido b
assertRaises(exc, fun, *args, **kwds)	<i>fun</i> lanza una excepción de tipo <i>exc</i> cuando es ejecutada

TESTING

unittest –
Framework de
pruebas.

doctest – Pruebas
embebidas en
documentación.

```
test_calculadora.py

import unittest
import logging
# Configurar logging
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')

def suma(a, b):
    """ new *
    resultado = a + b
    logging.info(f"Suma: {a} + {b} = {resultado}")
    return resultado

def division(a, b):
    """ 4 usages new *
    if b == 0:
        logging.error("Intento de división por cero")
        raise ValueError("No se puede dividir por cero")
    resultado = a / b
    logging.info(f"División: {a} / {b} = {resultado}")
    return resultado

class TestCalculadora(unittest.TestCase):
    """ new *

    def test_suma_enteros(self):
        """ new *
        resultado = suma(a: 4, b: 5)
        self.assertEqual(resultado, second: 9)
```

```
ejemplo_doctest.py

# archivo: ejemplo_doctest.py

def cuadrado(n):
    """ 2 usages new *
    """
    Devuelve el cuadrado de un número.

    >>> cuadrado(3)
    9
    >>> cuadrado(-2)
    4
    """
    return n * n

if __name__ == "__main__":
    import doctest
    print(doctest.testmod())
```

ASPECTOS AVANZADOS - TESTING

Testing con unittest: Ejemplo de uso

```
test_calculadora.py
import unittest
import logging
# Configurar logging
logging.basicConfig(level=logging.INFO, format='%(levelname)s: %(message)s')

def suma(a, b):
    resultado = a + b
    logging.info(f"Suma: {a} + {b} = {resultado}")
    return resultado
```

Los tests se ejecutan con: **python -m unittest nombre_archivo.py**

```
javier@MacBook-Pro pruebaActions %: python -m unittest test_calculadora.py

INFO: Suma: 2.5 + 3.1 = 5.6
INFO: Suma: 4 + 5 = 9
INFO: Suma: -2 + -3 = -5
.
-----
Ran 3 tests in 0.000s

OK
```

```
test_calculadora.py

class TestCalculadora(unittest.TestCase):

    def test_suma_enteros(self):
        resultado = suma(4, 5)
        self.assertEqual(resultado, 9)

    def test_suma_decimales(self):
        resultado = suma(2.5, 3.1)
        self.assertAlmostEqual(resultado, 5.6)

    def test_suma_negativos(self):
        resultado = suma(-2, -3)
        self.assertEqual(resultado, -5)

if __name__ == "__main__":
    unittest.main()
```

ASPECTOS AVANZADOS – INTEGRACIÓN CONTINUA (CI) Y DESPLIEGUE CONTINUO (CD)

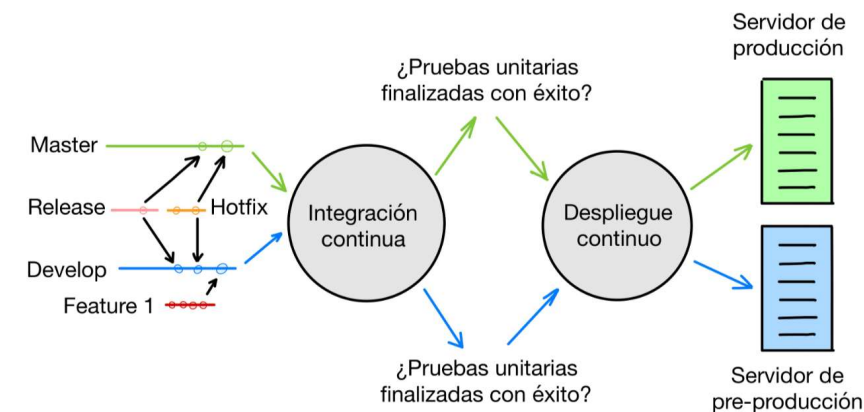
Entonces, ¿tengo que ejecutar los tests cuando estoy escribiendo código?

La respuesta corta es si, cuando estamos en el entorno de desarrollo local es normal ejecutar los tests y verificar que los resultados son los esperados.

Sin embargo, cuando la aplicación crece y el número de pruebas aumenta, puede volverse poco práctico ejecutarlas manualmente cada vez.

Automatizar la ejecución de tests ayuda a reducir errores humanos y facilita el desarrollo de nuevas funcionalidades. En entornos profesionales, los tests suelen integrarse en procesos de **Integración Continua (CI)** y **Despliegue Continuo (CD)**, de modo que cada cambio en el código activa automáticamente una cadena de validaciones y despliegues.

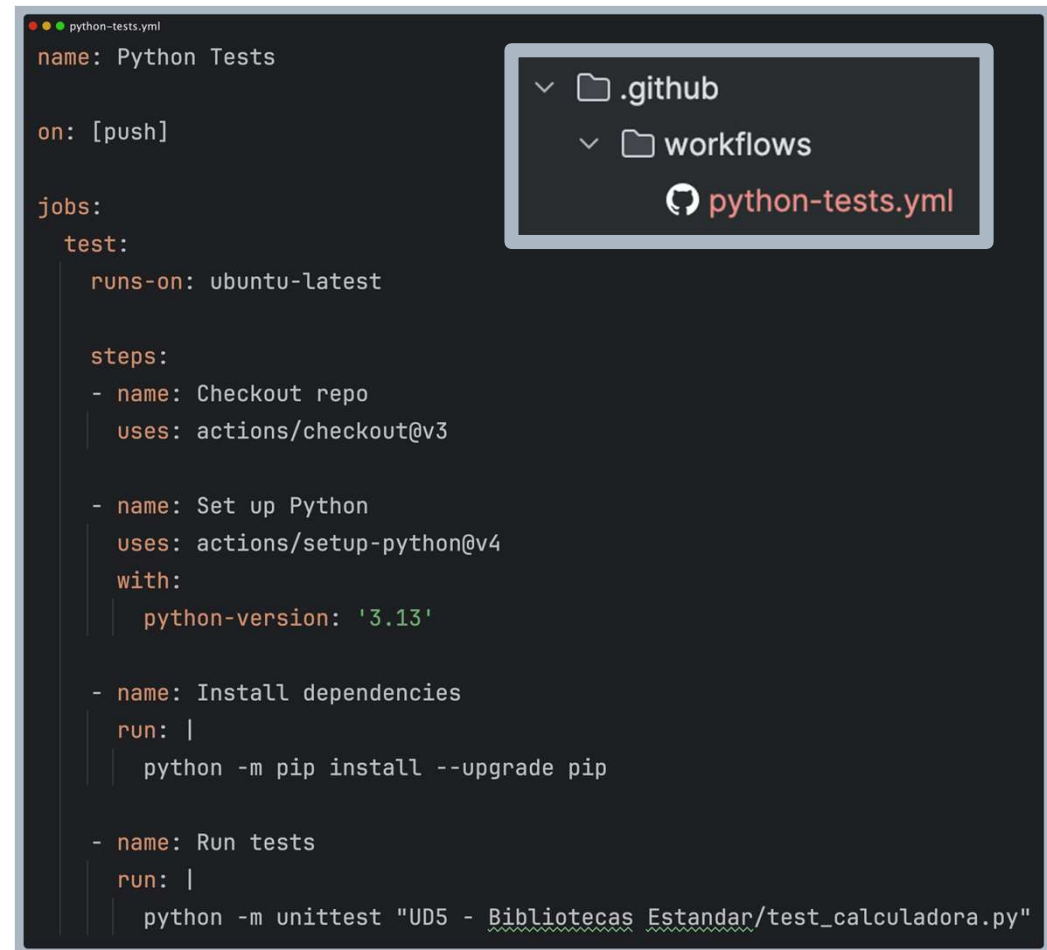
Utilizaremos **GitHub Actions** como herramienta de Integración Continua, ejecutará nuestros tests cuando hagamos push de nuestro código al repositorio.



ASPECTOS AVANZADOS – INTEGRACIÓN CONTINUA (CI) Y DESPLIEGUE CONTINUO (CD)

Crear nuestro primer Action.

1. Debemos crear la siguiente estructura en la raíz del proyecto:
2. El Action se define en un archivo YAML. Este archivo contiene:
 - a. El nombre del workflow.
 - b. Los eventos (on) que hacen que se ejecute el proceso (push, issues, pull_request, etc).
 - c. Las tareas (jobs) que se ejecutan.
 - d. Los pasos dentro de cada tarea (steps). Podemos ejecutar nuestros propios comandos (run) o utilizar acciones creadas por Github o la comunidad (uses).
3. Una vez hemos terminado, subimos el archivo a nuestro repositorio. Este creará el Action y pasará los tests a nuestro código.



The screenshot shows a code editor with a dark theme. On the right, a file explorer sidebar is open, showing a folder structure with '.github' and 'workflows'. Inside 'workflows', the file 'python-tests.yml' is highlighted. The main editor area displays the content of this file, which is a GitHub Actions workflow. The workflow is named 'Python Tests' and is triggered on a 'push' event. It contains a single job named 'test' that runs on 'ubuntu-latest'. The job has three steps: 'Checkout repo' (using 'actions/checkout@v3'), 'Set up Python' (using 'actions/setup-python@v4' with 'python-version: 3.13'), and 'Install dependencies' (using 'run: python -m pip install --upgrade pip'). The final step is 'Run tests' (using 'run: python -m unittest "UD5 - Bibliotecas Estandar/test_calculadora.py"').

```
python-tests.yml
name: Python Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repo
        uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.13'

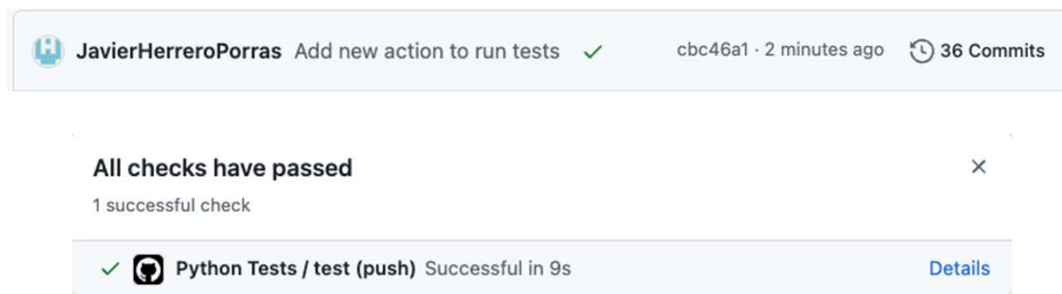
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip

      - name: Run tests
        run: |
          python -m unittest "UD5 - Bibliotecas Estandar/test_calculadora.py"
```

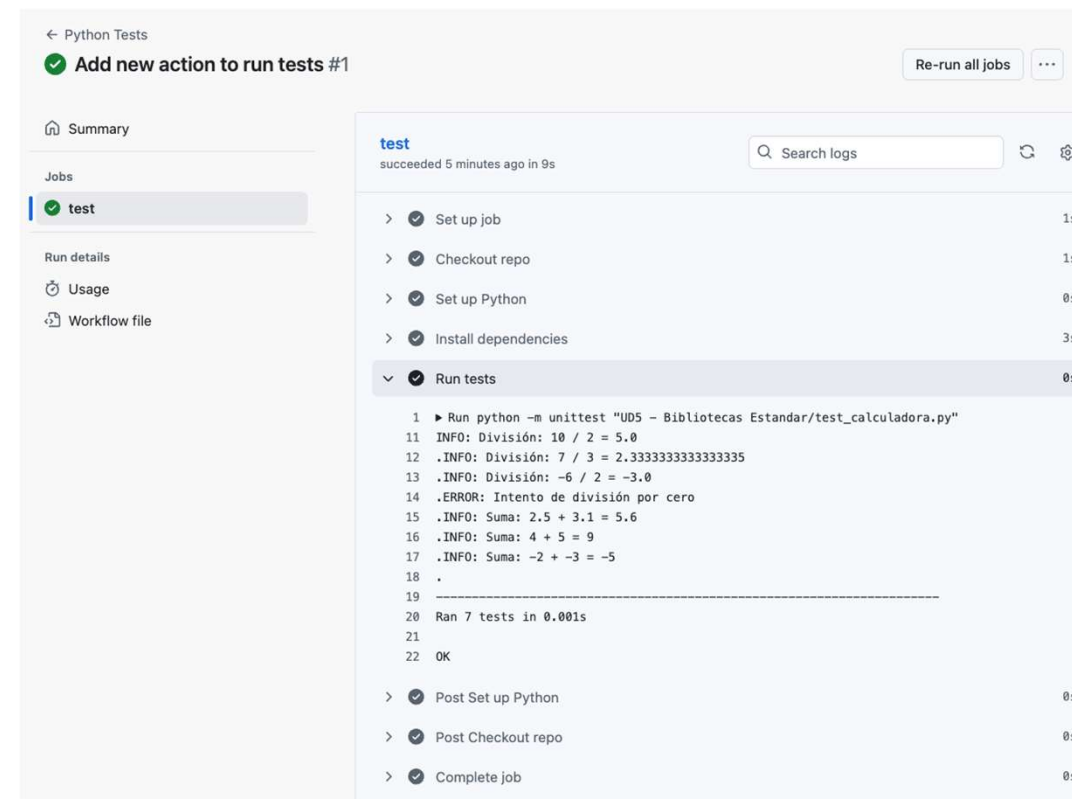
ASPECTOS AVANZADOS – INTEGRACIÓN CONTINUA (CI) Y DESPLIEGUE CONTINUO (CD)

Crear nuestro primer Action.

- Una vez hacemos push, se ejecutan los tests y obtenemos este resultado:



Con esta herramienta [podemos automatizar casi cualquier tarea](#): testing, calidad del código, construir aplicaciones, generar artefactos, desplegar en servidores, generar documentación, escanear vulnerabilidades, etc.



¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON



django

UD6:

DJANGO - INTRODUCCIÓN

—Y

Olga M. Moreno Martín - Javier Herrero Porras

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

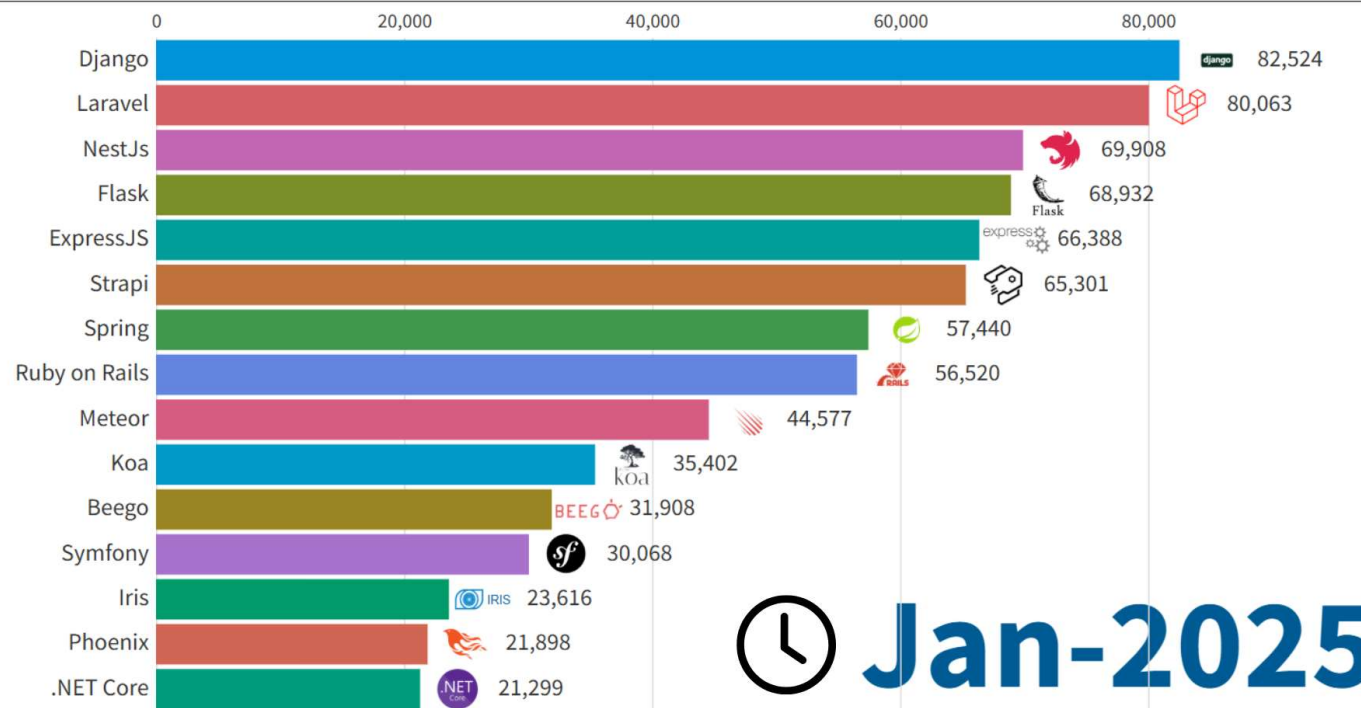
<https://creativecommons.org/licenses/by-nc-nd/4.0/>

ÍNDICE

- ▶ **Django**
- ▶ **Creación de un proyecto**
- ▶ **Archivos del proyecto**
- ▶ **Creación de una aplicación**
- ▶ **Archivos de la aplicación**
- ▶ **Crear una vista básica y su URL**
- ▶ **Crear una plantilla básica y renderizarla**

DJANGO

Most Popular Backend Frameworks





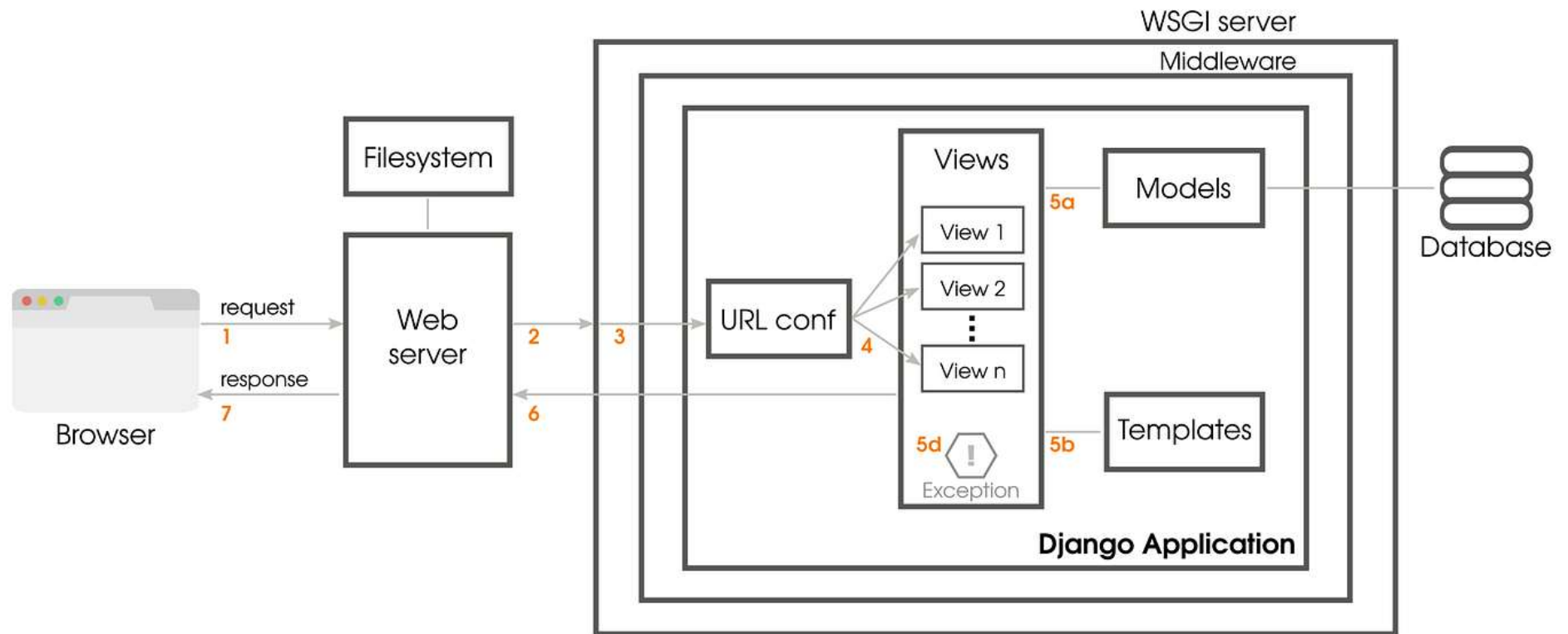
DJANGO

Django nace en 2004 para crear y mantener sitios muy grandes, URLs bien diseñadas, tener objetos Http Request y Response para cada petición, cubrir la necesidad de crear sitios web en poco tiempo y generar un ORM, es decir que te permite conectarte a la base de datos a través del framework. Django se toma la seguridad de las aplicaciones con mucha seriedad. Django es un framework muy escalable. Django es muy versátil, ha sido usado para todo tipo de proyectos, desde redes sociales hasta proyectos científicos.

Características

- Rápido desarrollo
- Viene listo para todo, es decir que trae consigo muchas herramientas como la autenticación de usuarios, la administración de contenido, mapas de sitio, etc.
- Seguro
- Escalable
- Versátil
- Django es Open Source.
- Django tiene el concepto (DRY(Don't Repeat Yourself)), donde dice que, si estás copiando y pegando algo en tu código, seguramente estás haciendo algo mal

DJANGO



DJANGO

¿Cómo funciona Django?

Una aplicación web se compone de dos partes principales: cliente (navegador web) y servidor (donde se ejecuta la lógica de la aplicación).

Cuando el cliente introduce una URL en el navegador, se envía una solicitud HTTP (request) al servidor. Django, que se ejecuta en el servidor, recibe esa solicitud, la procesa y genera una respuesta HTTP con el contenido solicitado. Esa respuesta (response) es enviada al cliente, que la muestra al usuario en el navegador.

Django organiza el código del lado del servidor siguiendo la arquitectura Modelo-Vista-Plantilla (**Model-View-Template, MVT**). Ahora pasaremos a abordar brevemente cada uno de los elementos:

- **Modelo (Model)**: Es la parte que maneja los datos. Son clases de Python que se encargan de definir la estructura de los datos, validarlos y comunicarse con la base de datos.
- **Vista (View)**: Es una función o clase que responde a las peticiones de los usuarios. Se encargan de acceder a los modelos, seleccionar las plantillas y generar la respuesta adecuada en base a una serie de reglas.
- **Plantilla (Template)**: Es un archivo HTML con etiquetas especiales de Django. Se encarga de mostrar la información al usuario. Se denominan así porque utilizan datos dinámicos que les proporcionan las vistas.

INSTALACIÓN DE DJANGO 5.2.3

Enlace a la instalación: <https://www.djangoproject.com/download/>

Con Django usaremos la versión 5.2 que hará uso de la versión 3.13 de Python.

Django tiene una instalación mediante comandos:

Mac/Linux:

```
python -m pip install Django==5.2.3
```

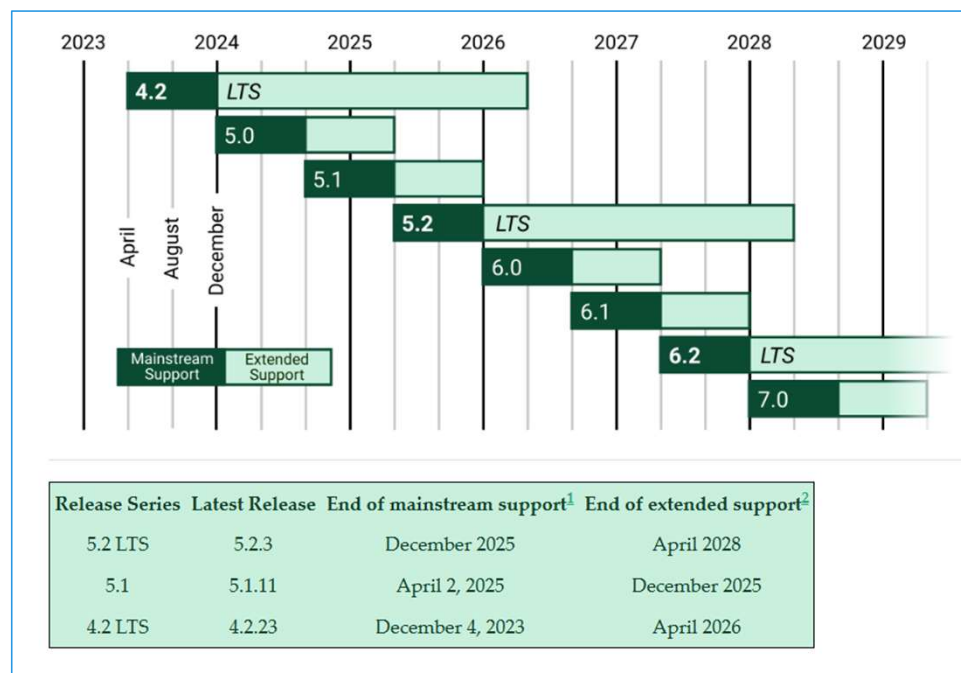
Windows:

```
py -m pip install Django==5.2.3
```

Lo utilizaremos más adelante, sin prisas.



The web framework for
perfectionists with
deadlines.



Olga M. Moreno Martín - Javier Herrero Porras

INSTALACIÓN DE COMPLEMENTOS

Configurar un Entorno Virtual

Es una buena práctica usar un entorno virtual para proyectos Django, un entorno virtual aísla las dependencias del proyecto, lo que permite que distintos proyectos utilicen diferentes bibliotecas y versiones.

Para crearlo utiliza los siguientes comandos:

pip install virtualenv

virtualenv myenv

- En Windows: **myenv\Scripts\activate**
- En macOS/Linux: **source myenv/bin/activate**

Instalar Herramientas Adicionales

Para una mejor experiencia de desarrollo, podrías querer instalar herramientas y paquetes adicionales:

- Django Extensions: Proporciona comandos de gestión adicionales y herramientas para proyectos

pip install django-extensions

- Herramientas para Bases de Datos: Dependiendo de tu elección de base de datos, podrías necesitar instalar controladores o bibliotecas adicionales.

CREAR NUESTRO PRIMER PROYECTO

Para crear nuestro primer proyecto, nos vamos a la terminal y ejecutamos:

pip install django

django-admin startproject myproject

Abrimos el proyecto que hemos creado:

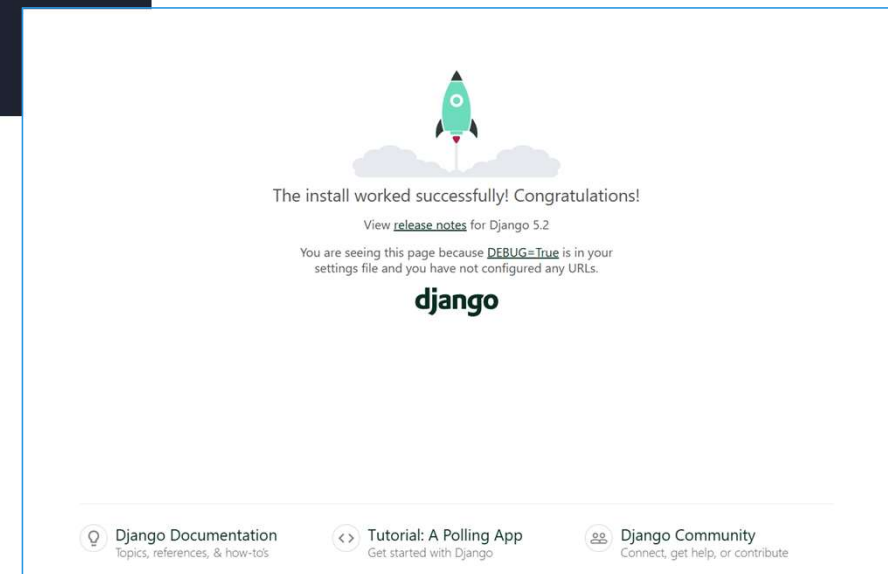
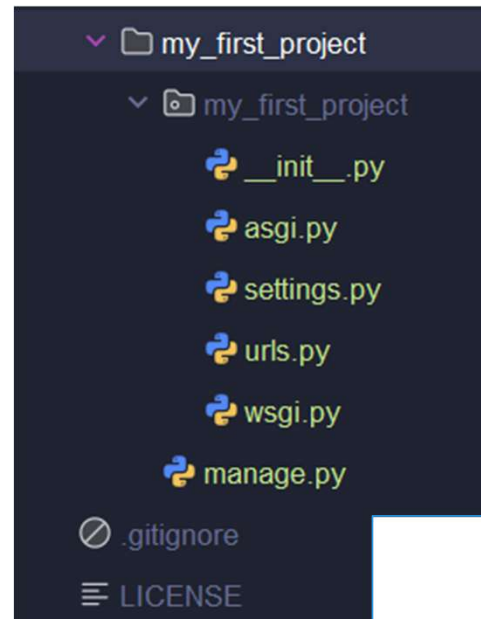
cd myproject

Para poner en marcha el servidor necesitaremos:

python manage.py runserver

Si todo ha ido bien tendremos django ejecutando por defecto en el puerto 8000

<http://127.0.0.1:8000/>



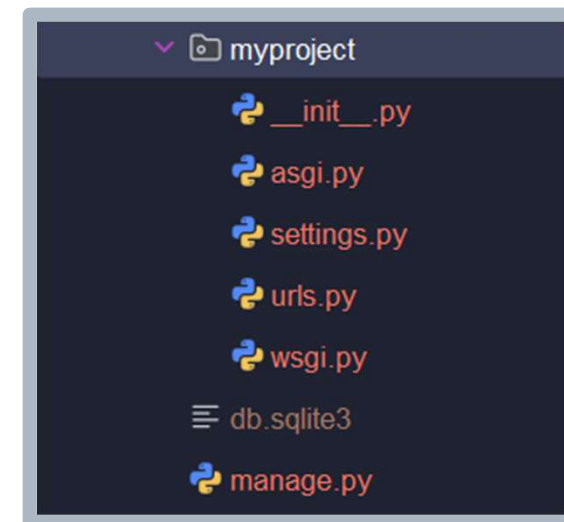
ARCHIVOS DE UN PROYECTO DJANGO

manage.py - Función: Script de utilidad para gestionar el proyecto Django desde la línea de comandos. Permite: ejecutar comandos como runserver, makemigrations, migrate, etc.

📁 Carpeta interna con el nombre del proyecto (**myproject/**)

Contiene la configuración principal del proyecto

- **__init__.py** - Función: Marca esta carpeta como un paquete de Python. Contenido: Puede estar vacío.
- **settings.py** - Función: Contiene todas las configuraciones del proyecto. Incluye: base de datos, apps instaladas, middleware, rutas estáticas, idioma, zona horaria, etc.
- **urls.py** - Función: Define las rutas principales del proyecto. Incluye: conexiones entre URLs y vistas (views) del proyecto o aplicaciones.
- **wsgi.py** - Función: Puerta de entrada del servidor WSGI (usado en producción). Se usa: para desplegar el proyecto en servidores como Gunicorn, uWSGI, etc.
- **asgi.py** - Función: Similar al wsgi.py, pero para servidores asíncronos como Daphne o Uvicorn (usado con Django Channels).



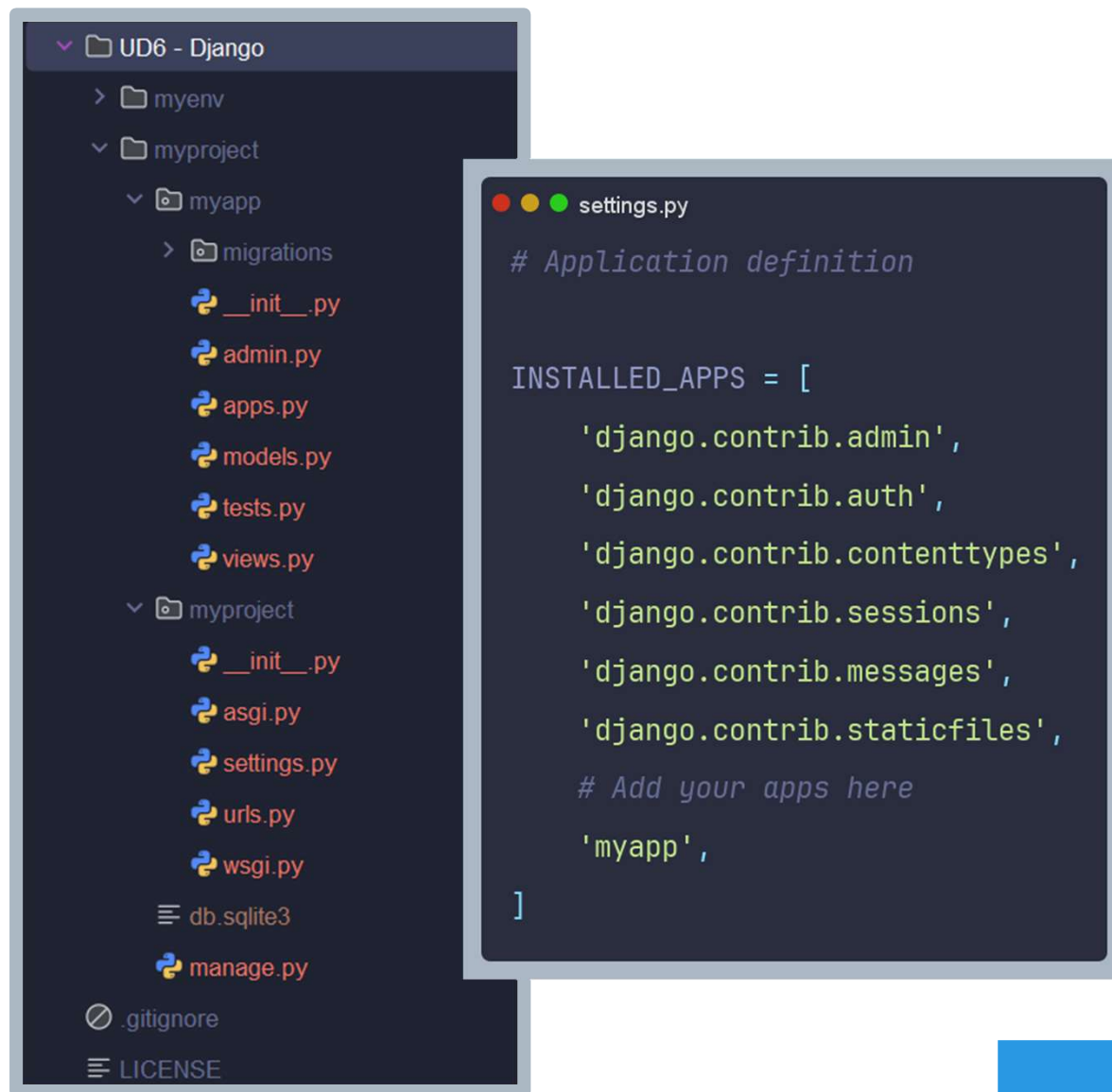
CREAR NUESTRA PRIMERA APLICACIÓN

Un proyecto Django puede estar compuesto de múltiples aplicaciones, para crear una aplicación utilizaremos:

`python manage.py startapp appname`

Luego tenemos que registrar esta app en el archivo settings.py del proyecto en la sección `INSTALLED_APPS`.

Llegados a este punto veremos una estructura de carpetas como la que aparece en la imagen.



ARCHIVOS DE UNA APLICACIÓN

Dentro de la carpeta de la aplicación encontraremos los siguientes archivos:

- **models.py** - Define los modelos de datos (tablas de la base de datos).
- **views.py** - Define las vistas o lógica de negocio que se ejecuta al visitar URLs.
- **urls.py (lo debes crear tú manualmente en la app)** - Define las rutas específicas de esta app. Se conecta luego al urls.py principal del proyecto.
- **admin.py** - Permite registrar modelos para que aparezcan en el panel de administración de Django.
- **apps.py** - Configuración básica de la app, como su nombre.
- **tests.py** - Se escriben aquí pruebas unitarias para asegurar que el código funciona correctamente.
- **📁 migrations/** - Contiene los archivos de migración de base de datos generados por Django (makemigrations y migrate).

CREAR UNA VISTA BÁSICA Y SU URL

Ahora vamos a crear una vista básica y establecer una URL para acceder a ella.

1º Definimos la función para la vista en views.py

2º Mapear la función con su URL en app/urls.py

3º Añadir las urls de la app al proyecto

```
views.py

from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.

def home_view(request): 2 usages
    """
    A simple view that returns a welcome message.
    """
    return HttpResponse("Welcome to my Django application!")
```

```
urls.py

from django.urls import path
from .views import home_view

urlpatterns = [
    path(route: '', home_view, name='home'),
]
```

```
urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path(route: 'admin/', admin.site.urls),
    # Include the URLs from your app
    path(route: '', include('myapp.urls')),
]
```

CREAR UNA PLANTILLA Y RENDERIZARLA

Para crear las interfaces de usuario usaremos plantillas (templates, hay que crear la carpeta) y luego uniremos una plantilla para renderizarla a una vista.



The image shows a code editor with three panels. The left panel displays the project structure of a Django application named 'UD6 - Django'. It includes folders for 'myenv', 'myproject', and 'myapp'. Inside 'myapp', there are 'migrations' and 'templates' folders. The 'templates' folder is selected, showing a file named 'home.html'. Below the folder view, a list of files is shown: '__init__.py', 'admin.py', 'apps.py', 'models.py', 'tests.py', 'urls.py', and 'views.py'. The middle panel shows the content of 'home.html', which is an HTML template with a head section containing meta tags for charset and viewport, a title 'Document', and a body section with a welcome message. The right panel shows the content of 'views.py', which imports 'render' from 'django.shortcuts' and 'HttpResponse' from 'django.http'. It defines a 'home_view' function that takes a 'request' object and returns a rendered template named 'home.html'.

```
home.html
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width">
  <title>Document</title>
</head>
<body>
  <h1>Welcome to Django App!</h1>
</body>
</html>
```

```
views.py
from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.

def home_view(request): 2 usages Olga M. Moreno *
    """
    A simple view that returns a welcome message.
    """
    return render(request, template_name: 'home.html')
```

CONFIGURAR Y LANZAR EL SERVIDOR

Nos queda configurar y poner en marcha el servidor para desarrollo para testear nuestro proyecto de forma local.

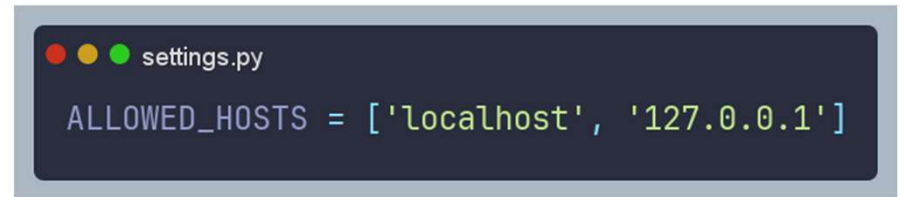
Abrimos `project_name/settings.py`:

Y en `ALLOWED_HOSTS` añadimos `['localhost', '127.0.0.1']`

Como hemos visto antes para poner en funcionamiento el servidor usaremos:

`python manage.py runserver`

Y podremos ver:

A screenshot of a code editor window titled 'settings.py'. The code shows the 'ALLOWED_HOSTS' variable being assigned a list containing 'localhost' and '127.0.0.1'.

```
ALLOWED_HOSTS = ['localhost', '127.0.0.1']
```



¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras



PYTHON



django

UD7: DJANGO -

Olga M. Moreno Martín - Javier Herrero Porras - Juan Carlos Jiménez Cabrera
Olga M. Moreno Martín - Javier Herrero Porras

LICENCIA



Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cCutacia Juez o Tribunal.

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

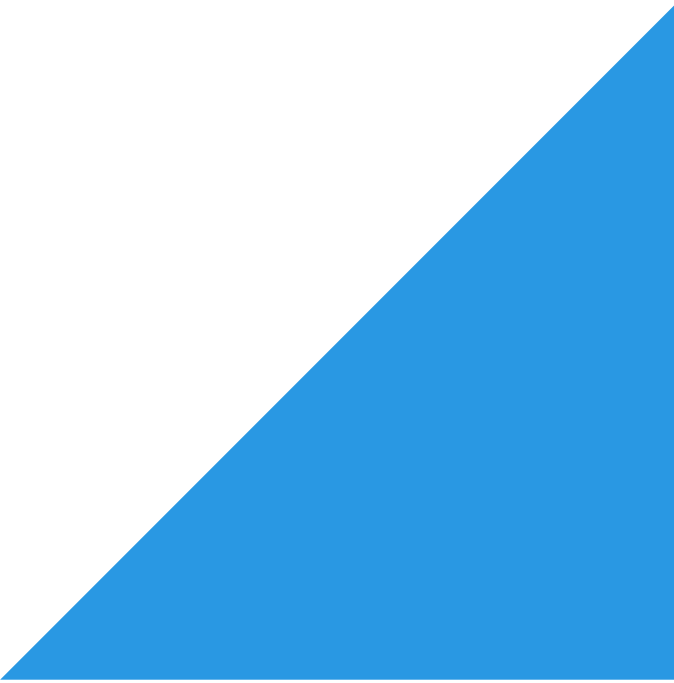
ÍNDICE

▶ **ORM**



▶ **ORM - Conceptos clave**





MODELOS. ORM

Olga M. Moreno Martín - Javier Herrero Porras

ORM (OBJECT RELATIONAL MODEL)

Un **ORM (Object-Relational Mapping o Mapeo Objeto-Relacional)** es una técnica de programación que se utiliza para convertir datos entre sistemas de tipos incompatibles en lenguajes de programación orientados a objetos. Permite interactuar con la base de datos utilizando código Python, en lugar de escribir consultas SQL puras.

El ORM de Django es una potente característica del framework que proporciona una abstracción de alto nivel para interactuar con bases de datos relacionales. El ORM permite a los desarrolladores interactuar con la base de datos utilizando objetos de Python en lugar de consultas SQL, lo que hace que las operaciones con bases de datos sean más intuitivas y manejables.

Ventajas del ORM de Django

- **Abstracción:** El ORM abstrae la complejidad de SQL y de las interacciones con la base de datos, permitiendo trabajar a un nivel más alto.
- **Consistencia:** Proporciona una interfaz consistente para diferentes bases de datos, facilitando el cambio entre ellas.
- **Productividad:** Simplifica tareas comunes con bases de datos, reduciendo la cantidad de código repetitivo.



ORM - CONCEPTOS CLAVE

Modelos

En Django, los modelos son clases de Python que representan tablas de la base de datos. Cada clase de modelo corresponde a una tabla, y cada atributo de la clase representa una columna de esa tabla.

Migraciones

Las migraciones son la forma en que Django propaga los cambios que haces en tus modelos al esquema de la base de datos. Permiten aplicar y deshacer cambios en el esquema a lo largo del tiempo

QuerySet

Un QuerySet es una colección de consultas a la base de datos. Permite recuperar, filtrar y manipular datos usando código Python. Los QuerySets son perezosos, lo que significa que no ejecutan la consulta hasta que realmente se necesita acceder a los datos.

Interfaz de Administración

La interfaz de administración de Django se genera automáticamente a partir de tus modelos. Proporciona una interfaz web para gestionar los datos sin necesidad de crear vistas o formularios personalizados.

ORM - CONCEPTOS CLAVE

Modelos: Tipos de datos

En un modelo podemos definir [multitud de campos](#), los cuales determinan el tipo de dato que se almacena en la columna de la base de datos. Son clases del módulo models ([django.db.models](#)). A continuación, veremos los más comunes:

- **BooleanField**: Valores booleanos (true/false).
- **CharField**: Cadenas de texto cortas. Necesita un argumento *max_length* para especificar el límite de caracteres.
- **DateField**: Fechas (sin hora)
- **DateTimeField**: Fechas con hora.
- **DecimalField**: Almacenar números decimales. Necesita argumentos *max_digits* y *decimal_places*.
- **IntegerField**: Números enteros. Existen subclases como `SmallIntegerField` y `BigIntegerField` para rangos específicos.
- **TextField**: Cadenas de texto largas, sin un límite definido.

Todos estos campos aceptan argumentos como: `max_length`, `null`, `blank`, `default`, `unique`, `primary_key`, etc.

OPERACIONES BÁSICAS

Definición de Modelos:

Para definir un modelo, crea una clase que herede de `django.db.models.Model`. Cada atributo de la clase representa un campo en la base de datos. Por ejemplo:

En `myapp/models.py`:

```
models.py

from django.db import models
# Create your models here.

class Author(models.Model): new *
    """
    Model representing an author.
    """
    name = models.CharField(max_length=100)
    birthdate = models.DateField()

    def __str__(self): new *
        return self.name
```

```
models.py

class Book(models.Model): new *
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)

    def __str__(self): new *
        return self.title
```

ORM - CONCEPTOS CLAVE

Modelos: Tipos de datos

¿Cómo defino la clave primaria de mi modelo?

Se utiliza un argumento **primary_key=True**. Por ejemplo:

```
isbn = models.CharField(max_length=13, primary_key=True)
```

Si en nuestro modelo no especificamos la clave primaria, Django añade de forma automática un campo como la clave primaria, que además es autoincremental:

```
id = models.AutoField(primary_key=True)
```

Además, Django nos ofrece un alias para referirnos a la clave primaria, denominado **pk**. Ejemplo:

```
models.py
class Book(models.Model):
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, primary_key=True)
```

```
demo_datos.py
def consultar_datos():
    # Metodos que devuelven QuerySets
    print("\n Todos los libros:")
    for libro in Book.objects.all():
        print(f"id: {libro.pk} - {libro.title} - {libro.author.name}")
```

```
>>> consultar_datos()

Todos los libros:
id: 9788497592208 - Cien años de soledad - Gabriel García Márquez
id: 9780307389732 - El amor en los tiempos del cólera - Gabriel Gar
```

```
>>> consultar_datos()

Todos los libros:
id: 1 - Cien años de soledad - Gabriel García Márquez
id: 2 - El amor en los tiempos del cólera - Gabriel García Márquez
```

ORM - CONCEPTOS CLAVE

Modelos: Tipos de datos

¿Y si la clave primaria es compuesta?

Django no soporta tener dos campos con atributos `primary_key`. En su lugar, podemos simular una clave primaria compuesta usando los metadatos del modelo ([Meta options](#)). Para ello se debe definir una clase interna dentro de nuestro modelo:

```
models.py

class PremioLiterario(models.Model):
    nombre = models.CharField(max_length=100)
    año = models.PositiveIntegerField()
    país = models.CharField(max_length=50)

    class Meta:
        # No puede repetirse un premio el mismo año
        constraints = [
            models.UniqueConstraint(fields=['nombre', 'año'], name='unique_premio_año')
        ]
```

Los metadatos del modelo sirven para especificar todo lo que no es un campo: establecer el orden de ordenación, cómo se llamará la tabla en la base de datos, índices, restricciones, etc.

ORM - CONCEPTOS CLAVE

Modelos: Tipos de datos

Para poder modelar las relaciones Django nos ofrece estos campos, los cuales son clases del módulo models ([django.db.models](#)):

- **ForeignKey**: Permite modelar una relación uno a muchos (1:N). Necesita dos argumentos: la clase con la que se relaciona y *on_delete*, que especifica lo que debe ocurrir con el registro cuando el objeto al que apunta la clave foránea se elimina.
- **ManyToManyField**: Permite modelar una relación muchos a muchos (N:M). Necesita un argumento: la clase con la que se relaciona.
- **OneToOneField**: Permite modelar una relación uno a uno (1:1). Necesita dos argumentos: la clase con la que se relaciona y *on_delete*, que especifica lo que debe ocurrir con el registro cuando el objeto al que apunta la clave foránea se elimina.

Estos campos pueden aceptar argumentos como *related_name*, *null*, *blank*, *verbose_name*, etc.

OPERACIONES BÁSICAS

Aplicación de la definición de los modelos:

Para la creación de tablas y esquemas tenemos que generar archivos para las migraciones de datos (**migrations**):

`python manage.py makemigrations`

```
Migrations for 'myapp':  
  myapp\migrations\0001_initial.py  
    + Create model Author  
    + Create model Book
```

Y para ejecutar las migraciones:

`python manage.py migrate`

```
Operations to perform:  
  Apply all migrations: admin, auth, contenttypes, myapp, sessions  
Running migrations:  
  Applying contenttypes.0001_initial... OK  
  Applying auth.0001_initial... OK  
  Applying admin.0001_initial... OK  
  Applying admin.0002_logentry_remove_auto_add... OK
```

OPERACIONES BÁSICAS: ALGO MÁS ACERCA DE LAS MIGRACIONES

Las **migraciones** son ficheros que permiten a Django mantener sincronizado el código de los modelos con la estructura real de la base de datos.

Las migraciones son como un sistema de control de versiones para el esquema de la base de datos, ya que guardan todas las operaciones y el orden en el que se han realizado. Si las ejecutamos en cualquier sistema, estas producirán siempre el mismo resultado, lo que nos asegura consistencia entre los distintos entornos de desarrollo.

Existen varios tipos de migraciones:

- Migraciones automáticas
- Migraciones manuales, por ejemplo, para cargar una serie de datos o ejecutar SQL específico.

Por último, **es posible revertir migraciones** a un estado anterior (si es los cambios se pueden deshacer):

`python manage.py migrate myapp 0003`



OPERACIONES BÁSICAS - CREAR DATOS

Para crear y guardar objetos tenemos las siguientes opciones:

- Constructor + save()
- Model.objects.create()
- Model.objects.bulk_create(array)
- Model.objects.get_or_create(): devuelve el objeto y un booleano que indica si se ha creado o ya existía, es útil para evitar duplicados.
- Model.objects.update_or_create(): si encuentra el registro lo actualiza, y si no existe lo crea.

```
demo_datos.py

# Crear un autor y guardarlo en la base de datos
autor = Author(name='Gabriel García Márquez', birthdate='1927-03-06')
autor.save()

# Crear un autor y guardarlo en la base de datos
autor = Author.objects.create(name='Isabel Allende', birthdate='1942-08-02')

# Crear varios autores de una vez
autores = [
    Author(name='Julio Cortázar', birthdate='1914-08-26'),
    Author(name='Mario Vargas Llosa', birthdate='1936-03-28'),
]
Author.objects.bulk_create(autores)

# Obtener o crear un autor, sin duplicados
autor2, creado2 = Author.objects.get_or_create(name='Borges', birthdate='1899-08-24')

# Obtener o crear un autor, sin duplicados
autor3, creado3 = Author.objects.update_or_create(
    name='J. K. Rowling',
    defaults={'birthdate': '1965-07-31'}
)
```

OPERACIONES BÁSICAS - ACTUALIZAR DATOS

Si cambiamos el estado de un objeto, debemos reflejar esos cambios en la base de datos usando `save()`.

Cuando se llama sin argumentos, se actualizan todos los campos del modelo, incluso aunque solo hayas modificado uno. Esto suele estar bien en la mayoría de casos, pero puede ser ineficiente si trabajas con modelos grandes o si se realizan muchas actualizaciones frecuentes.

Para optimizar este comportamiento, puedes usar `save(update_fields=[...])`, indicando explícitamente qué campos quieres guardar. Esto hace que Django genere una consulta SQL que solo actualiza los campos indicados, reduciendo el tráfico con la base de datos y evitando efectos secundarios innecesarios.

Olga M. Moreno Martín - Javier Herrero Porras

```
class BookAward(models.Model):
    nombre = models.CharField(max_length=100)
    año = models.PositiveIntegerField()
    país = models.CharField(max_length=50)
    organizador = models.CharField(max_length=100)
    categoria = models.CharField(max_length=50)
    idioma = models.CharField(max_length=30)
    dotacion_economica = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)
    fecha_entrega = models.DateField(null=True, blank=True)
    ciudad = models.CharField(max_length=50)
    edicion = models.PositiveIntegerField(help_text="Número de edición del premio", null=True, blank=True)
    ambito = models.CharField(max_length=50, choices=[('internacional', 'Internacional'), ('nacional', 'Nacional')])
    descripcion = models.TextField(blank=True)
    pagina_oficial = models.URLField(blank=True)
    esactivo = models.BooleanField(default=True)
    fecha_creacion = models.DateTimeField(auto_now_add=True)

class Meta:
    constraints = [
        models.UniqueConstraint(fields=['nombre', 'año', 'país', 'organizador', 'categoria', 'idioma', 'dotacion_economica', 'fecha_entrega', 'ciudad', 'edicion', 'ambito', 'descripcion', 'pagina_oficial', 'esactivo'], name='unique_book_award')
    ]
```

```
def crear_y_actualizar_premio():
    # Crear el premio
    premio = BookAward.objects.create(
        nombre="Premio Cervantes",
        año=2025,
        país="España",
        organizador="Ministerio de Cultura",
        categoria="Narrativa",
        idioma="Español",
        dotacion_economica=125000.00,
        fecha_entrega=date(2025, 4, 23),
        ciudad="Alcalá de Henares",
        edicion=50,
        ambito="nacional",
        descripcion="Máximo galardón de las letras españolas",
        pagina_oficial="https://www.culturaydeporte.gob.es",
        esactivo=True
    )

    # Guardar todos los campos (generará una UPDATE completa)
    premio.descripcion = "Premio más prestigioso de literatura en español"
    premio.save()

    # Guardar solo el campo 'ciudad' para optimizar la consulta SQL
    premio.ciudad = "Madrid"
    premio.save(update_fields=["ciudad"])
```

OPERACIONES BÁSICAS - CONSULTAR DATOS

Una vez se han creado los modelos, el ORM de Django permite acceder y manipular datos de la base de datos de forma intuitiva utilizando objetos Python a través de los **QuerySets**.

Lo primero que debemos hacer es tener datos para poder consultarlos. Podemos crear una instancia del modelo y llamar al método `save()`.

Es importante destacar que un `QuerySet` es una lista evaluable de objetos. Es por ello que debemos diferenciar entre dos tipos de métodos:

- Métodos que devuelven `QuerySets`: `all()`, `filter()`, `exclude()`, `order_by()`, `reverse()`, `annotate()`, `none()`, etc.
 - Su principal característica es que pueden encadenarse unos con otros, recorrer con bucles, ordenarse, etc.
 - Ejemplo:

```
demo_datos.py
def consultar_datos_encadenamiento():
    # Obtener libros de aquellos autores nacidos antes de 1930, que en su título no contengan "ficciones"
    # y ordenados por fecha de publicación descendente
    libros = Book.objects \
        .filter(author__birthdate__lt="1930-01-01") \
        .exclude(title__icontains="ficciones") \
        .order_by('-published_date')

    for libro in libros:
        print(libro.get_summary())
```

OPERACIONES BÁSICAS - CONSULTAR DATOS

Otros ejemplos de uso

- Obtener todos los registros:

```
authors = Author.objects.all()
```

- Filtrar: `authors = Author.objects.filter(name__startswith='J')`

```
authors = Author.objects.filter(birthdate__year=1990)
```

```
authors = Author.objects.filter(name__icontains='smith')
```

- Recuperar un solo objeto, si no existiese Lanza `DoesNotExist`; y si hay varios, `MultipleObjectsReturned`:

```
author = Author.objects.get(id=1)
```

- Excluir registros:

```
authors = Author.objects.exclude(name__icontains='smith')
```

OPERACIONES BÁSICAS - CONSULTAR DATOS

Es importante destacar que un QuerySet es una lista evaluable de objetos. Es por ello que debemos diferenciar entre dos tipos de métodos:

- Métodos que devuelven otro tipo de dato:
 - `get()`: Devuelve un solo objeto si lo encuentra. Si no existiese, lanza `DoesNotExist`; y si hay varios, `MultipleObjectsReturned`.
 - `count()`, `exists()`, `first()`, `last()`, `aggregate()`: Devuelven números enteros, booleanos, objetos o `None`, dependiendo del método. Más información en la [documentación](#).
 - Ejemplo:

```
demo_datos.py
def consultar_datos():
    # Métodos que devuelven un tipo distinto de dato a QuerySet
    # Contar cuántos libros hay en total
    total_libros = Book.objects.count()
    print(f"Total de libros: {total_libros}")

    # Obtener el primer libro (por orden por defecto, generalmente por ID)
    primer_libro = Book.objects.first()
    print(f"Primer libro creado: {primer_libro.title} ({primer_libro.published_date})")
```

OPERACIONES BÁSICAS - BORRAR DATOS

Podemos eliminar objetos de la base de datos usando el método `delete()` que proporciona el ORM. Este método devuelve una tupla: (número de objetos eliminados, {modelo: n° objetos eliminados}).

Ejemplos:

- Eliminar un solo objeto:

```
libro = Book.objects.get(pk=1)
```

```
libro.delete()
```

- Eliminar un conjunto de objetos:

```
Book.objects.filter(published_date__lt="1950-01-01").delete()
```

 Debes tener cuidado con los objetos relacionados. Por ejemplo, si borramos un autor y está relacionado con un libro, borraremos todos los libros asociados a este.

CONSULTAR Y TRABAJAR CON DATOS - EJEMPLO

Vamos a realizar un breve ejemplo con los siguientes modelos trabajando con la shell de Django:

`python manage.py shell`

Es un intérprete que se utiliza para realizar pruebas. Nos permite ejecutar scripts o trabajar con la base de datos. Está adaptada por Django para que funcione (como por ejemplo importar modelos).

En el archivo `utils/demo_datos.py` tenemos dos funciones:

- `crear_datos()`
- `consultar_datos()`

Debemos importarlas en nuestro shell para poder trabajar con ellas:

```
javier@MacBook-Pro myproject %: python manage.py shell
8 objects imported automatically (use -v 2 for details).

Python 3.13.0 (main, Jun 12 2025, 18:12:32) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from myapp.utils.demo_datos import crear_datos, consultar_datos
>>>
```

```
models.py

class Author(models.Model):
    """
    Model representing an author.
    """
    name = models.CharField(max_length=100)
    birthdate = models.DateField()

    def __str__(self):
        return self.name

class Book(models.Model):
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)

    def __str__(self):
        return self.title
```

CONSULTAR Y TRABAJAR CON DATOS - EJEMPLO

Si ejecutamos la función `crear_datos()` y después `consultar_datos()` obtendremos aquellos objetos que cumplen las condiciones:

- La primera consulta obtiene todos los libros.
- La segunda obtiene los autores ordenados de forma alfabética.
- La tercera obtiene los libros publicados antes de 1970.

```
javier@MacBook-Pro myproject %: python manage.py shell
8 objects imported automatically (use -v 2 for details).

Python 3.13.0 (main, Jun 12 2025, 18:12:32) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from myapp.utils.demo_datos import crear_datos, consultar_datos
>>> crear_datos()
>>> consultar_datos()

Todos los libros:
id: 1 - Cien años de soledad - Gabriel García Márquez
id: 2 - El amor en los tiempos del cólera - Gabriel García Márquez
id: 3 - La casa de los espíritus - Isabel Allende
id: 4 - Paula - Isabel Allende
id: 5 - La ciudad y los perros - Mario Vargas Llosa
id: 6 - Conversación en La Catedral - Mario Vargas Llosa
id: 7 - Ficciones - Jorge Luis Borges
id: 8 - El Aleph - Jorge Luis Borges
id: 9 - Como agua para chocolate - Laura Esquivel
id: 10 - La ley del amor - Laura Esquivel
```

```
demo_datos.py
from myapp.models import Author, Book
from datetime import date

def crear_datos():
    # Crear autores
    a1 = Author.objects.create(name="Gabriel García Márquez", birthdate=date(1927, 3, 6))
    a2 = Author.objects.create(name="Isabel Allende", birthdate=date(1942, 8, 2))
    a3 = Author.objects.create(name="Mario Vargas Llosa", birthdate=date(1926, 3, 28))
    a4 = Author.objects.create(name="Jorge Luis Borges", birthdate=date(1899, 8, 24))
    a5 = Author.objects.create(name="Laura Esquivel", birthdate=date(1950, 9, 30))

    # Libros de Gabriel García Márquez
    Book.objects.create(title="Cien años de soledad", author=a1, published_date=date(1967, 5, 30), isbn="9788497592288")
    Book.objects.create(title="El amor en los tiempos del cólera", author=a1, published_date=date(1985, 9, 5), isbn="9780307389732")

    # Libros de Isabel Allende
    Book.objects.create(title="La casa de los espíritus", author=a2, published_date=date(1982, 11, 1), isbn="9788408105024")
    Book.objects.create(title="Paula", author=a2, published_date=date(1994, 1, 1), isbn="9780806092721")

    # Libros de Mario Vargas Llosa
    Book.objects.create(title="La ciudad y los perros", author=a3, published_date=date(1963, 10, 10), isbn="9788420431379")
    Book.objects.create(title="Conversación en La Catedral", author=a3, published_date=date(1969, 4, 15), isbn="9788439725077")

    # Libros de Jorge Luis Borges
    Book.objects.create(title="Ficciones", author=a4, published_date=date(1944, 1, 1), isbn="9788491050283")
    Book.objects.create(title="El Aleph", author=a4, published_date=date(1949, 1, 1), isbn="9789580426056")

    # Libros de Laura Esquivel
    Book.objects.create(title="Como agua para chocolate", author=a5, published_date=date(1989, 9, 1), isbn="9780307743855")
    Book.objects.create(title="La ley del amor", author=a5, published_date=date(1995, 3, 1), isbn="9788466329514")

def consultar_datos():
    print("\n Todos los libros:")
    for libro in Book.objects.all():
        print(f"id: {libro.pk} - {libro.title} - {libro.author.name}")

    print("\n Todos los autores ordenados alfabéticamente:")
    for autor in Author.objects.order_by('name'):
        print(f"{autor.name} ({autor.birthdate})")

    print("\n Libros publicados antes de 1970:")
    for libro in Book.objects.filter(published_date__lt="1970-01-01"):
        print(f"{libro.title} - {libro.published_date}")

    print("\n Libros cuyo título contiene 'amor':")
    for libro in Book.objects.filter(title__icontains='amor'):
        print(libro.title)
```

CONSULTAR Y TRABAJAR CON DATOS - EJERCICIO

Ahora es tu turno. Abre una shell de Django e importa y ejecuta la función `crear_datos()` para insertar autores y libros en la base de datos.

A continuación, y dentro de la misma sesión de shell, realiza las siguientes consultas:

- Libros ordenados por fecha de publicación descendente
- Libros de Gabriel García Márquez
- Libros de autores nacidos antes de 1930
- Libros cuyo ISBN empieza por '97884'
- Libros que no son de Borges
- Libros escritos por Borges o por Esquivel
- Número total de libros
- Número de libros por autor
- Primer libro escrito (de acuerdo con la fecha)

ACCESO A OBJETOS RELACIONADOS

¿Cómo navego entre modelos conectados?

Si nos basamos en las siguientes clases, ya sabemos cómo acceder al autor de un libro:

```
libro = Book.objects.get(pk=1)

print(libro.autor.name)
```

Pero, si tenemos un autor, ¿podemos acceder a los libros que ha escrito? La respuesta es sí, Django nos permite trabajar con objetos relacionados (**related objects**), que son atributos generados para acceder a los datos desde el lado inverso de una relación:

```
author = Author.objects.get(pk=1)

author.books_set.all()
```

```
models.py

class Author(models.Model):
    """
    Model representing an author.
    """
    name = models.CharField(max_length=100)
    birthdate = models.DateField()

    def __str__(self):
        return self.name

class Book(models.Model):
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)

    def __str__(self):
        return self.title
```

ACCESO A OBJETOS RELACIONADOS

¿Cómo navego entre modelos conectados?

Imaginando la relación entre las clases Autor y Libro:

Relación	Desde el modelo Autor	Desde el modelo Libro
1 Autor – N Libros	author.book_set.all()	libro.author

El nombre por defecto del atributo es el nombre del modelo en minúscula + set: book_set. Se puede cambiar a través del atributo *related_name*:

```
author = models.ForeignKey(Author, ..., related_name="libros")
```

```
author.libros.all()
```

```
models.py

class Author(models.Model):
    """
    Model representing an author.
    """
    name = models.CharField(max_length=100)
    birthdate = models.DateField()

    def __str__(self):
        return self.name

class Book(models.Model):
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)

    def __str__(self):
        return self.title
```

ACCESO A OBJETOS RELACIONADOS

¿Cómo navego entre modelos conectados?

Imaginando la relación entre las clases Autor y Libro:

Relación	Desde el modelo Autor	Desde el modelo Libro
1 Autor – 1 Libro	author.book.title	libro.author

El nombre por defecto del atributo es el nombre del modelo en minúscula: book. Se puede cambiar a través del atributo *related_name*

```
models.py

class Author(models.Model):
    """
    Model representing an author.
    """
    name = models.CharField(max_length=100)
    birthdate = models.DateField()

    def __str__(self):
        return self.name

class Book(models.Model):
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    author = models.OneToOneField(Author, on_delete=models.CASCADE)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)
```

ACCESO A OBJETOS RELACIONADOS

¿Cómo navego entre modelos conectados?

Imaginando la relación entre las clases Autor y Libro:

Relación	Desde el modelo Autor	Desde el modelo Libro
N Autores – N Libros	author.book_set.all()	libro.authors.all()

El nombre por defecto del atributo es el nombre del modelo en minúscula + set: book_set. Se puede cambiar a través del atributo *related_name*

```
models.py

class Author(models.Model):
    """
    Model representing an author.
    """
    name = models.CharField(max_length=100)
    birthdate = models.DateField()

    def __str__(self):
        return self.name

class Book(models.Model):
    """
    Model representing a book.
    """
    title = models.CharField(max_length=200)
    authors = models.ManyToManyField(Author)
    published_date = models.DateField()
    isbn = models.CharField(max_length=13, unique=True)
```

ENRIQUECIMIENTO DE MODELOS

Enriquecer la funcionalidad de un modelo no es más que añadir métodos personalizados y propiedades.

Podemos añadir una propiedad para saber si un libro es anterior a 1970 y decir que es un “clásico”, o crear un método resumen para obtener el libro, su autor y el año de publicación.

```
models.py

@property new *
def is_classic(self):
    """
    Check if the book is a classic (published before 1970).
    """
    return self.published_date.year < 1970

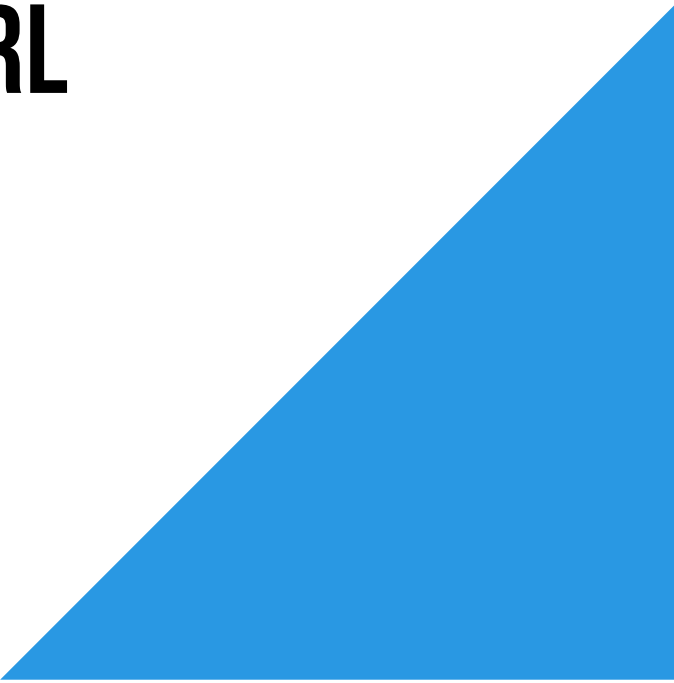
def get_summary(self): new *
    """
    Return a summary of the book.
    """
    return f"{self.title} by {self.author.name}, published on {self.published_date.year('%Y')}"
```

REGISTRAR LOS MODELOS EN EL ADMIN DE DJANGO

Una vez hemos creado nuestros modelos, y los hemos enriquecido si es necesario, debemos hacer que el admin de Django los tenga disponibles, para ellos nos iremos a `myapp/admin.py`. Importaremos `admin` y los modelos que hemos creado y también los registraremos.

```
admin.py

from django.contrib import admin
from .models import Author, Book
# Register your models here.
admin.site.register(Author)
admin.site.register(Book)
```



VISTAS Y ENRUTADO DE URL

Olga M. Moreno Martín - Javier Herrero Porras

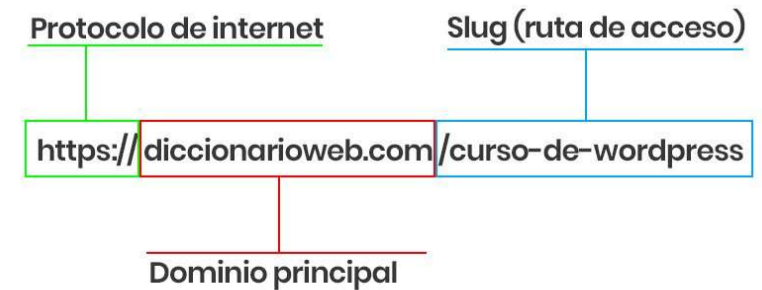
ENRUTADO DE URL

En Django, las vistas y los patrones de URL son componentes fundamentales del framework que trabajan juntos para gestionar solicitudes web y generar respuestas adecuadas. Vamos a tomar una visión general de cómo funcionan estos componentes y cómo utilizarlos de manera eficaz.

Una **vista** es una función que recibe una petición web (request) y devuelve una respuesta (response) en forma de página HTML, JSON u otro contenido.

Para que una vista sea accesible desde el navegador, debe estar asociada a una URL específica. Esta asociación necesita de un módulo de Python **URLconf** (URL configuration), que permite asociar una URL con una vista. Este proceso lo realizaremos en **urls.py**.

Cabe destacar que las URLs en Django pueden estructurarse de forma jerárquica. Es decir, un archivo de configuración de URLs (URLconf) puede incluir rutas que delegan en otros módulos de URLs, permitiendo una organización modular del enrutado en la aplicación.



```
urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home_view, name='home'),
    path('contact', views.contact_view, name='contact'),
]
```

ENRUTADO DE URL

¿Cómo procesa Django un request?

Cuando se realiza una petición a nuestro sitio se siguen estos pasos:

1. Django determina cuál es el módulo URLconf raíz.
2. Carga el módulo y busca la variable urlpatterns. Itera sobre dicha variable buscando el patrón que coincida con la URL de la petición.
 1. Si encuentra una coincidencia, se importa la vista asociada y se realiza una llamada a la misma. A esta se le pasa una instancia de la petición (HttpRequest) y argumentos que contenga la petición.
 2. Si no hay ninguna coincidencia o si se lanza una excepción durante el proceso, Django invoca una vista para manejar la situación (404, 500, etc).

Requests:

<http://localhost:8000/>
<http://localhost:8000/articulos/2003/>
<http://localhost:8000/articulos/2005/>
<http://localhost:8000/articulos/2003/03/>

Raíz

```
urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.inicio_sitio, name='index'),
    path('admin/', admin.site.urls),
    path('articulos/', include('articulos.urls')), # Incluye las rutas de la app 'articulos'
]
```

Artículos

```
urls.py
from django.urls import path

from . import views

urlpatterns = [
    path("2003/", views.special_case_2003),
    path("<int:year>/", views.year_archive),
    path("<int:year>/<int:month>/", views.month_archive),
]
```

ENRUTADO DE URL

Pensemos...

Tenemos que implementar una vista para consultar la información de un producto concreto a partir de su id. Tenemos que pasar a la vista ese identificador, y lo haremos a través de la URL:

`http://localhost:8000/articulos/7/`

Cuando se realice esa consulta, nuestra aplicación necesita saber que necesita mostrar los datos del producto con ID 7. Es por ello que utilizaremos los **path converters** de Django, que permiten capturar valores desde la URL y pasarlos como argumentos a la vista correspondiente.

`http://localhost:8000/articulos/7/`

Vista

Se encargará de obtener el producto con id=7 y devolver su información.

(Le pasaremos el id como parámetro para poder utilizarla con cualquier valor)

ENRUTADO DE URL

Path converters

Son una forma de capturar valores dinámicos desde una URL y pasarlos como argumentos a la vista correspondiente. Se usan dentro de la función `path()` y permiten restringir el tipo de datos que se aceptan en una determinada parte de la URL.

Uso:

```
path('ruta/<converter:nombre_parametro>/', vista)
```

Donde:

- `converter`: Es el tipo de dato esperado
- `nombre_parámetro`: Nombre de la variable que recibirá la vista como argumento. Dicha variable será del tipo especificado por el `converter` y Django valida previamente el valor. Si no es válido se devuelve un 404.

Converter	Tipo esperado	Ejemplo
str	Cadena de texto	/usuario/juan/
int	Número entero positivo	/producto/23/
slug	Cadena con letras, números y guiones	/blog/mi-post-2025/
uuid	UUID válido	/archivo/550e8400-e29b.../
path	Cadena de texto (puede incluir /)	/media/fotos/2025/julio/

ENRUTADO DE URL

Ejercicio

A partir de los datos proporcionados, ¿qué vistas se ejecutan? ¿Qué valores se pasan a la vista?

```
urls.py
urlpatterns = [
    path('', views.inicio, name='inicio'),
    path('saludo/<str:nombre>/', views.saludo, name='saludo'),
    path('producto/<int:id>/', views.detalle_producto, name='detalle_producto'),
    path('blog/<slug:slug>/', views.ver_articulo, name='ver_articulo'),
    path('archivo/<uuid:clave>/', views.descargar_archivo, name='descargar_archivo'),
    path('ruta/<path:subruta>/', views.mostrar_ruta, name='mostrar_ruta'),
]
```

/saludo/maria/
/producto/42/
/producto/abc/
/blog/django-con-path-converters/
/archivo/550e8400-e29b-41d4-a716-446655440000/
/archivo/12345/
/ruta/media/fotos/2025/

ENRUTADO DE URL

Usando expresiones regulares

A veces necesitamos definir las rutas con mayor control del que nos permiten los path converters. Para ello Django ofrece el uso de expresiones regulares mediante la función `re_path()`. Las expresiones regulares permiten especificar patrones exactos (que el nombre de usuario no contenga más de 7 caracteres) o validar estructuras complejas (fechas).

Uso:

```
re_path(r'^ruta/(?P<parametro>expresion)/$', vista)
```

Donde:

- `r''`: Indica que es una expresión regular
- `^` y `$`: Indican el inicio y el final de la URL
- `(?P<nombre>expresion)`: Define que la expresión regular se capturará y se pasará a la vista con un *nombre*. Es una forma especial de capturar una cadena que cumple la expresión, y se guarda con un nombre.

ENRUTADO DE URL

Usando expresiones regulares

Expresión regular (patrón)	Qué coincide / Significado	Ejemplo de URL
^\$	Cadena vacía (inicio del sitio)	/
^[a-z]+/\$	Una o más letras minúsculas (a a z)	/saludo/
^[A-Z]+/\$	Una o más letras mayúsculas (A a Z)	/ADMIN/
^[a-zA-Z]+/\$	Una o más letras (mayúsculas o minúsculas)	/Marta/, /juan/
^[0-9]+/\$	Uno o más dígitos (0 a 9)	/123/
^[0-9]{4}/\$	Exactamente 4 dígitos	/2025/
^[a-z0-9_-]+/\$	Letras minúsculas, dígitos, guiones y guiones bajos	/mi-post_2025/
^[a-z]+/[0-9]+/\$	Dos segmentos: letras minúsculas, barra, luego números	/producto/23/
^([a-z]+)/([0-9]+)/\$	Igual que anterior, pero sin nombres de grupo	/producto/23/
^archivo/(?P<clave>[0-9a-f-]{36})/\$	UUID: 36 caracteres hexadecimales y guiones	/archivo/550e84.../
^ruta/(?P<subruta>.+)/\$	Cualquier texto (incluye /), al menos un carácter	/ruta/media/julio/

ENRUTADO DE URL

Ejercicio

A partir de los datos proporcionados, ¿qué vistas se ejecutan? ¿Qué valores se pasan a la vista?

```
urls.py
from django.urls import re_path
from . import views

urlpatterns = [
    re_path(r'^$', views.inicio, name='inicio'),
    re_path(r'^usuario/(?P<nombre>[a-zA-Z]+)/$', views.perfil_usuario, name='perfil_usuario'),
    re_path(r'^curso/(?P<codigo>[A-Z]{3}[0-9]{2})/$', views.detalle_curso, name='detalle_curso'),
    re_path(r'^blog/(?P<slug>[a-z0-9_-]+)/$', views.ver_post, name='ver_post'),
    re_path(r'^media/(?P<ruta>.+)/$', views.descargar_media, name='descargar_media'),
    re_path(r'^anio/(?P<anio>[0-9]{4})/$', views.filtrar_por_anio, name='filtrar_por_anio'),
]
```

/
/usuario/Maria/
/usuario/maria23/
/curso/abc123/
/curso/DAW01/
/blog/proyecto-final_2025/
/blog/Proyecto-Final/
/media/videos/clase1.mp4/
/anio/24/
/anio/2024/

VISTAS

Podemos distinguir dos tipos de vistas:

Vistas Basadas en Funciones (**Function-Based Views** - FBVs):

Las vistas en Django son funciones o clases de Python que reciben solicitudes web como entrada y devuelven una respuesta. Son ideales para vistas “directas” que no requieren lógica compleja.

Vistas Basadas en Clases (**Class-Based Views** - CBVs): Son más estructuradas y ofrecen más funcionalidades. Se usan cuando se requiere lógica adicional o manejo de distintos métodos HTTP (GET, POST, etc.).

En esta primera parte nos centraremos en las Vistas Basadas en Funciones.

```
views.py

def my_view(request): new *
    return HttpResponse("¡Hola, mundo!")
```

```
views.py

def MyView(View): new *
    def get(self, request): new *
        return HttpResponse("Hello, world.")
```

VISTAS

Una **vista** es una función que recibe una petición web (request) y devuelve una respuesta (response) en forma de página HTML, JSON, XML, imagen, un error 404...

Las vistas pueden estar en cualquier fichero del proyecto, aunque generalmente se colocan en **views.py**.

Vamos a ver un par de ejemplos de vistas simples:

1. En el primero, la vista se llama `my_view`, que toma como primer parámetro un objeto de tipo `HttpRequest`. Normalmente se llama `request`.
 1. Esta vista solamente devuelve una respuesta `Http` (`HttpResponse`) con el contenido "¡Hola, mundo!"
2. En la segunda, observamos que podemos construir una página HTML. Además, también podemos pasarle el código de respuesta como parámetro (200, 404, 403, etc).

```
views.py

def my_view(request): new *
    return HttpResponse("¡Hola, mundo!")
```

```
views.py

def devuelve_saludo(request):
    html = '<html><body>Hello, world!</body></html>'
    return HttpResponse(html, status=403)
```

VISTAS

¿Qué nombre podemos darles a nuestras vistas?

Como hemos visto anteriormente, una vista es una función. Por tanto, podemos darle el nombre que deseemos (intentamos que sea descriptivo respecto a la tarea que realiza).

Además, podemos darle un nombre a esa vista para hacer referencia a ella desde plantillas, redirecciones y otras funciones del lenguaje usando el parámetro **name** en el archivo **urls.py**.

Name		X	Headers	Preview	Response	Initiator	Timing
	redirect/	▼ General Request URL http://localhost:8000/redirect/ Request Method GET Status Code ● 302 Found Remote Address 127.0.0.1:8000 Referrer Policy strict-origin-when-cross-origin					
	localhost						
		▼ Response Headers ☐ Raw Content-Length 0 Content-Type text/html; charset=utf-8 Cross-Origin-Opener-Policy same-origin Date Thu, 24 Jul 2025 08:39:18 GMT					
2 requests	618 B transfer	Location		/			

Si accedemos a <http://localhost:8000/redirect>, estaríamos ejecutando la vista `redirecciona_saludo`. Esta vista ejecuta la función **redirect()**, que devuelve un código 302 al navegador, que le indica que debe hacer una nueva petición a <http://localhost:8000/>.

```
urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.devuelve_saludo, name='saludo'),
    path('redirect/', views.redirecciona_saludo, name='redireccion'),
]
```

```
views.py
from django.shortcuts import redirect
from django.http import HttpResponseRedirect

def devuelve_saludo(request):
    html = '<html><body>Hello, world!</body></html>'
    return HttpResponseRedirect(html)

def redirecciona_saludo(request):
    return redirect("saludo")
```

VISTAS

¿Qué podemos hacer en una vista?

La vista es el centro del flujo de control de la aplicación. Desde estas podemos acceder a los datos de la petición (request), procesar lógica de negocio (trabajar con modelos), lanzar excepciones, devolver distintos tipos de respuesta, trabajar con formularios, etc.

Lo primero es determinar, ¿con qué métodos queremos que trabaje nuestra vista?

Podemos especificarlo a través del decorador **@require_http_methods([])**. Le pasaremos como argumentos una lista con métodos: GET, POST, PUT, DELETE...

Si sabemos que nuestra lista solo aceptará GET o POST, podemos usar **@require_GET()** o **@require_POST()**

```
views.py
from django.views.decorators.http import require_http_methods

@require_http_methods(["GET", "POST"])
def restriccion_metodos(request):
    if request.method == "POST":
        # Se ha hecho una petición POST
        pass
    elif request.method == "GET":
        # Se ha hecho una petición GET
        pass
```

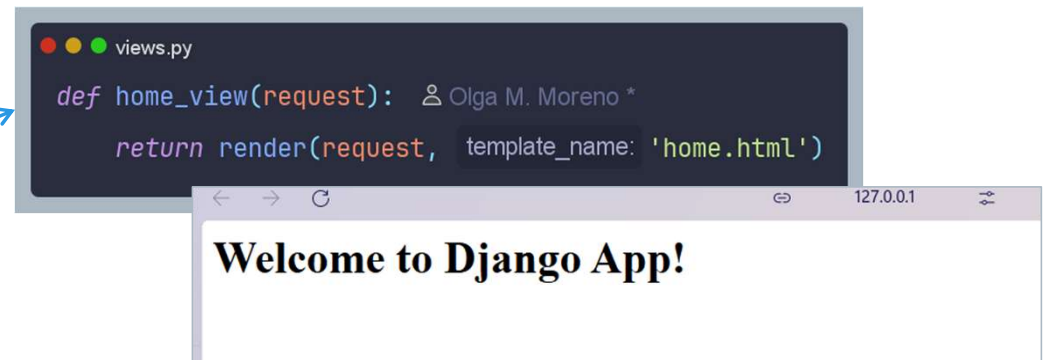
RENDERIZACIÓN DE PLANTILLAS (TEMPLATES)

Las vistas de Django suelen renderizar plantillas HTML para proporcionar una interfaz al usuario. Las plantillas se almacenan en el directorio templates del proyecto.

Renderizar Plantillas desde una Vista

Contexto de Plantilla

Las vistas pueden pasar datos a las plantillas mediante un contexto. Esto permite generar contenido HTML dinámico.

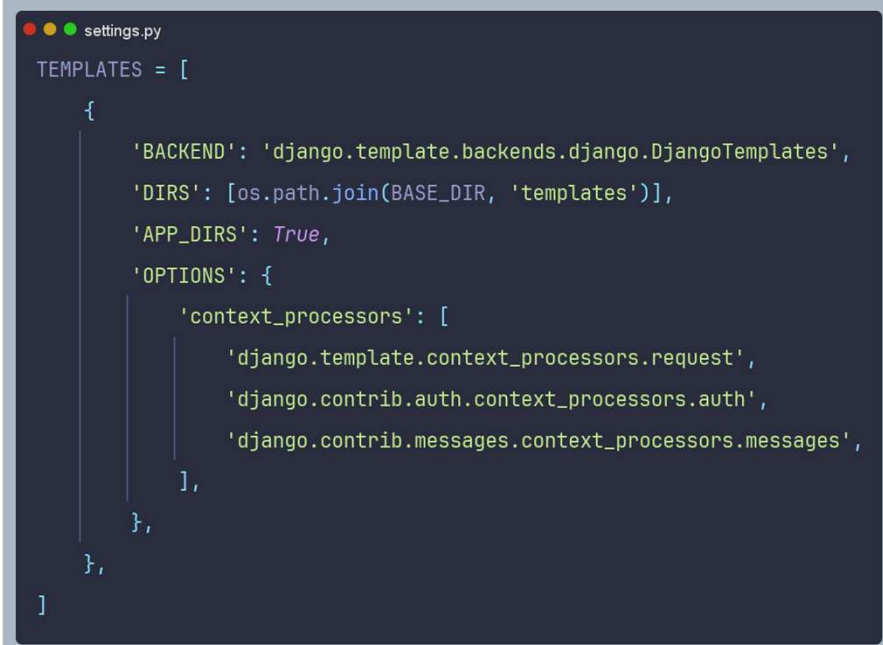


CONFIGURAR CONTEXTO DE PLANTILLA

Para que Django pueda encontrar y renderizar correctamente las plantillas HTML, necesitas configurar la carpeta templates/ en el archivo **settings.py** de tu proyecto.

¿Qué significa esto?

- DIRS: le dice a Django dónde buscar plantillas globales (como home.html). **os.path.join(BASE_DIR, 'templates')**: apunta a la carpeta templates/ en la raíz del proyecto.
- APP_DIRS: True: permite que Django también busque plantillas dentro de cada aplicación (por ejemplo: **myapp/templates/home.html**).



```
settings.py

TEMPLATES = [

    {

        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },

]
```

¿PREGUNTAS?

“Ignorar para preguntar y preguntar para saber, eso es aprender.”

Olga M. Moreno Martín - Javier Herrero Porras

GRACIAS [!*]

Olga M. Moreno Martín - Javier Herrero Porras

